



**Universidad
Tecnológica
del Perú**

FACULTAD DE INGENIERÍA

ESCUELA ACADÉMICA PROFESIONAL DE INGENIERÍA

CURSO

CURSO INTEGRADOR 2

DOCENTE

JONATHAN LOJA YOPLAC

TEMA

AVANCE DE PROYECTO FINAL 3

TÍTULO

**DESARROLLO E IMPLEMENTACIÓN DEL SISTEMA DE GESTIÓN DE PEDIDOS
PARA EL RESTAURANTE DRAGON DORADO**

INTEGRANTES GRUPO 5

**BURGA DÁVILA, ANDERSON DANIEL
GÁLVEZ LEZAMA, LUCILA JACKELINE
PEVES RAMIREZ, JANETH STEFANIE
SALGUERO BELLUZ, ANGEL
YAURI DE LA CRUZ, RENZO RODRIGO**

PERÚ

2026

ÍNDICE

CAPÍTULO I: FUNDAMENTOS DEL PROYECTO	4
1.1. Análisis del Contexto Empresarial	4
1.1.1. Visión	4
1.1.2. Misión	4
1.1.3. Entorno	4
1.2. Diagnóstico del Problema y Oportunidad de Innovación	4
1.2.1. Diagnóstico del Problema	5
1.2.2. Oportunidad de Innovación	5
1.3. Modelo de Negocio (Business Model Canvas)	5
1.4. Alcance de la Solución Informática	6
1.4.1. Alcance Funcional	6
1.4.2. Alcance No Funcional (Atributos de Calidad)	6
1.4.3. Exclusiones	7
1.4.4. Beneficios Esperados	7
CAPÍTULO II: PLANIFICACIÓN ÁGIL (FRAMEWORK SCRUM)	8
2.1. Project Charter y Acta de Constitución	8
2.2. Gestión del Calendario (Diagrama de Gantt)	11
2.3. Organización del Equipo: Roles, Artefactos y Eventos	12
2.3.1. Roles Scrum	12
2.3.2. Artefactos Scrum	12
2.3.3. Eventos Scrum	12
2.4. Product Backlog y Planificación del Sprint 1 y 2	12
CAPÍTULO III: INGENIERÍA DE REQUERIMIENTOS Y DISEÑO UX	14
3.1. Especificación de Requerimientos de Software (SRS)	14
3.1.1. Descripción General	14
3.1.2. Perspectiva del Producto	14
3.1.3. Funciones del Producto	14
3.1.4. Restricciones	14
3.1.5. Suposiciones y Dependencia	14
3.1.6. Requerimientos Generales	15
3.1.7. Requerimientos Funcionales	15
3.1.8. Requerimientos No Funcionales	17
3.2. Prototipado de Alta Fidelidad: Wireframes y Mockups Interactivos	18
3.2.1. Etapas del Proceso de Diseño	18
3.2.2. Pantallas Principales Diseñadas	19
3.2.3. Wireframes	19
3.2.4. Mockups Interactivos	21

3.3. Acuerdos de Niveles de Servicio (SLA) y KPIs del Sistema.....	24
3.3.1. Acuerdos de Nivel de Servicio (SLA)	24
3.3.2. KPIs del Sistema.....	25
CAPÍTULO IV: GESTIÓN DE RIESGOS Y ENTORNO TÉCNICO	27
4.1. Plan de Gestión de Riesgos (Identificación y Mitigación)	27
4.2. Configuración del Stack Tecnológico (IDE Java, Git/GitHub, Cloud)	29
4.2.1. Stack de Desarrollo	29
4.2.2. Configuración del Repositorio GitHub	30
4.2.3. Arquitectura Cloud	30
CAPÍTULO V: DISEÑO Y MODELADO DE LA SOLUCIÓN	31
5.1. Modelado de Procesos de Negocio (BPM)	31
5.2. Diseño de Base de Datos: Modelo Lógico y Físico.....	42
5.3. Diagrama de Clases UML y Documentación Técnica (Markdown).....	44
CAPÍTULO VI: CONSTRUCCIÓN Y SEGURIDAD	53
6.1. Implementación del Back-End (Mínimo 50% Java)	53
6.2. Estrategia de Replicación y Administración de BD	55
6.3. Controles de Seguridad: Autenticación, Cifrado y Defensa Web	55
CAPÍTULO VII: VERIFICACIÓN Y DESPLIEGUE V1	58
7.1. Pruebas Funcionales y No Funcionales (Validación del Servicio).....	58
7.2. Evidencia de Despliegue en Plataforma Cloud (Versión 1)	63
7.3. Retrospectiva del Sprint 4	65
CAPÍTULO VIII: GESTIÓN DE SOLICITUDES Y CAMBIOS.....	67
8.1. Catálogo de Solicitudes de Servicio	67
8.2. Proceso de Gestión de Cambios (infraestructura y Aplicaciones)	69
CAPÍTULO IX: INGENIERÍA DE LA PLATAFORMA	73
9.1. Pipeline de Integración Continua (CI) y Herramientas.....	73
9.2. Documentación de Automatización de Despliegues	74
9.2.1. Contexto Inicial de la Infraestructura	74
9.2.2. Flujo de Automatización Esperado	74
9.2.3. Configuración de Accesos No Interactivos (SSH).....	74
9.2.4. Mitigación de Errores y Diagnóstico Técnico.....	75
9.2.5. Archivo de Workflow Final (deploy-frontend.yml).....	75
9.2.6. Validación del Despliegue.....	76
CAPÍTULO X: GESTIÓN DE CAPACIDAD Y RENDIMIENTO	78
10.1. Análisis de Capacidad de Servicio	78
10.2. Planificación y Herramientas de Monitoreo de Rendimiento	79
10.3. Retrospectiva del Sprint 6	80
CONCLUSIONES	83

CAPÍTULO I: FUNDAMENTOS DEL PROYECTO

1.1. Análisis del Contexto Empresarial

El Restaurante Dragón Dorado es un establecimiento gastronómico del tipo Chifa, especializado en cocina chino-peruana. Opera en un sector de alta rotación de clientes donde la eficiencia operativa representa un factor crítico de competitividad.

1.1.1. Visión

Ser el restaurante Chifa líder en la región, reconocido por la excelencia en el servicio al cliente, la innovación en la gestión operativa y la calidad culinaria, apoyándose en tecnología de vanguardia para garantizar una experiencia gastronómica superior.

1.1.2. Misión

Brindar a nuestros clientes una experiencia gastronómica memorable, combinando sabores tradicionales de la cocina chino-peruana con un servicio ágil, preciso y tecnológicamente respaldado, asegurando la satisfacción total en cada visita, tanto en salón como en servicio a domicilio.

1.1.3. Entorno

El Restaurante Dragón Dorado opera en un entorno altamente competitivo dentro del sector gastronómico de Lima, específicamente en el rubro de Chifas de alta rotación. El análisis se divide en dos dimensiones:

- Entorno Interno: Actualmente, el establecimiento cuenta con una infraestructura física sólida pero una infraestructura tecnológica inexistente. La comunicación depende de la interacción verbal y el uso de papel, lo que genera un ambiente propenso al error humano bajo presión.
- Entorno Externo: El mercado demanda cada vez más rapidez y precisión. La competencia directa (otros Chifas de la zona) ha empezado a adoptar soluciones tecnológicas básicas. Además, el auge del servicio de delivery ha cambiado las expectativas del cliente, quien ahora exige trazabilidad y tiempos de entrega mínimos.

1.2. Diagnóstico del Problema y Oportunidad de Innovación

En el sector gastronómico actual, especialmente en establecimientos de alta rotación como los Chifas, la gestión operativa suele presentar deficiencias críticas debido a la dependencia de procesos manuales como con comandas físicas.

1.2.1. Diagnóstico del Problema

Los problemas detectados que este proyecto busca resolver son:

- **Errores en la toma de pedidos:** Pérdida de información o escritura ilegible que genera retrasos y desperdicio de insumos.
- **Desincronización entre Salón y Cocina:** Falta de comunicación en tiempo real que aumenta los tiempos de espera del cliente.
- **Falta de control en Delivery:** Dificultad para rastrear pedidos externos y asignar repartidores de manera eficiente.
- **Información Financiera Opaca:** Imposibilidad de obtener reportes de ventas, platos más vendidos o flujo de caja de forma inmediata y precisa.

1.2.2. Oportunidad de Innovación

Desarrollar e implementar un sistema integral de gestión para clientes internos que centralice la toma de pedidos, la comunicación con cocina y el control financiero, optimizando la eficiencia operativa y mejorando la experiencia del cliente tanto en salón como en el servicio a domicilio.

1.3. Modelo de Negocio (Business Model Canvas)

Socios clave • Proveedores de Insumos Especializados: Importadores de productos asiáticos (sillao, salsa de osetión, canela china, etc.) y proveedores de vegetales específicos (cebolla china, pac choi). • Proveedores de Materia Prima Fresca: Mercados mayoristas o distribuidores de carnes (pollo, chancho, mariscos) y abarrotes (arroz, aceite). • Empresas de Reparto: Motorizados independientes o las apps de delivery asociadas. • Proveedores de Empaques: Empresas que venden los tapers, cajas y bolsas para el servicio para llevar.	Actividades clave • Operación de Cocina (Producción). • Gestión de Compras e Inventarios. • Atención al Cliente y Despacho: Toma de pedidos sin errores, servicio de mesa eficiente y logístico de empaque para el delivery. • Mantenimiento y Salubridad: Limpieza profunda de trampas de grasa, cocina y salón para cumplir con las normas de salubridad. • Gestión Financiera: Cuadre de caja diario, pago a proveedores. Recursos clave • Humanos: El Maestro Chifera (el sabor depende de él), ayudantes de cocina, mozos, cajero y administrador. • Físicos: Local comercial bien ubicado, cocina industrial especializada, mobiliario del salón, inventario de utensilios e insumos. • Tecnológicos: Para controlar inventarios, emitir comprobantes, POS de pagos y una tablet para ventas en apps de terceros. • Intelectuales: Las recetas estandarizadas, registro de la marca y los permisos de sanidad municipales.	Propuesta de valor • Experiencia Gastronómica Auténtica: Fusión peruano-china de alta calidad, con el sabor tradicional del "wok", usando ingredientes frescos y recetas bien ejecutadas. • Abundancia y Relación Calidad-Precio: Platos de porciones generosas, ideales para compartir, que garantizan que el cliente sienta que recibe mucho valor por su dinero. • Rapidez y Eficiencia en el Servicio: Reducción drástica de los tiempos de espera entre la toma de la orden y la llegada del plato a la mesa (aquí es donde el sistema que están programando aporta valor al negocio, permitiendo que la comanda vuele a la cocina). • Ambiente Acogedor: Un local limpio, temático y cómodo que invita a quedarse y disfrutar en familia.	Relaciones con los clientes • Atención Personalizada en Salón: Trato cálido, atento y rápido por parte de los mozos y el administrador. El cliente debe sentirse reconocido. • Programas de Fidelización: Promociones recurrentes (ej. "Por día hay un descuento en un plato específico" o bebida de cortesía en cumpleaños). • Soporte y Comunicación Digital: Resolución rápida de quejas por pedidos de delivery y respuestas activas a reseñas en redes sociales. Canales • Atención presencial en Salón: El local físico es el canal principal de venta y experiencia. • Canales Digitales Propios: Redes sociales (Facebook, Instagram) para antojarse visualmente, y WhatsApp Business para toma directa de reservas y pedidos. • Plataformas de Delivery (Terceros): Presencia en apps como Rappi o PedidosYa para captar clientes que no quieren salir de casa.	Segmento de clientes • Familias y Grupos Sociales: Familias locales y grupos de amigos que buscan compartir una comida abundante, tradicional y a buen precio, especialmente durante fines de semana y celebraciones. • Trabajadores y Oficinistas: Personal de empresas y negocios cercanos que buscan menús ejecutivos rápidos, económicos y contundentes durante su hora de refrigerio (lunes a sábado). • Usuarios de Delivery / Comida Rápida: Residentes de la zona que prefieren la comodidad de su hogar y solicitan pedidos a domicilio para la cena o fines de semana a través de apps o teléfono.
Estructura de costos • Costos Fijos: Alquiler del local comercial, pago de planillas (sueldos del personal), servicios básicos (gran consumo de gas comercial, luz, agua, internet), mantenimiento del sistema informático y marketing mensual. • Costos Variables: Compra de materia prima e insumos (varía según la venta), comisiones pagadas a las apps de delivery (que suelen cobrar un %, por lo que suben si hay más ventas), compra de empaques para delivery y productos de limpieza. • Gastos Administrativos: Permisos, licencias, contabilidad e impuestos.	Flujo de ingresos • Venta directa de platos, a la carta y bebidas: El ingreso principal generado por los comensales en el salón y pedidos para llevar (take-out). • Venta de Menús: Ingreso de alto volumen y menor margen durante el horario de almuerzo. • Ventas por Delivery: Ingresos generados a través de despachos a domicilio. • Reservas para Eventos: Ingresos adicionales por separar áreas del restaurante para celebraciones grandes o eventos corporativos de fin de año.			

1.4. Alcance de la Solución Informática

1.4.1. Alcance Funcional

A. Módulo de Gestión de Ventas y Salón

- **Mapa de Mesas:** Visualización gráfica del estado de las mesas (disponible, ocupada, reservada, por limpiar).
- **Toma de Pedidos (Comanda Digital):** Registro de pedidos vinculados a una mesa, con capacidad de añadir notas especiales (ej. "sin cebolla", "bien frito").
- **Gestión de Carta/Menú:** Administración dinámica de categorías, platos, precios y disponibilidad de stock en tiempo real.

B. Módulo de Cocina y Producción

- **Monitor de Cocina (KDS):** Interfaz para los cocineros donde visualizan los pedidos por orden de llegada y prioridad.
- **Cambio de Estados:** Los cocineros podrán marcar pedidos como "En preparación" o "Listo para servir", notificando automáticamente al personal de salón.

C. Módulo de Delivery y Despacho

- **Gestión de Pedidos Externos:** Registro de datos del cliente (nombre, dirección, teléfono) y asignación de motorizados.
- **Control de Estados de Envío:** Seguimiento desde que el pedido sale del local hasta que es entregado.

D. Módulo de Facturación y Finanzas

- **Cierre de Cuenta:** Emisión de pre-comprobante de pago y solicitud de comprobante.
- **Apertura y Cierre de Caja:** Registro de ingresos y egresos diarios para evitar descuadres.
- **Reportes Gerenciales:** Generación de estadísticas de ventas diarias, semanales y mensuales, así como ranking de platos más rentables.

E. Módulo de Administración y Seguridad

- **Gestión de Usuarios:** Control de accesos mediante roles (Administrador, Mozo, Cocinero, Cajero).
- **Seguridad de Datos:** Encriptación de contraseñas y backups periódicos de la base de datos.

1.4.2. Alcance No Funcional (Atributos de Calidad)

- **Usabilidad:** Interfaz intuitiva y adaptada a dispositivos móviles

(tablets/celulares) para los administradores.

- **Disponibilidad:** El sistema debe operar el 99.9% del tiempo durante el horario comercial.
- **Rendimiento:** El tiempo de respuesta entre el envío de la comanda y la recepción en cocina no debe superar los 2 segundos.
- **Escalabilidad:** Arquitectura preparada para añadir más mesas o sucursales en el futuro.

1.4.3. Exclusiones

- No se encargará de la compra de hardware (tabletas, impresoras térmicas).
- No incluye la gestión de nómina o pago de planillas de empleados.
- No incluye marketing digital ni gestión de redes sociales del restaurante.
- No se realizará integración directa con aplicaciones de terceros (de delivery y/o facturación), en esta primera fase, se registrarán manualmente.

1.4.4. Beneficios Esperados

1. Eliminación total de errores por legibilidad de comandas.
2. Mayor transparencia en el manejo de efectivo y reportes de rentabilidad.
3. Control efectivo de las ventas.

CAPÍTULO II: PLANIFICACIÓN ÁGIL (FRAMEWORK SCRUM)

2.1. Project Charter y Acta de Constitución

Título	Desarrollo e implementación del sistema de gestión de pedidos para el			Jefe de Proyecto	Janeth Stefanie Peves Ramirez
Fecha inicio	30/03/2026	Fecha fin	14/06/2026	Sponsor	Dragón Dorado
Necesidad del negocio					
Optimizar la operatividad de un restaurante de alta rotación eliminando procesos manuales que causan errores en pedidos, retrasos en comunicación y falta de control financiero.					
Alcance				Entregables	
Módulo de Gestión de Ventas y Salón				1. Planificación, Negocio y UX	
Módulo de Cocina y Producción				2. Arquitectura, Back-End y Despliegue Inicial	
Módulo de Delivery y Despacho				3. Gestión de Servicios y Rendimiento	
Módulo de Facturación, Finanzas, Administración y				4. Operación, Continuidad y Cierre del Proyecto.	
Riesgos y problemas				Suposiciones y dependencias	
Resistencia al cambio del personal, fallos técnicos en la base de datos y vulnerabilidades de seguridad de acceso.				El personal brindará retroalimentación oportuna, se mantendrá el cronograma y habrá una red Wi-Fi estable en el local. Las dependencias serían con el fefe de meseros, administrador y jefe de cocina.	
Recursos					
Computadoras, IDEs de desarrollo, información operativa del restaurante y el tiempo del equipo de ingeniería					

COMPONENTE	DESCRIPCIÓN
Nombre del Proyecto	Desarrollo e implementación del sistema de gestión de pedidos para el restaurante Dragón Dorado.
Equipo del Proyecto	- Líder del Proyecto: Janeth Stefanie Peves Ramirez. Responsable de la planificación general, coordinación de tareas, seguimiento del avance y presentación final del proyecto. - Responsable del Sistema: Anderson Daniel Burga Dávila. Encargado del diseño y desarrollo del sistema, así como de las pruebas técnicas y funcionamiento general del software. - Operación General: Renzo Rodrigo Yauri de la Cruz, Lucila Jackeline Gálvez Lezama y Ángel Salguero Belluz. Encargados de la documentación, pruebas, levantamiento de información y tareas requeridas por los otros responsables.
Descripción del Proyecto	El proyecto consiste en el desarrollo de un sistema integral de gestión diseñado para optimizar la operatividad de un restaurante tipo Chifa de alta rotación. La solución centraliza la toma de pedidos mediante comandas digitales, facilita la comunicación en tiempo real con la cocina y permite un

	control financiero preciso de las ventas y el flujo de caja.
Justificación del Proyecto	Actualmente, el restaurante presenta deficiencias críticas por el uso de procesos manuales, tales como errores en la toma de pedidos por letra ilegible, retrasos en la comunicación con cocina y falta de trazabilidad en el servicio de delivery. El sistema busca eliminar estos cuellos de botella y brindar transparencia a la información financiera.
Objetivos del Proyecto	- Implementar un sistema que centralice la toma de pedidos y la gestión de ventas. - Optimizar la comunicación entre el salón y la cocina para reducir tiempos de espera. - Automatizar el control de inventario de platos y la gestión de estados de delivery. - Generar reportes gerenciales detallados sobre rentabilidad y flujo de caja.
Alcance del Proyecto	Incluye: - Módulo de Ventas y Salón (Mapa de mesas y comanda digital). - Módulo de Cocina (Monitor KDS y gestión de estados). - Módulo de Delivery (Asignación de repartidores y seguimiento). - Módulo de Finanzas (Caja chica, facturación y reportes). - Módulo de Administración (Gestión de roles y seguridad). No Incluye: - Compra de hardware (tablets o impresoras). - Gestión de planillas o nómina de empleados. - Marketing digital o redes sociales. - Integración automática con apps de terceros (registro manual).
Entregables Principales	- Documento de análisis de requerimientos (SRS). - Prototipos de interfaz de usuario (UX/Wireframes). - Sistema funcional desplegado en plataforma cloud. - Manuales de usuario y técnicos. - Informe final y presentación del proyecto.

Recursos del Proyecto	- Computadoras y herramientas de desarrollo (IDE, gestores de base de datos). - Información operativa proporcionada por el restaurante Dragón Dorado. - Tiempo y dedicación del equipo de ingeniería.
Supuestos y Riesgos	Supuestos: - El personal del restaurante brindará retroalimentación oportuna sobre los procesos actuales. - Se mantendrá el cronograma de trabajo según las fases de planificación y desarrollo. Riesgos: - Cambios en los requerimientos funcionales durante el desarrollo. - Problemas técnicos con la arquitectura de base de datos o conectividad. - Dificultades en la coordinación de tareas entre los miembros del equipo.

Aprobación del Project Charter

Nombre	Rol	Firma
Janeth Stefanie Peves Ramirez	Líder del Proyecto	
Anderson Daniel Burga Dávila	Responsable del Sistema	
Renzo Rodrigo Yauri de la Cruz	Operación General	
Lucila Jackeline Gálvez Lezama	Operación General	
Ángel Salguero Belluz	Operación General	

2.2. Gestión del Calendario (Diagrama de Gantt)

DESARROLLO E IMPLEMENTACIÓN DEL SISTEMA DE GESTIÓN DE PEDIDOS PARA EL RESTAURANTE DRAGON DORADO

DRAGÓN DORADO
PEVES RAMÍREZ, JANETH STEFANIE

Fecha de Inicio del proyecto:

30/03/2026

Incremento de desplazamiento:

50

Leyenda:

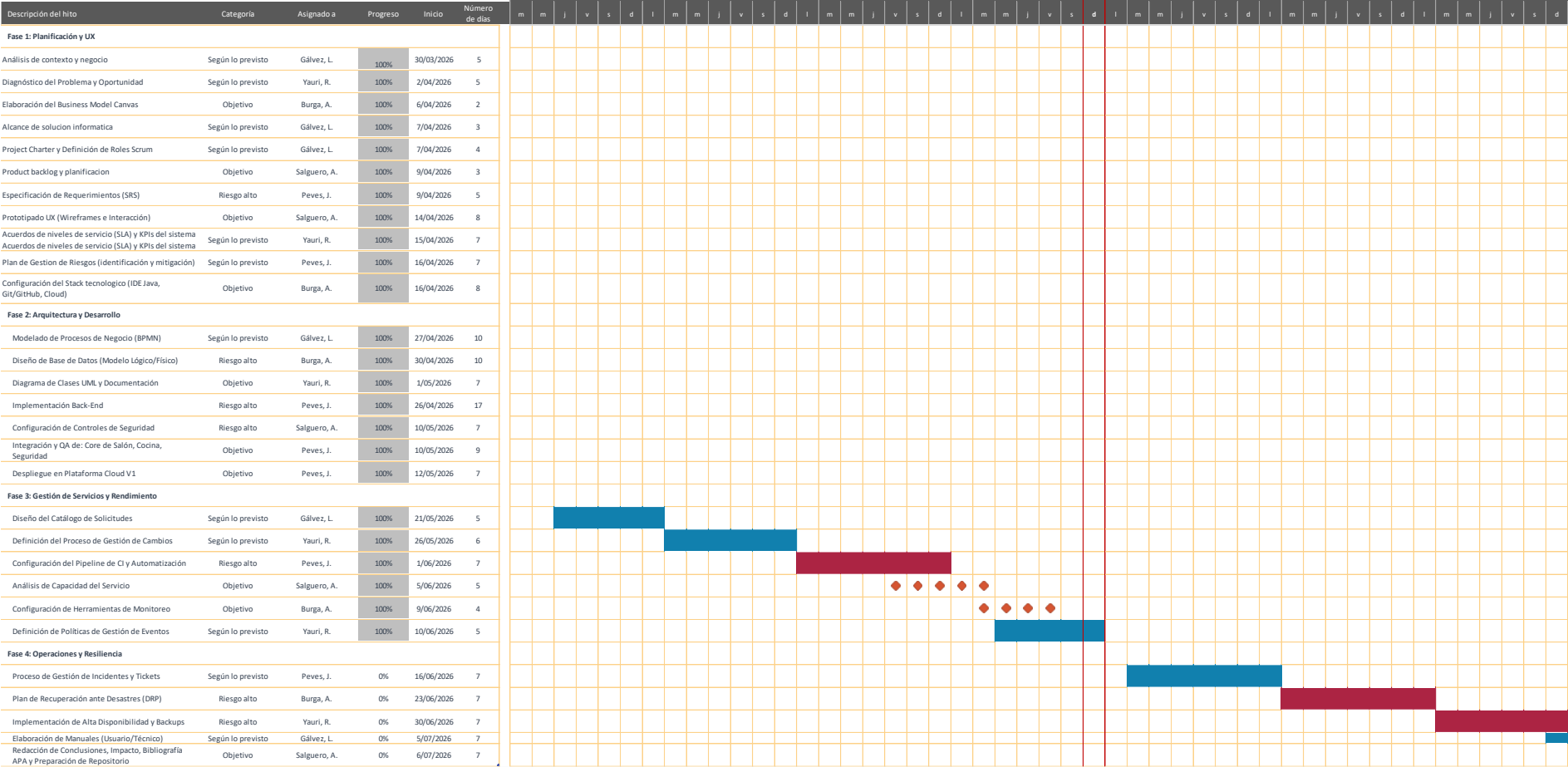
Según lo previsto

Riesgo bajo

Riesgo medio

Riesgo alto

Sin asignar



2.3. Organización del Equipo: Roles, Artefactos y Eventos

2.3.1. Roles Scrum

Rol	Integrante	Responsabilidades Principales
Product Owner	Gálvez Lezama, Lucila J.	Priorizar el backlog, representar al cliente, validar entregables
Scrum Master	Peves Ramirez, Janeth S.	Facilitar ceremonias Scrum, eliminar impedimentos, asegurar cumplimiento del marco ágil
Dev. Backend	Burga Dávila, Anderson D.	Implementación de lógica de negocio, APIs y base de datos
Dev. Frontend/UX	Salguero Belluz, Angel	Prototipado, wireframes, diseño de interfaz y experiencia de usuario
Dev. QA/Cloud	Yauri de la Cruz, Renzo R.	Pruebas, despliegue en nube, documentación técnica

2.3.2. Artefactos Scrum

- Product Backlog: Lista priorizada de todas las funcionalidades del sistema.
- Sprint Backlog: Subconjunto de ítems del Product Backlog seleccionados para cada sprint.
- Incremento: Versión funcional y testeada del sistema al final de cada sprint.

2.3.3. Eventos Scrum

- Sprint Planning: Al inicio de cada sprint para definir objetivos y tareas.
- Daily Scrum: Reunión diaria de 15 minutos para sincronizar el equipo.
- Sprint Review: Al final del sprint para demostrar el incremento al Product Owner.
- Sprint Retrospective: Revisión del proceso para mejora continua.

2.4. Product Backlog y Planificación del Sprint 1 y 2

A continuación, se presenta el Product Backlog priorizado con las historias de usuario más representativas y su asignación a los primeros dos sprints:

ID	Historia de Usuario	Prioridad	Puntos	Sprint	Módulo
US-01	Como mozo, quiero registrar pedidos digitalmente vinculados a una mesa para evitar errores de legibilidad.	Alta	8	Sprint 1	Ventas / Salón
US-02	Como cocinero, quiero visualizar los pedidos en el KDS para prepararlos en orden de prioridad.	Alta	8	Sprint 1	Cocina / KDS
US-03	Como mozo, quiero ver el mapa de mesas con su estado actual para asignar correctamente a los clientes.	Alta	5	Sprint 1	Ventas / Salón
US-04	Como cajero, quiero que los mozos puedan emitir el pre-comprobantes de pago por si hay correcciones y para que al confirmar el cobro pueda cuadrar correctamente las cuentas desde el sistema.	Alta	8	Sprint 2	Facturación
US-05	Como administrador, quiero generar reportes de ventas diarias y ranking de platos.	Media	5	Sprint 2	Finanzas
US-06	Como despachador, quiero registrar pedidos de delivery y asignar repartidores.	Media	5	Sprint 2	Delivery
US-07	Como administrador, quiero gestionar usuarios con roles diferenciados para controlar accesos.	Alta	3	Sprint 1	Adminis- tración
US-08	Como gerente, quiero ver el cierre del día con registro de ingresos y egresos del día.	Alta	5	Sprint 2	Finanzas

CAPÍTULO III: INGENIERÍA DE REQUERIMIENTOS Y DISEÑO UX

3.1. Especificación de Requerimientos de Software (SRS)

3.1.1. Descripción General

El sistema surge para solucionar problemas críticos como errores en la toma de pedidos, desincronización entre salón y cocina, y falta de control financiero en procesos manuales.

3.1.2. Perspectiva del Producto

El software funcionará como un sistema independiente con arquitectura preparada para la escalabilidad. Se enfocará en la movilidad, permitiendo que el personal de salón utilice tablets o celulares para interactuar con un servidor central que comunica los datos a la cocina y caja en tiempo real.

3.1.3. Funciones del Producto

- **Gestión de Salón:** Visualización del estado de mesas y toma de pedidos con notas especiales.
- **Gestión de Cocina:** Recepción de pedidos por prioridad y actualización de estados ("En preparación" / "Listo").
- **Gestión de Delivery:** Registro de clientes externos y seguimiento de despachos.
- **Gestión Financiera:** Apertura/cierre de caja y generación de reportes de rentabilidad.
- **Administración:** Gestión de menús, precios y perfiles de usuario.

3.1.4. Restricciones

- No incluye la compra de hardware (impresoras, tablets).
- No se integrará automáticamente con apps de terceros (Rappi, PedidosYa) en esta fase.

3.1.5. Suposiciones y Dependencia

- Se asume que el restaurante contará con una red Wi-Fi estable para la comunicación entre dispositivos.
- El personal operativo recibirá una capacitación básica para el uso de la interfaz táctil.

3.1.6. Requerimientos Generales

- Página de Inicio: El sistema debe ofrecer una pantalla de Login con campos usuario y password, y un botón “Iniciar sesión”.
- Interfaz de Usuario (UI): Diseño intuitivo, responsivo y adaptado a dispositivos móviles para los administradores y mozos.
- Interfaz de Cocina: Pantalla de alto contraste con botones grandes para facilitar la interacción del personal de cocina.

3.1.7. Requerimientos Funcionales

Código del Requerimiento RF01	
Nombre	Login
Descripción	El sistema debe permitir iniciar sesión enviando usuario y password al backend y recibir un token junto con los datos del usuario.
Prioridad	Alta

Código del Requerimiento RF02	
Nombre	Verificación de sesión
Descripción	El sistema debe verificar la validez del token consultando el backend y reconstruir el estado de sesión si corresponde.
Prioridad	Alta

Código del Requerimiento RF03	
Nombre	Refresh automático de token
Descripción	Ante expiración o token inválido (HTTP 401), el sistema debe intentar refrescar el token una única vez y reintentar la solicitud.
Prioridad	Alta

Código del Requerimiento RF04	
Nombre	Logout

Descripción	El sistema debe limpiar la caché persistida del cliente, cerrar sesión y redirigir a /login.
Prioridad	Alta

Código del Requerimiento RF05	
Nombre	Control de Accesos
Descripción	El sistema debe restringir funciones mediante roles (Administrador, Mozo, Cocinero, Cajero).
Prioridad	Alta

Código del Requerimiento RF06	
Nombre	Mapa de Mesas
Descripción	El sistema debe mostrar gráficamente el estado de las mesas (disponible, ocupada, reservada, por limpiar).
Prioridad	Alta

Código del Requerimiento RF07	
Nombre	Comanda Digital
Descripción	El sistema debe permitir el registro de pedidos vinculados a una mesa con capacidad de añadir notas especiales.
Prioridad	Alta

Código del Requerimiento RF08	
Nombre	Monitor de Cocina (KDS)
Descripción	El sistema debe mostrar una interfaz para cocineros con pedidos organizados por orden de llegada y prioridad.
Prioridad	Alta

Código del Requerimiento RF09	
Nombre	Cambio de Estados
Descripción	El sistema debe permitir marcar pedidos como "En preparación" o "Listo", notificando al personal de salón.
Prioridad	Alta

Código del Requerimiento RF10	
Nombre	Gestión de Delivery
Descripción	El sistema debe registrar datos del cliente externo (nombre, dirección, teléfono) y asignar motorizados.
Prioridad	Media

Código del Requerimiento RF11	
Nombre	Cierre de Caja
Descripción	El sistema debe permitir el registro de ingresos y egresos diarios para evitar descuadres financieros.
Prioridad	Alta

3.1.8. Requerimientos No Funcionales

Código del Requerimiento RNF01	
Nombre	Usabilidad
Descripción	La interfaz debe ser intuitiva y adaptada a dispositivos móviles para facilitar el uso operativo.
Prioridad	Alta

Código del Requerimiento RNF02	
Nombre	Disponibilidad

Descripción	El sistema debe operar el 99.9% del tiempo durante el horario comercial del restaurante.
Prioridad	Alta

Código del Requerimiento RNF03	
Nombre	Rendimiento
Descripción	El sistema debe usar paginación, lazy loading y caching para optimizar el rendimiento.
Prioridad	Alta

Código del Requerimiento RNF04	
Nombre	Seguridad
Descripción	El sistema debe encriptar contraseñas y realizar backups periódicos de la base de datos.
Prioridad	Alta

Código del Requerimiento RNF05	
Nombre	Escalabilidad
Descripción	La arquitectura debe estar preparada para añadir más mesas o sucursales en el futuro.
Prioridad	Media

3.2. Prototipado de Alta Fidelidad: Wireframes y Mockups Interactivos

El diseño UX del sistema sigue los principios de usabilidad de Nielsen y se orienta a usuarios con nivel básico de experiencia tecnológica (mozos, cocineros, cajeros). El proceso de prototipado contempla las siguientes etapas:

3.2.1. Etapas del Proceso de Diseño

1. Investigación de Usuarios: Entrevistas con el personal del Dragon Dorado para identificar flujos de trabajo actuales, puntos de dolor y expectativas del sistema.

2. Arquitectura de Información: Definición del árbol de navegación y flujos de usuario para cada rol (administrador, mozo, cocinero, cajero).
3. Wireframes de Baja Fidelidad: Bocetos esquemáticos de las pantallas principales para validar estructura y flujo antes del detalle visual.
4. Mockups de Alta Fidelidad: Diseños visuales completos con colores, tipografía e iconografía definitiva, elaborados en Figma.
5. Prototipo Interactivo: Simulación navegable del flujo completo para validación con usuarios finales antes del desarrollo.

3.2.2. Pantallas Principales Diseñadas

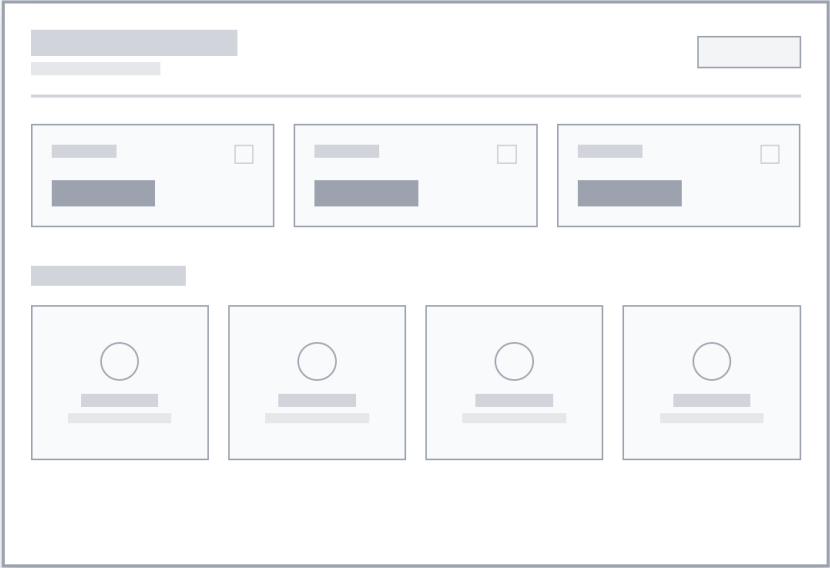
- Pantalla de Login: Autenticación por usuario y contraseña.
- Dashboard Principal: Vista general con accesos rápidos a módulos según rol.
- Mapa de Mesas: Vista gráfica interactiva del salón con estados codificados por color.
- Toma de Pedidos: Pantalla de comanda digital con menú categorizado y campo de notas.
- Monitor KDS: Interfaz de cocina con tarjetas de pedidos ordenadas por prioridad.
- Panel de Delivery: Formulario de pedido externo con asignación de repartidor.
- Cierre de Caja: Resumen de transacciones del día con cuadre automático.
- Reportes: Dashboard gerencial con gráficos de ventas y ranking de platos.

3.2.3. Wireframes

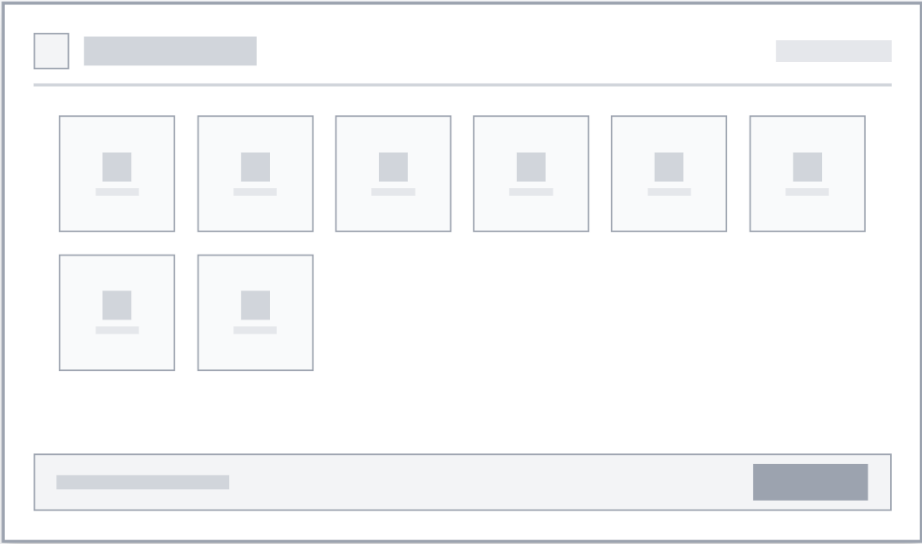
En esta etapa se desarrollaron los esquemas visuales de baja fidelidad para definir la estructura de la información y el comportamiento de la interfaz sin distracciones estéticas. Se priorizó la organización de los elementos clave, como la división de pantalla en el módulo de acceso (Sidebar corporativo vs. Formulario de autenticación) y la distribución del espacio táctil para las futuras interfaces de los mozos y cocineros.



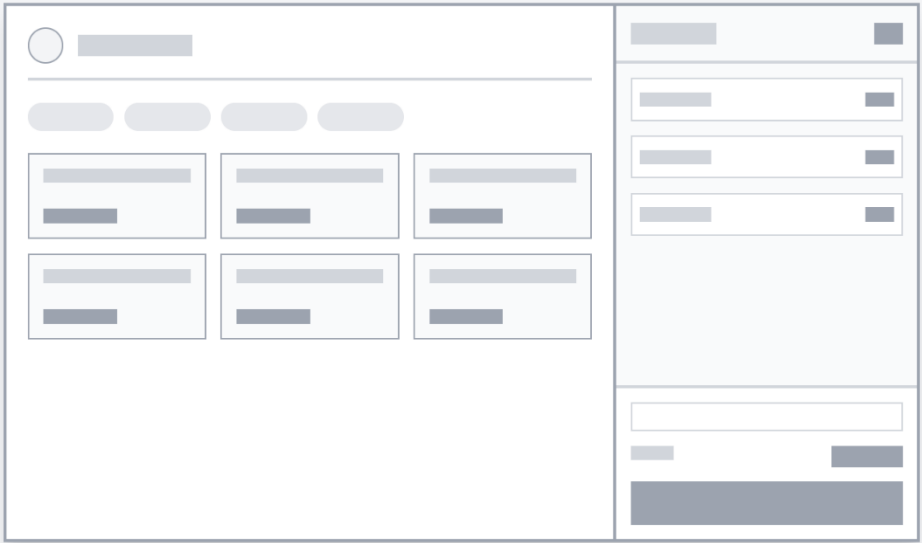
Wireframe: Dashboard Principal



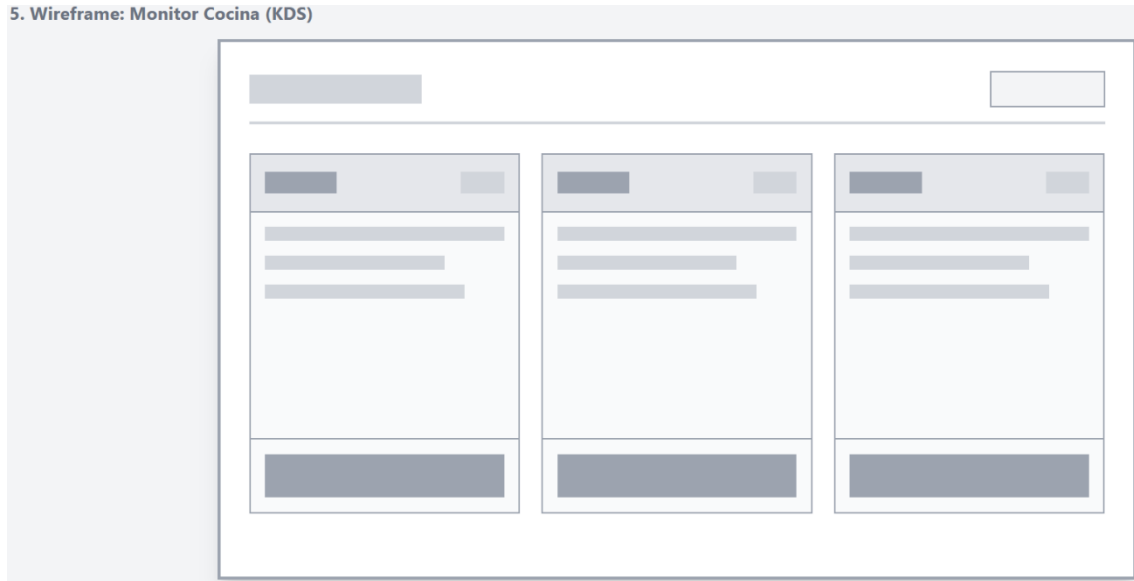
3. Wireframe: Mapa de Mesas



4. Wireframe: Comanda Digital



5. Wireframe: Monitor Cocina (KDS)



3.2.4. Mockups Interactivos

A partir de los wireframes aprobados, se construyeron los prototipos de alta fidelidad aplicando la identidad corporativa del restaurante Dragón Dorado. Se estableció una paleta de colores basada en fondos oscuros (Cyber-Minimalista / #1a1a1a) para reducir la fatiga visual del personal, contrastada con acentos en rojo institucional y dorado (#d4af37) para resaltar las acciones principales.

A continuación, se presentan las interfaces principales correspondientes al Sprint 1:

1. Módulo de Seguridad: Pantalla de Login (RF01) Diseñada con un enfoque "Mobile-First" y responsive, garantizando que la tarjeta de autenticación se adapte tanto a las pantallas de escritorio de caja como a los dispositivos móviles del personal.



DRAGON DORADO

Sistema de Gestión de Pedidos

v1.0 - 2026

Iniciar Sesión

Usuario / Correo

Ej: admin_chifa

Contraseña

INGRESAR AL SISTEMA

2. Módulo de Administración: Dashboard Principal (US-07) Vista general diseñada para roles administrativos o de caja. Presenta un resumen de métricas clave (KPIs operativos) y funciona como el centro de navegación (Hub) con accesos rápidos hacia los demás módulos del sistema, manteniendo la estética Cyber-Minimalista.

PANEL DE CONTROL
SESIÓN INICIADA: ADMIN_CHIFA

CERRAR SESIÓN

VENTAS DEL DÍA

S/ 1,450.50

MESAS ACTIVAS

05 / 08

PEDIDOS CRÍTICOS

03

• NAVEGACIÓN POR MÓDULOS

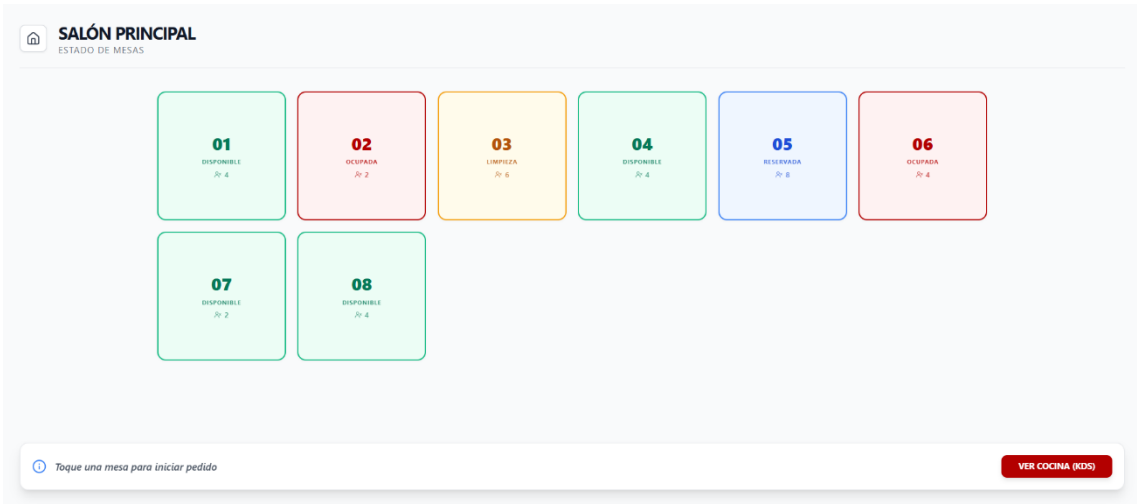
**Gestión de Salón**
Mapa de mesas y comandas.
INGRESAR →

**Monitor de Cocina**
Control de pedidos (KDS).
INGRESAR →

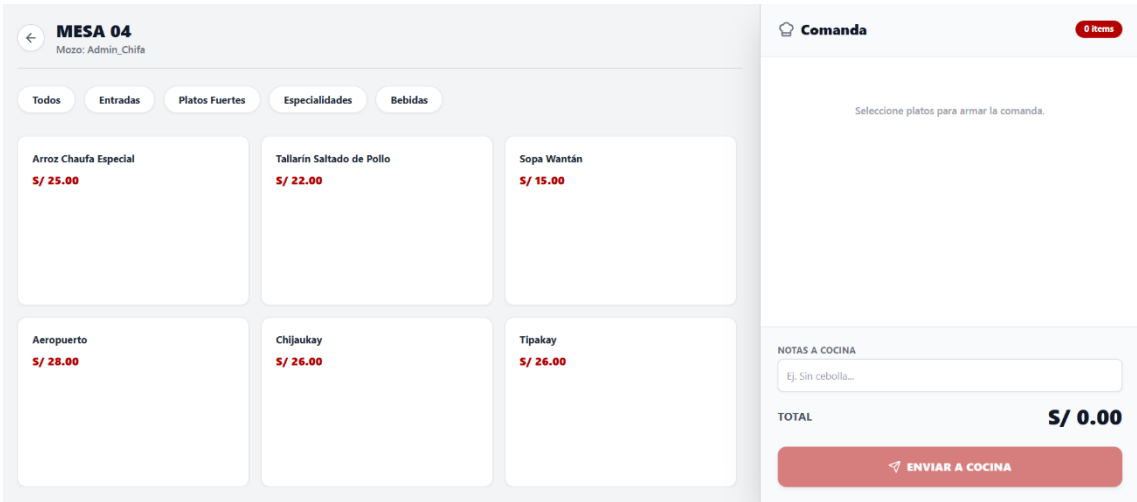
**Delivery**
Seguimiento de pedidos.
INGRESAR →

**Finanzas y Reportes**
Cierre de caja y rentabilidad.
INGRESAR →

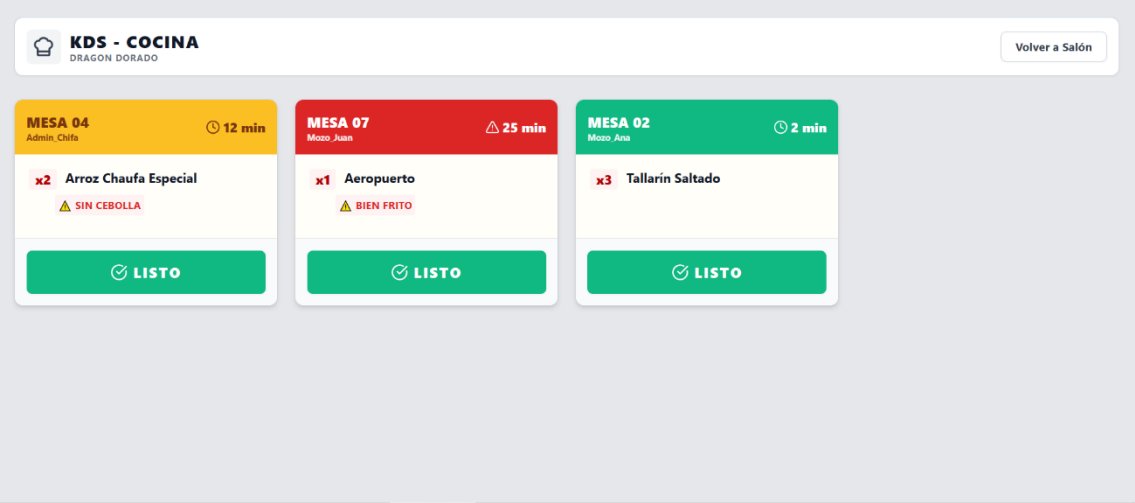
3. Módulo de Ventas: Mapa de Mesas (US-03 / RF06) Vista general del salón con indicadores de colores semánticos (verde: libre, rojo: ocupada, ámbar: limpieza) que permite al mozo identificar la disponibilidad en tiempo real.



4. Módulo de Ventas: Comanda Digital (US-01 / RF07) Interfaz táctil para la toma de pedidos. Incluye el menú categorizado mediante filtros rápidos y el panel de resumen con la opción vital de agregar "Notas a Cocina" para personalizar los platos.



5. Módulo de Cocina: Monitor KDS (US-02 / RF08) Pantalla de alto contraste diseñada específicamente para el entorno bajo presión de la cocina. Muestra tarjetas de pedidos ordenadas por prioridad de tiempo y botones de gran tamaño para cambiar el estado a "Listo para servir".



3.3. Acuerdos de Niveles de Servicio (SLA) y KPIs del Sistema

Los Acuerdos de Nivel de Servicio definen los compromisos de calidad del sistema frente al cliente interno (el restaurante). Los KPIs permiten medir el cumplimiento de dichos acuerdos.

3.3.1. Acuerdos de Nivel de Servicio (SLA)

Servicio	Métrica	Objetivo	Prioridad
Disponibilidad del Sistema	Uptime durante horario comercial	≥ 99.9%	Crítica
Tiempo de Respuesta Comanda	Latencia de envío a cocina	≤ 2 segundos	Crítica
Tiempo de Generación de Reportes	Carga de dashboard gerencial	≤ 5 segundos	Alta
Tiempo de Resolución de Incidentes	Respuesta ante falla crítica	≤ 4 horas	Alta
Backup de Datos	Frecuencia de respaldo	Cada 24 horas	Media
Tiempo de Recuperación (RTO)	Restauración ante desastre	≤ 8 horas	Alta

3.3.2. KPIs del Sistema

KPIs Operativos - Pedidos y cocina				
ID	Indicador	Descripción y método de medición	Meta	Línea base
KPI-01	Tasa de error en pedidos	Pedidos con datos incorrectos o incompletos / total pedidos del día. Medición: correcciones manuales registradas en el sistema.	< 0.5% al mes 3	~15% con comandas físicas
KPI-02	Tiempo promedio de atención	Desde que el cliente ordena hasta que el plato llega a mesa. Medición: timestamp comanda enviada vs. estado Listo para servir en el sistema.	<= 12 min (hora punta)	~18 min estimado actual
KPI-03	Tiempo de despacho delivery	Desde registro del pedido delivery hasta que el motorizado sale del local Medición: timestamp registro vs. estado Despachado en módulo delivery.	<= 20 min	Sin referencia digital previa

KPIs Financieros y administrativos				
ID	Indicador	Descripción y método de medición	Meta	Línea base
KPI-04	Tiempo de cierre de caja	Tiempo para realizar el cuadre diario desde apertura del proceso en el sistema. Medición: registro de inicio y fin del módulo de caja.	<= 10 min	25 min proceso manual

KPIs de Adopción y satisfacción				
ID	Indicador	Descripción y método de medición	Meta	Línea base
KPI-05	Satisfacción del usuario interno	Puntuación de mozos, cocineros y cajeros en encuesta post-implementación (escala 1-5). Frecuencia: mensual, primeros 3 meses	>= 4.2 / 5	Sin referencia previa

KPI-06	Tasa de adopción del sistema	Porcentaje del personal que usa el sistema activamente (≥ 1 acción por turno). Medición: log de sesiones activas por usuario.	80% sem1 - 100% sem3	0% - proceso manual
--------	------------------------------	--	-------------------------------	---------------------

CAPÍTULO IV: GESTIÓN DE RIESGOS Y ENTORNO TÉCNICO

4.1. Plan de Gestión de Riesgos (Identificación y Mitigación)

El Plan de Gestión de Riesgos identifica las amenazas potenciales al proyecto, evalúa su probabilidad e impacto y define estrategias de mitigación y planes de contingencia para garantizar la continuidad del desarrollo.

	MATRIZ DE RIESGO					
	RIESGO	PROBABILIDAD	IMPACTO	NIVEL DE RIESGO	MEDIDAS DE MITIGACIÓN	
INFRA-ESTRUCTURA	Caída del servidor donde corre el backend o frontend	BAJO	ALTO	ALTO	Configurar monitoreo de disponibilidad con alertas automáticas al equipo técnico ante caídas del servicio.	Reiniciar los contenedores y verificar la conexión con la base de datos antes de reanudar operaciones.
INFRA-ESTRUCTURA	Fallo o corrupción de la base de datos PostgreSQL, impidiendo el registro de pedidos, pagos y operaciones de caja.	BAJO	ALTO	ALTO	Configurar backups automáticos diarios de la base de datos en la nube con retención mínima de 7 días.	Restaurar la base de datos desde el ultimo backup disponible y verificar la integridad de los datos antes de reanudar.
TÉCNICO	Corte de la conexión WebSocket entre el salón y el monitor de cocina (KDS) interrumpiendo las notificaciones de pedidos en tiempo real.	MEDIO	ALTO	MEDIO	Realizar pruebas unitarias de estabilidad de la conexión en el entorno de producción previo al despliegue.	El personal de cocina puede consultar los pedidos pendientes directamente desde la interfaz web mientras se restablece la conexión.

INTEGRACIÓN	Indisponibilidad de Cloudinary impide subir imágenes de productos, bloqueando la creación o actualización de platos del menú.	BAJO	MEDIO	MEDIO	Gestionar las actualizaciones del menú fuera de los horarios de alta demanda del restaurante.	Pospones la carga de imágenes hasta que el servicio se restablezca. Los platos existentes siguen operativos sin interrupción.
GESTIÓN	Descoordinación entre los miembros del equipo que genere trabajo duplicado o entregables incompletos al final del sprint.	MEDIO	MEDIO	MEDIO	Mantener reuniones de seguimiento periódicas y usar el tablero de tareas del proyecto para que todos conozcan el estado de cada módulo.	Redistribuir las tareas pendientes entre los integrantes disponibles y ajustas las fechas de entrega internas con el Scrum Master.
DESARROLLO	Retraso en la implementación de módulos clave (cocina, delivery) por dificultades técnicas no previstas en la estimación del sprint.	MEDIO	ALTO	ALTO	Estimas las tareas con margen de tiempo adicional e identificar los puntos de mayor complejidad técnica antes de comenzar el sprint.	Priorizar las funcionales esenciales del módulo afectado y dejas las secundarias para un sprint posterior, informando al Product Owner.
DESPLIEGUE	Error durante el despliegue del sistema en el entorno cloud (AWS) que deje la aplicación inoperativa o con datos inconsistentes.	MEDIO	ALTO	ALTO	Realizar el despliegue primero en u entorno de pruebas y validar el correcto funcionamiento de todos los módulos antes	Revertir a la versión anterior estable del sistema y analizar los errores del despliegue antes de intentarlo

					de pasar a producción.	nuevamente.
SEGURIDAD	Robo del refresh token de un usuario, permitiendo al atacante obtener nuevos access tokens y suplantar su sesión de forma prolongada	MEDIO	ALTO	ALTO	Transmitir siempre el refresh token sobre HTTPS y configurar un tiempo de expiración corto.	Eliminar el registro del refresh token en la tabla refresh_tokens de la base de datos, forzando al usuario afectado a iniciar sesión nuevamente.

4.2. Configuración del Stack Tecnológico (IDE Java, Git/GitHub, Cloud)

El entorno técnico del proyecto está configurado para soportar el desarrollo colaborativo, el despliegue en nube y la integración continua del sistema.

4.2.1. Stack de Desarrollo

Capa	Tecnología	Versión / Herramienta	Justificación
Backend	Java / Spring Boot	JDK 21 / Spring Boot 3.x	Robusto, escalable, amplio soporte empresarial.
Frontend	HTML5, CSS3, JavaScript	React.js 18 / Tailwind CSS	SPA reactiva, ideal para interfaces dinámicas en tablet.
Base de Datos	PostgreSQL	v15	Relacional, open-source, excelente rendimiento.
IDE	IntelliJ IDEA / VS Code	Community / v1.8x	Soporte completo para Java y desarrollo web.
Control de Versiones	Git / GitHub	Git 2.x	Gestión colaborativa del código fuente.
Cloud	VPS contabo	Nginx	Alta disponibilidad, escalabilidad y backups

			automáticos.
Contenedores	Docker	Docker 24.x	Portabilidad y consistencia entre entornos.
Testing	Postman	Postman v12	Pruebas unitarias, de integración y de APIs.

4.2.2. Configuración del Repositorio GitHub

- Estructura de Ramas: Se utilizará una rama principal denominada main para centralizar el código fuente, la documentación y las versiones estables del sistema.
- Gestión de Cambios: Las actualizaciones se realizarán mediante commits directos a la rama principal para facilitar la integración rápida de los avances de cada integrante.
- Sincronización de Entornos: Se utilizarán archivos de configuración (properties) para diferenciar los parámetros de ejecución entre el entorno de desarrollo local y el entorno de despliegue.
- Control de Versiones: Se mantendrá un historial cronológico de cambios que permita la trazabilidad de las funcionalidades implementadas en cada sesión de trabajo.
- Despliegue: La carga de nuevas versiones a la nube se realizará de forma manual tras validar el funcionamiento local de los módulos correspondientes a cada sprint.

4.2.3. Arquitectura Cloud

La arquitectura de despliegue en nube contempla los siguientes componentes:

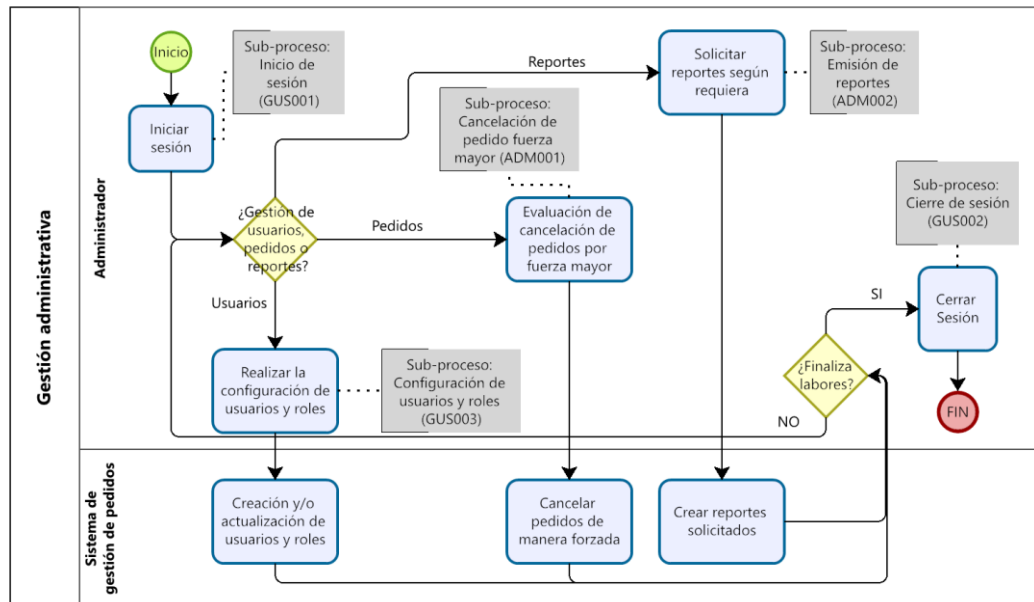
- Servidor de Aplicaciones: Instancia EC2 (o equivalente) con Docker para alojar el backend Spring Boot.
- Base de Datos Gestionada: Servicio RDS (PostgreSQL) con réplica de lectura para alta disponibilidad.
- Almacenamiento de Archivos: Bucket S3 para respaldos de base de datos y assets estáticos.
- Monitoreo: Configuración de alertas de CPU, memoria y disponibilidad con notificación al equipo técnico.

CAPÍTULO V: DISEÑO Y MODELADO DE LA SOLUCIÓN

5.1. Modelado de Procesos de Negocio (BPM)

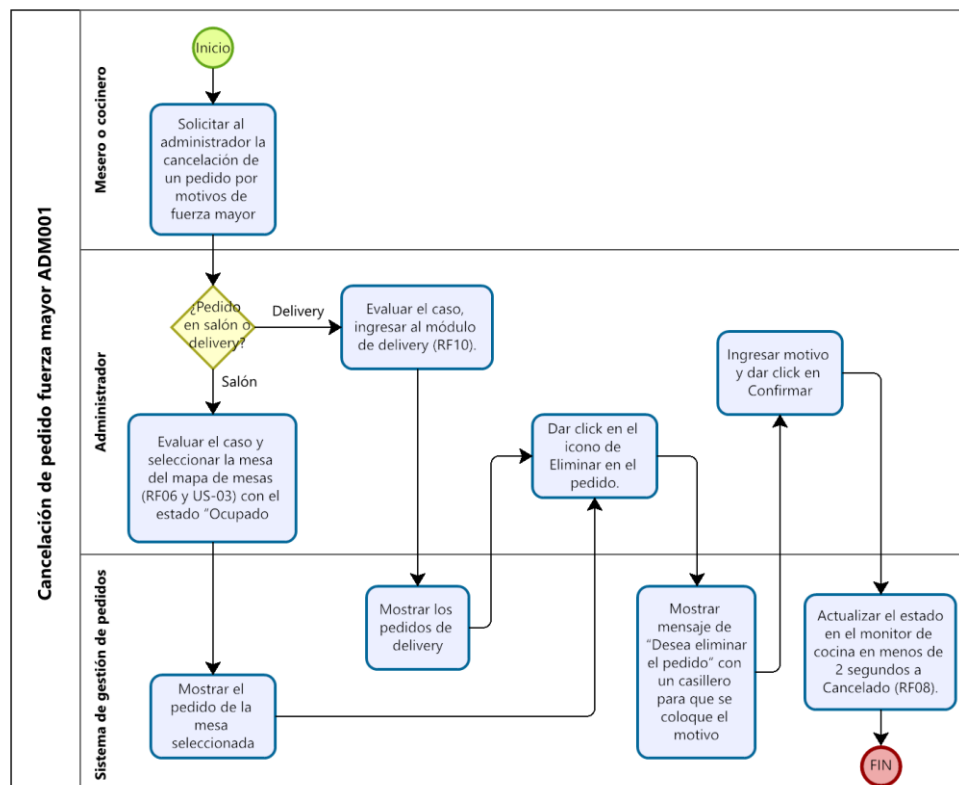
Los procesos de negocio identificados son 5, involucrando en ellos 16 sub-procesos los cuales están determinados de la siguiente manera:

1. Gestión Administrativa

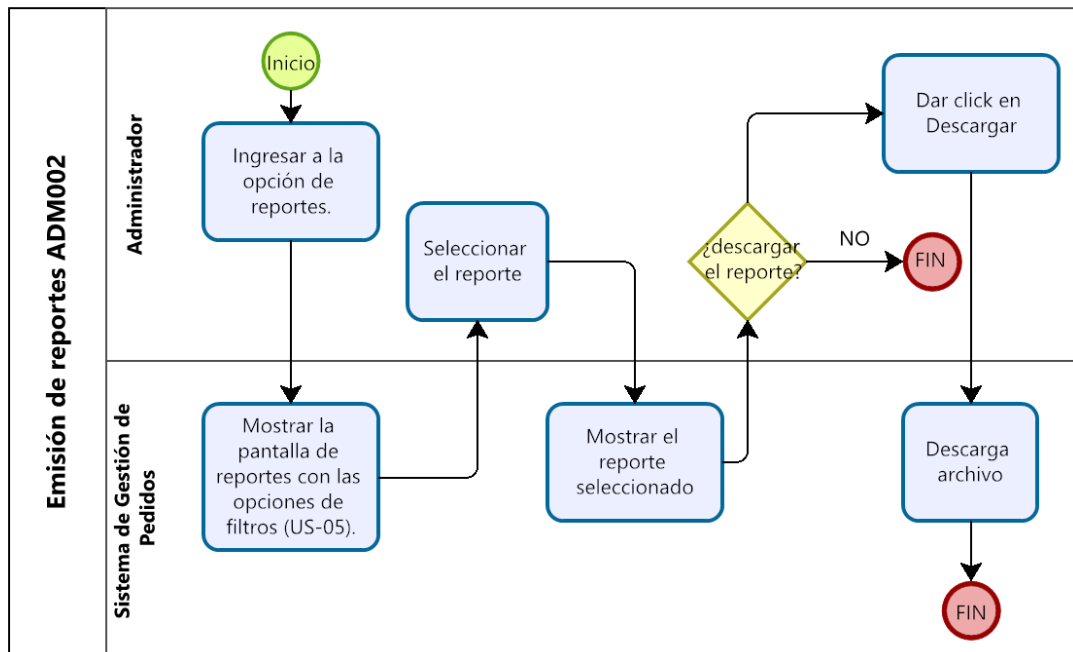


Sub-procesos relacionados:

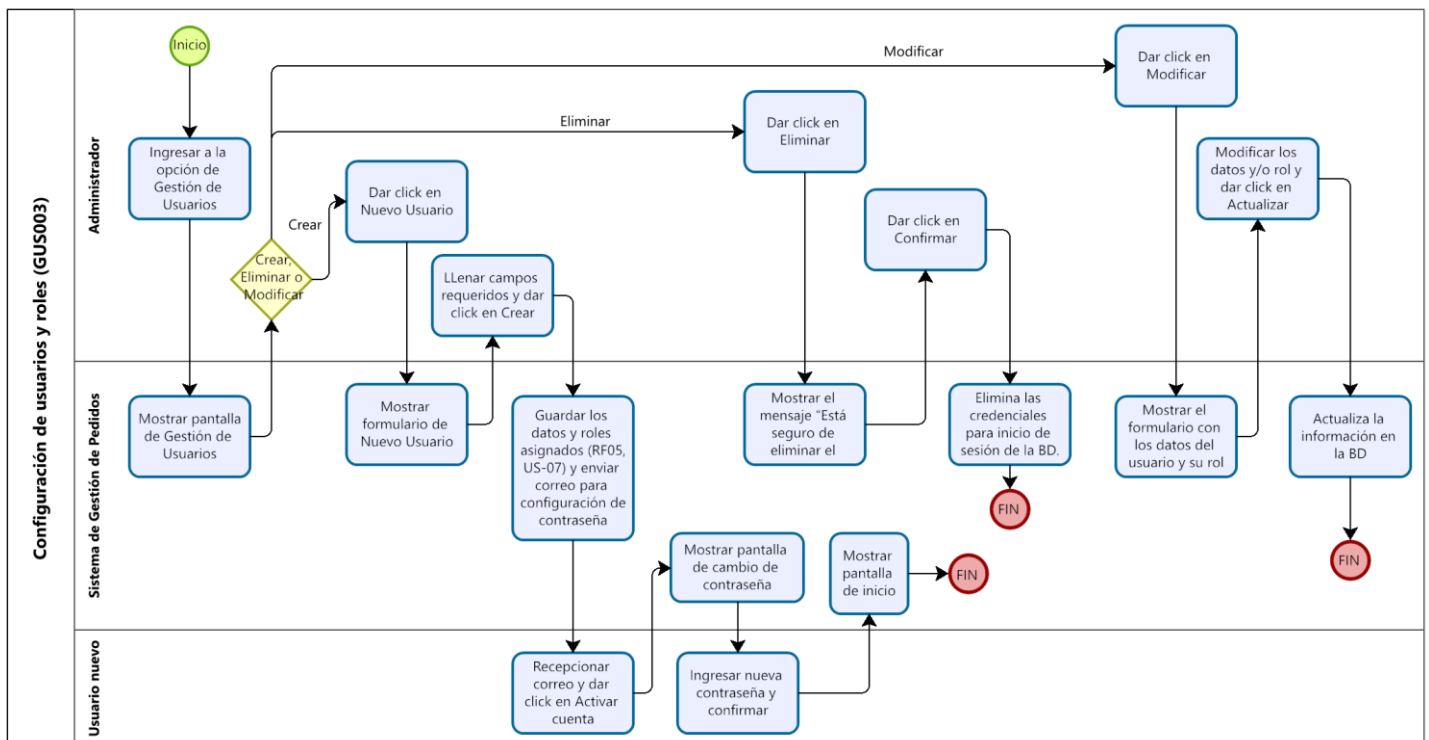
a. Cancelación de pedido fuerza mayor



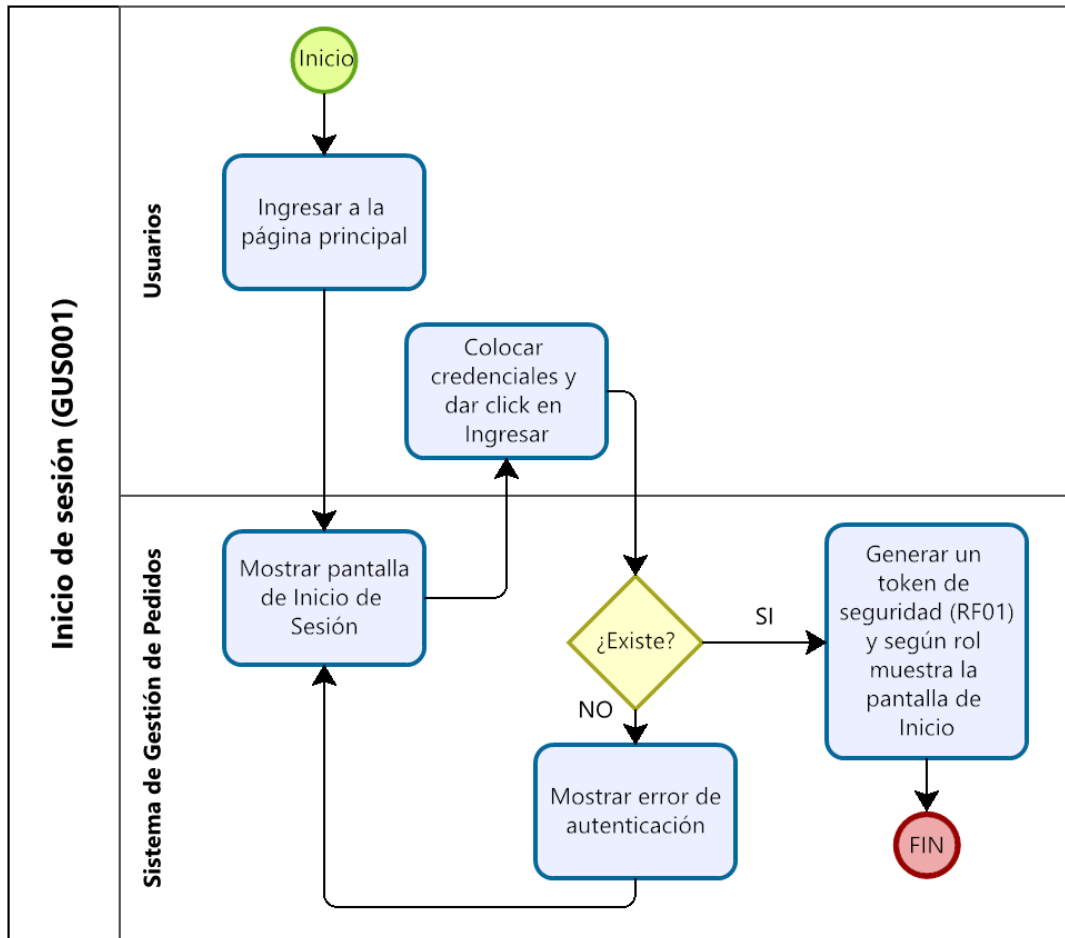
b. Emisión de reportes



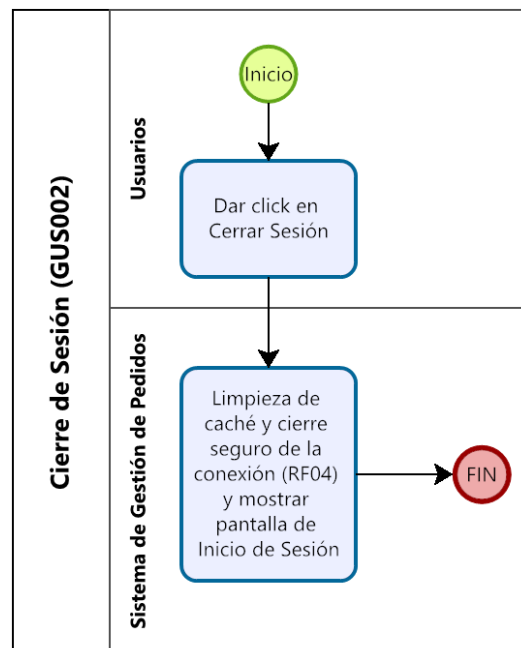
c. Configuración de usuarios y roles



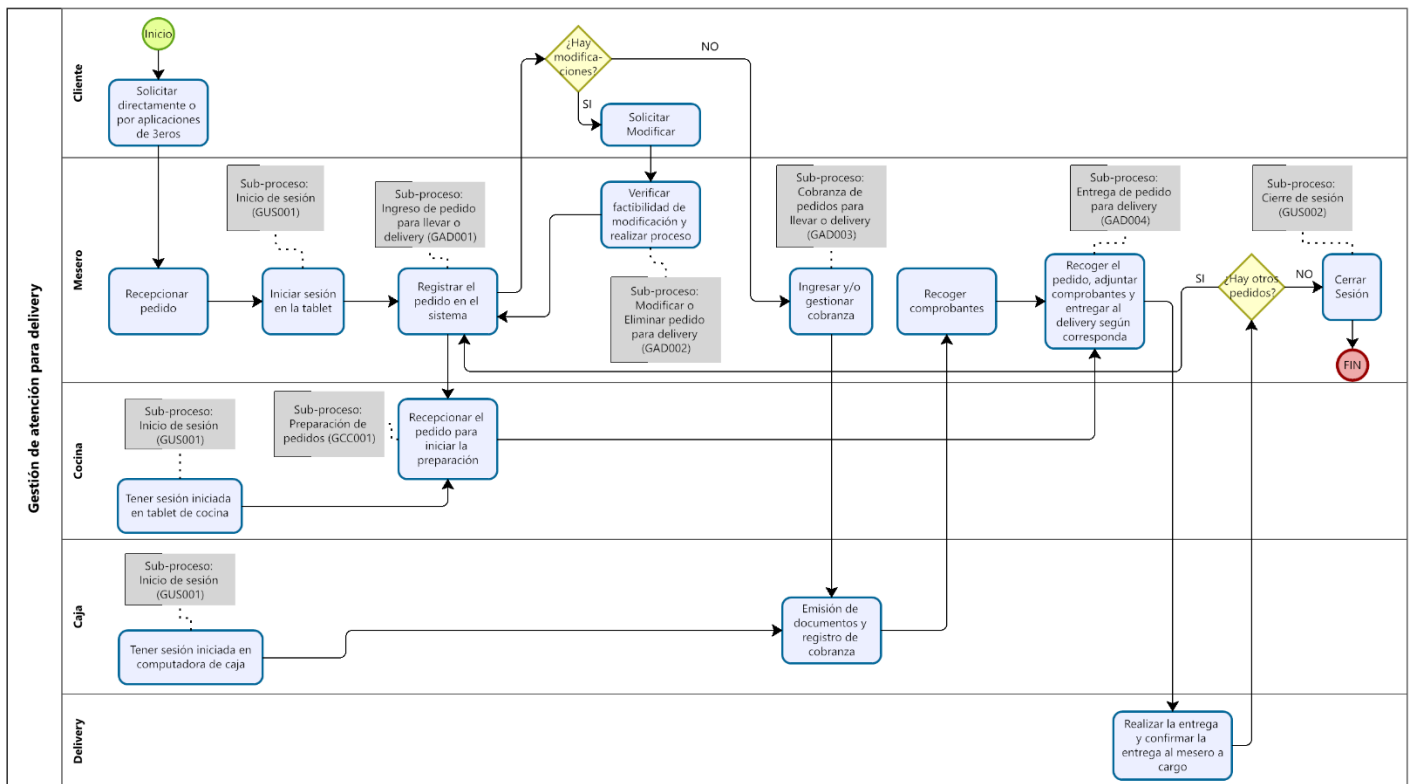
d. Inicio de sesión



e. Cierre de sesión

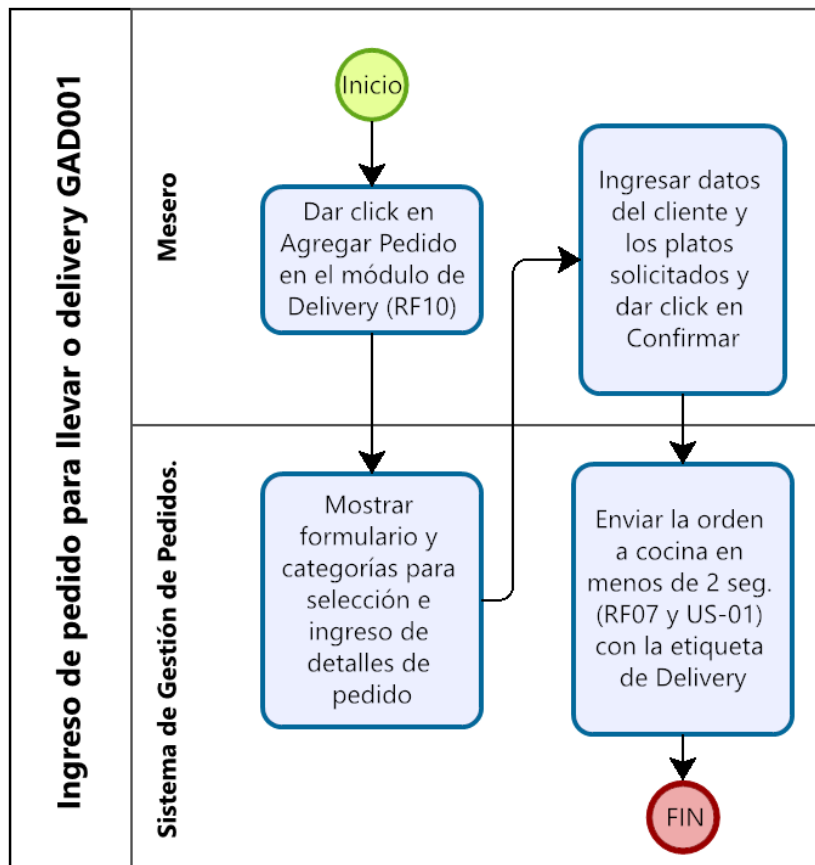


2. Gestión de Atención para Delivery

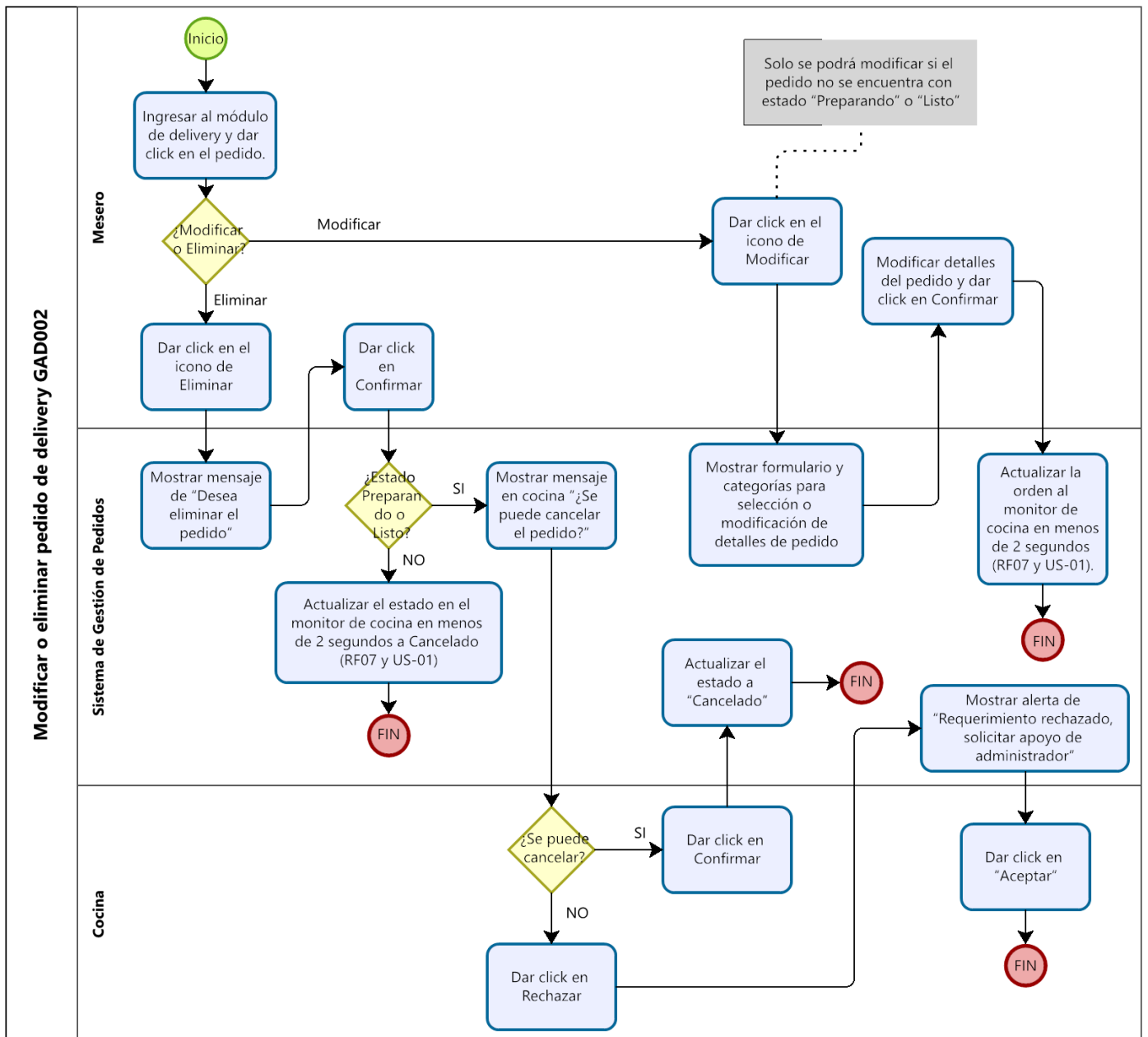


Sub-procesos relacionados:

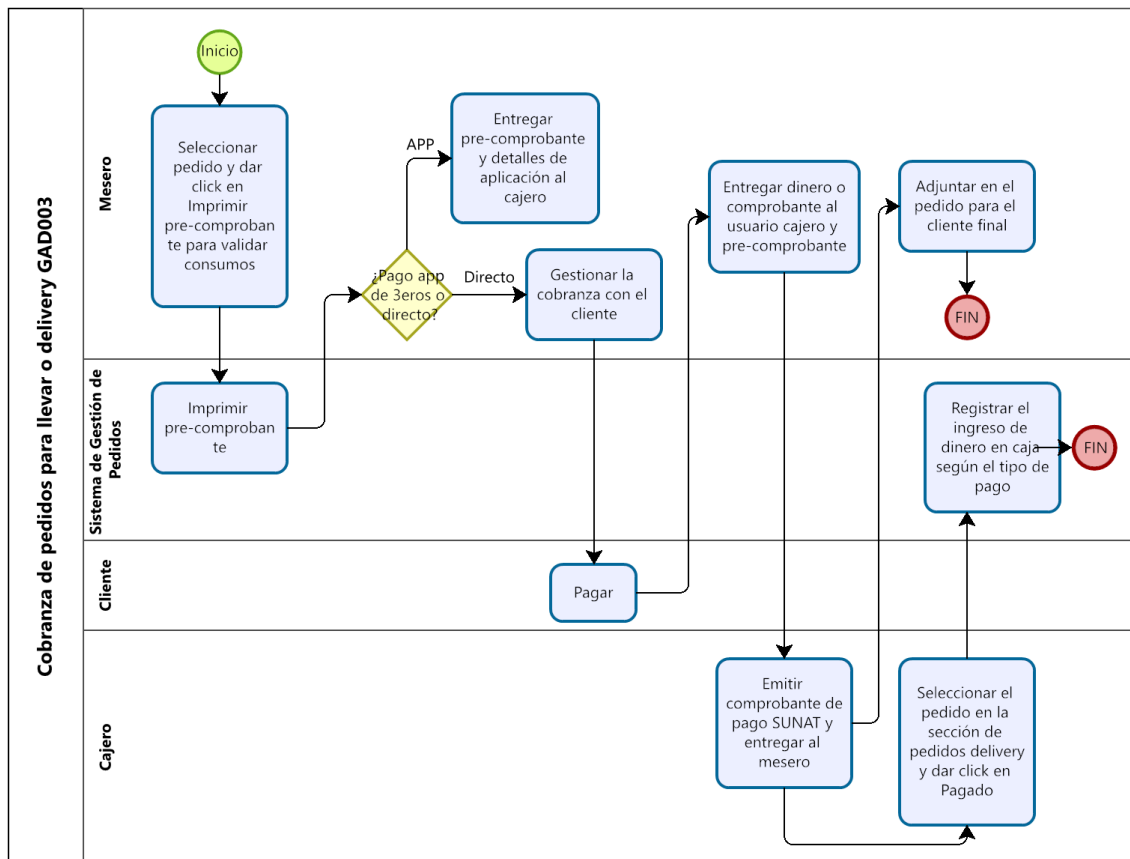
a. Ingreso de pedido para llevar o delivery



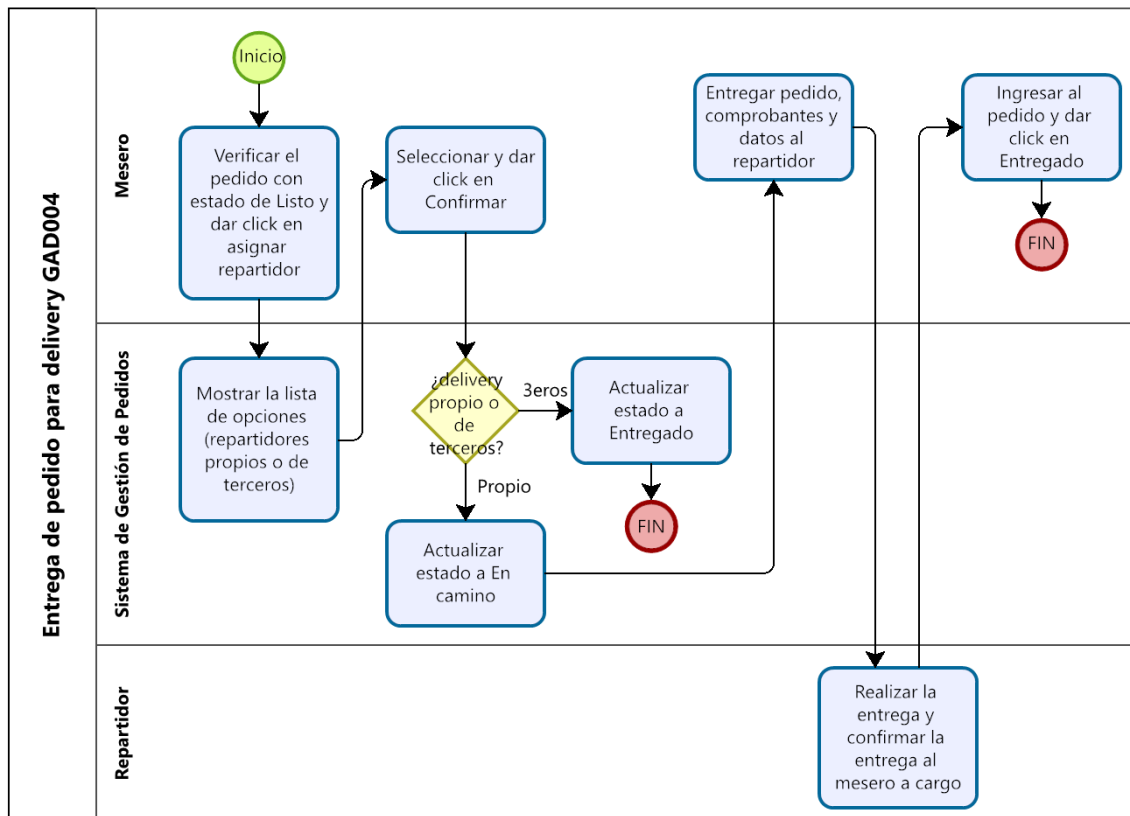
b. Modificar o eliminar pedido para delivery



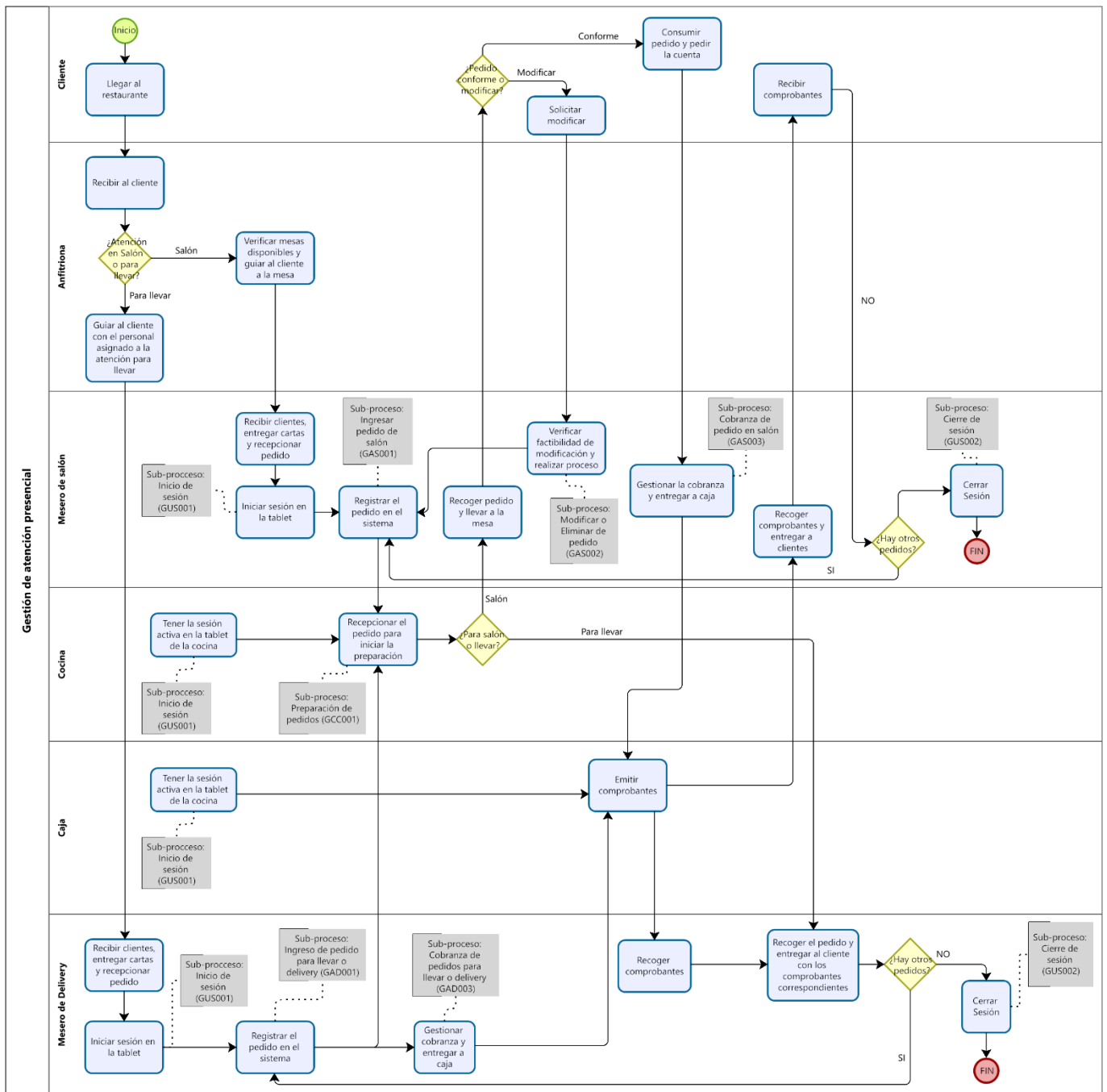
c. Cobranza de pedido para llevar o delivery



d. Entrega de delivery

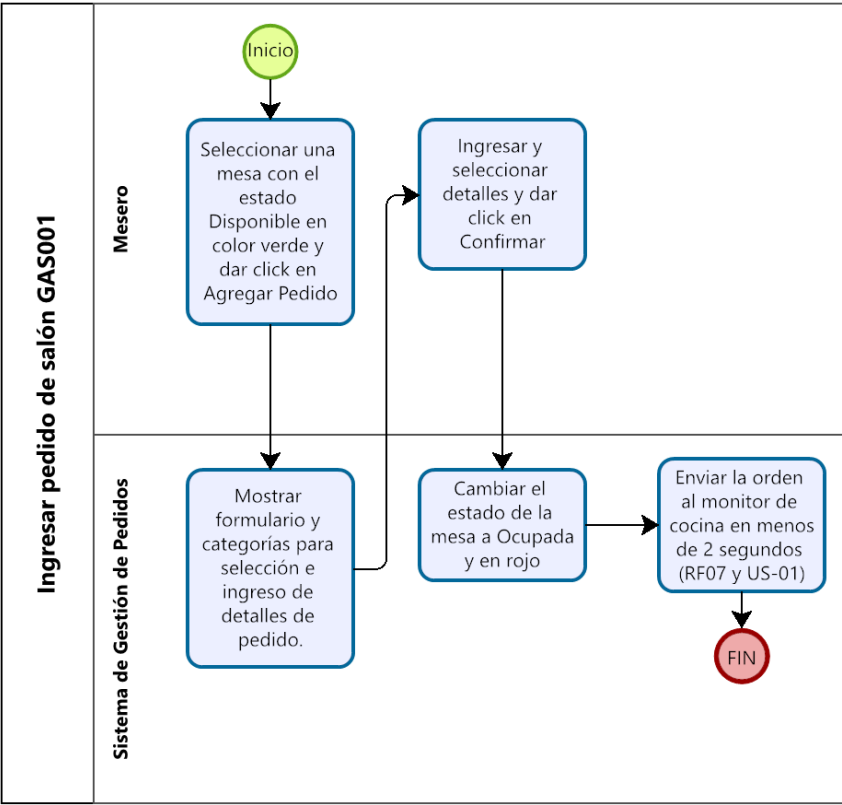


3. Gestión de Atención Presencial

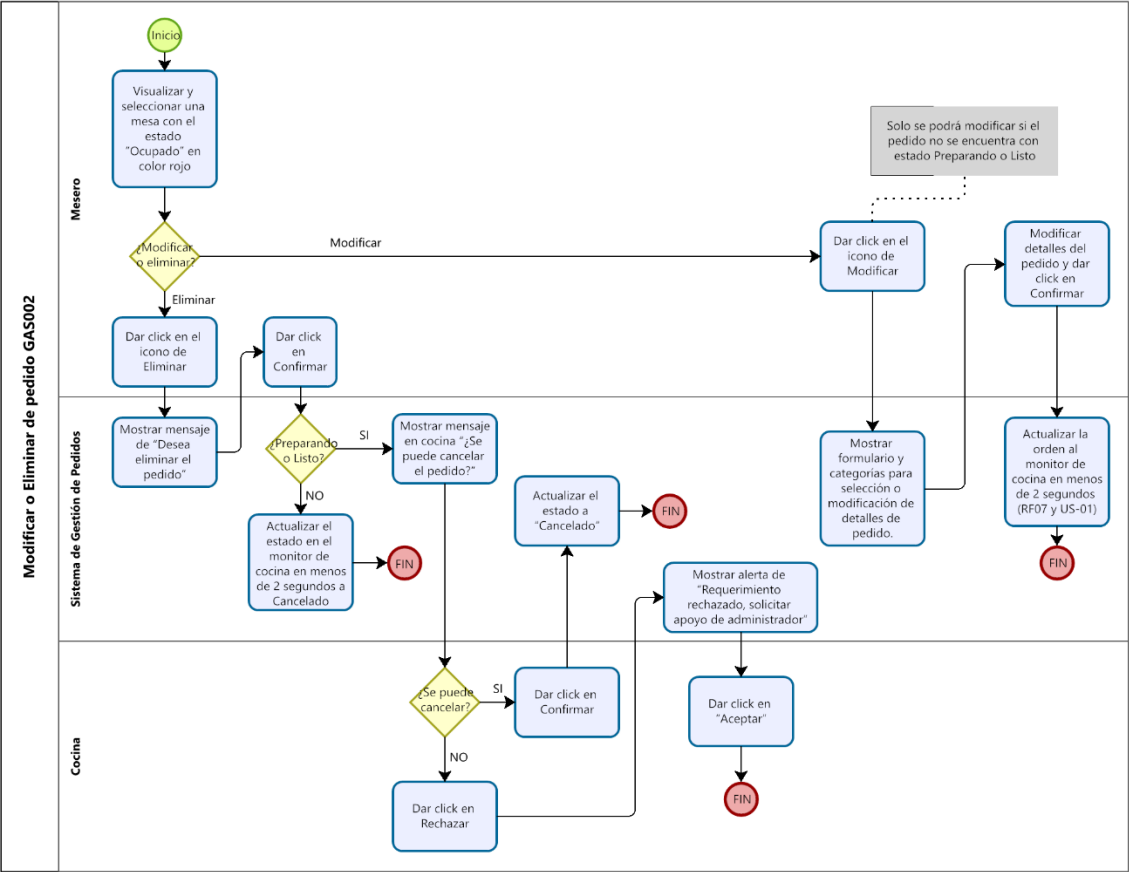


Sub-procesos relacionados:

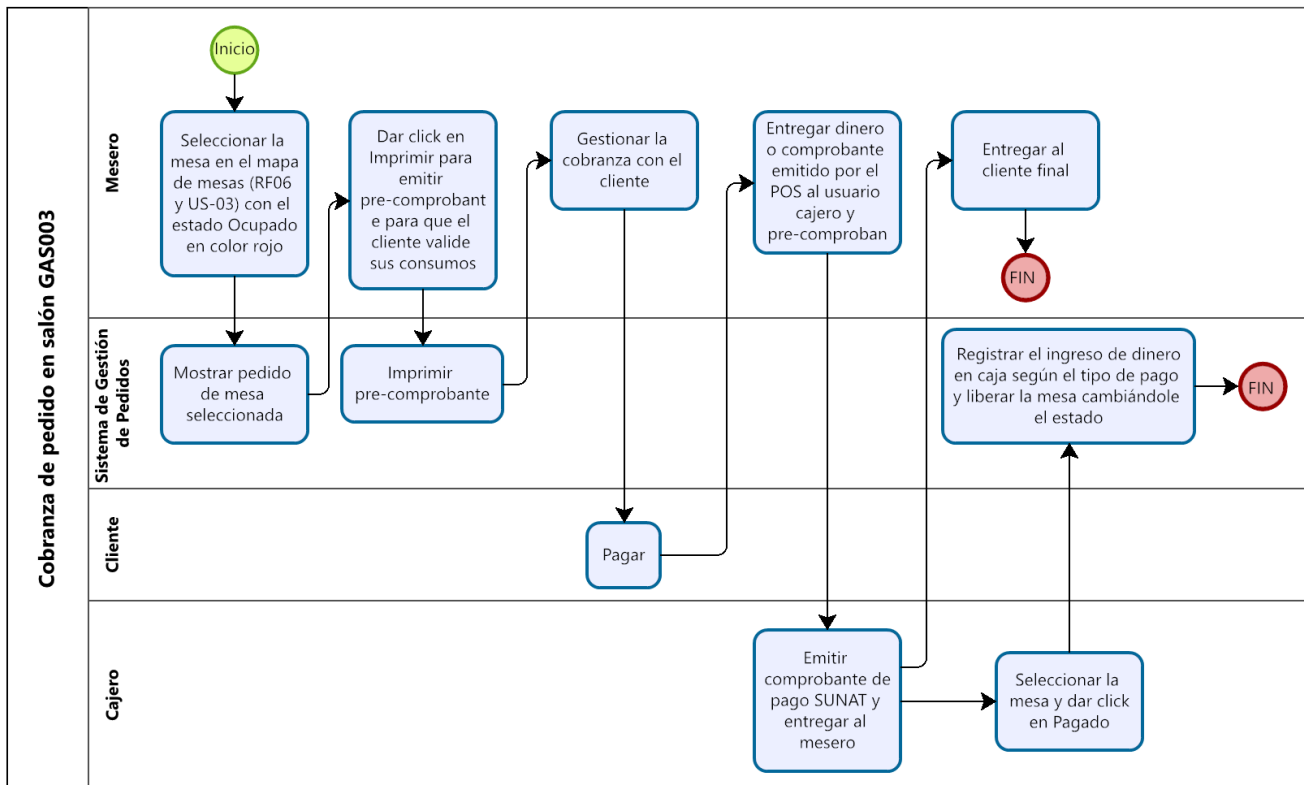
a. Ingresar pedido de salón



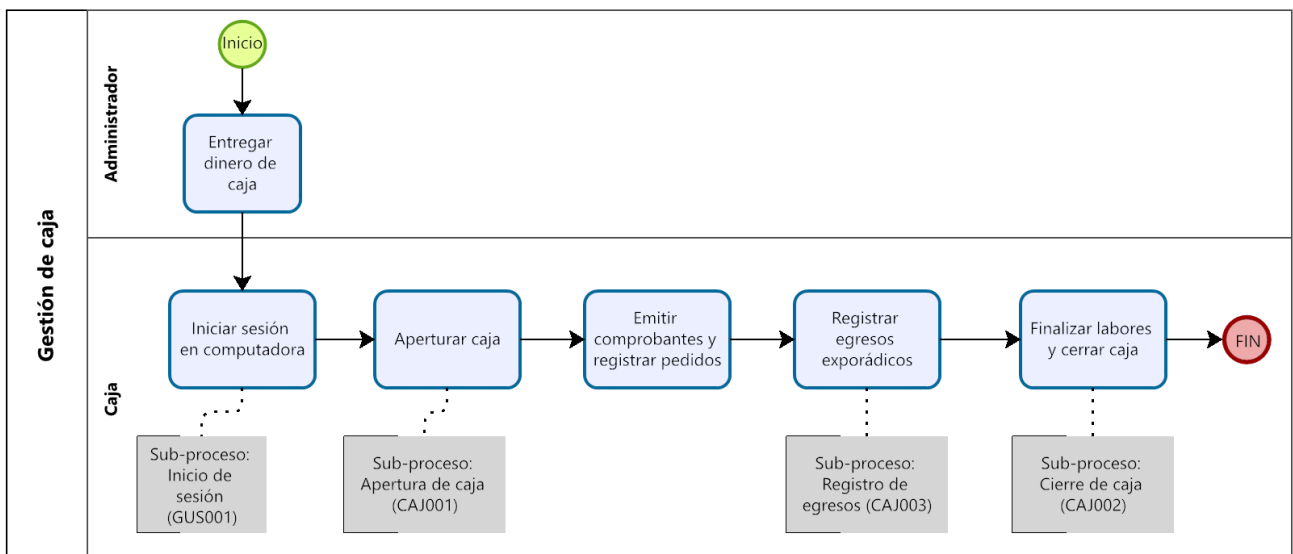
b. Modificar o Eliminar pedido



c. Cobranza de pedido en salón

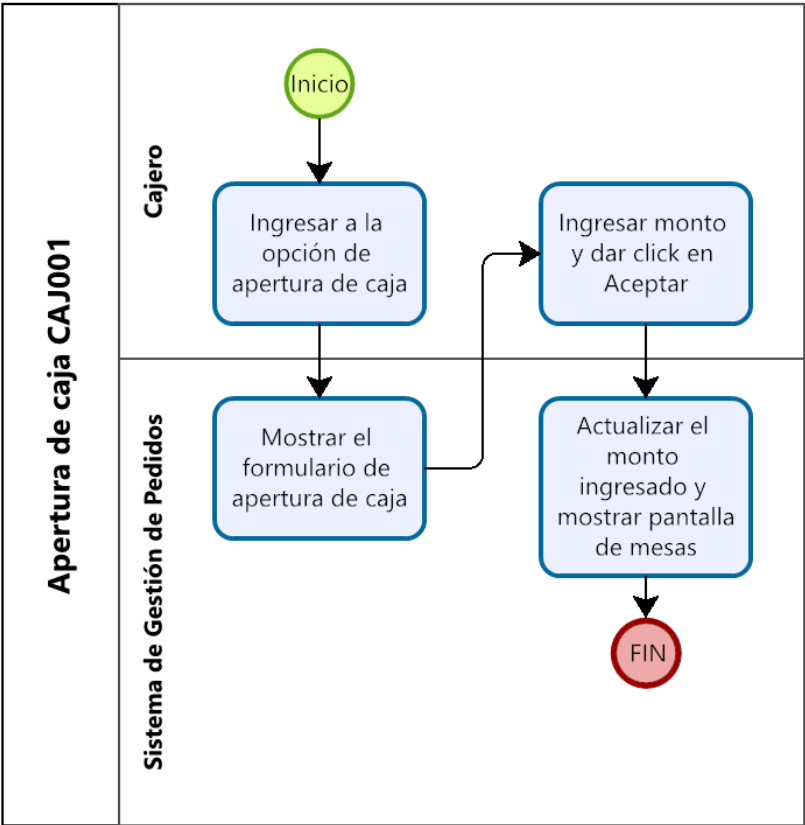


4. Gestión y Control de Caja

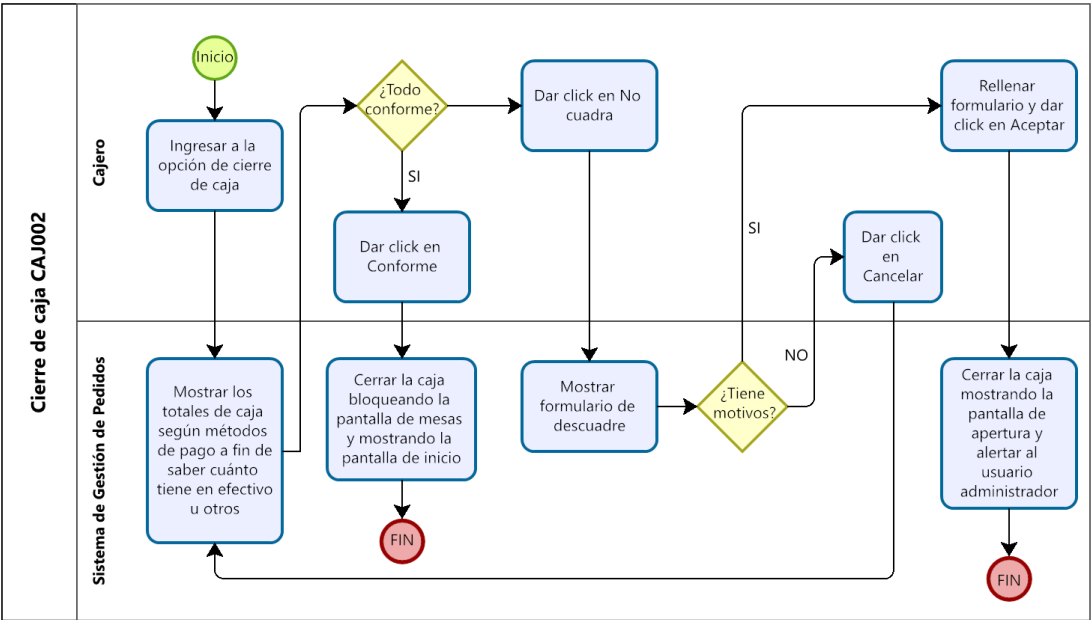


Sub-procesos relacionados:

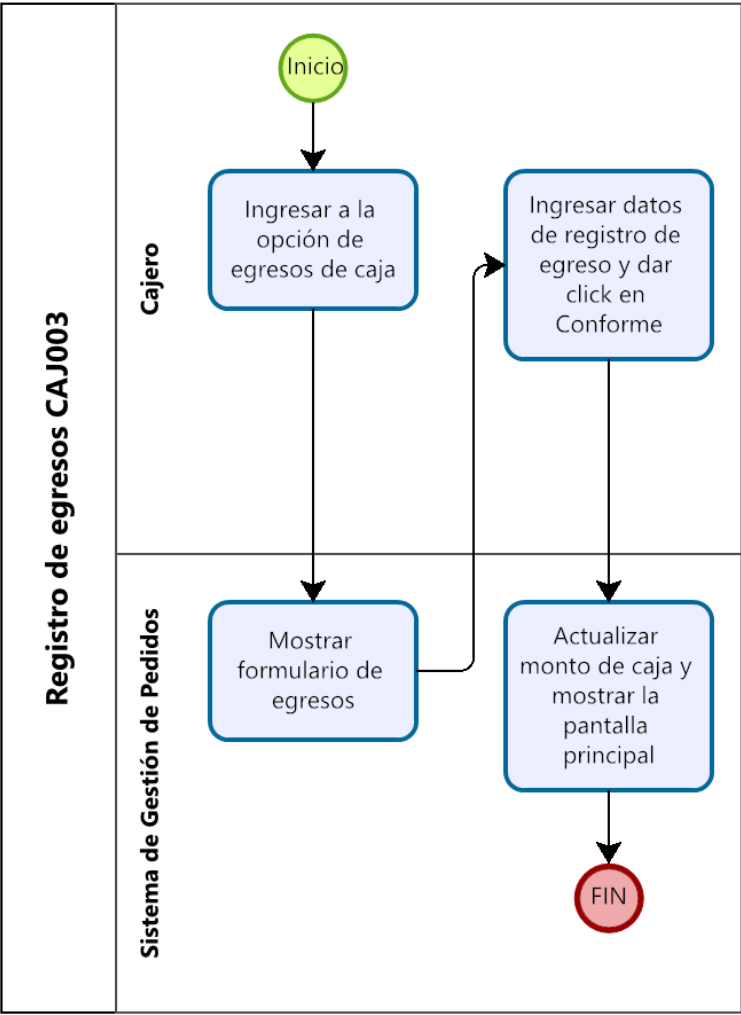
a. Apertura de caja



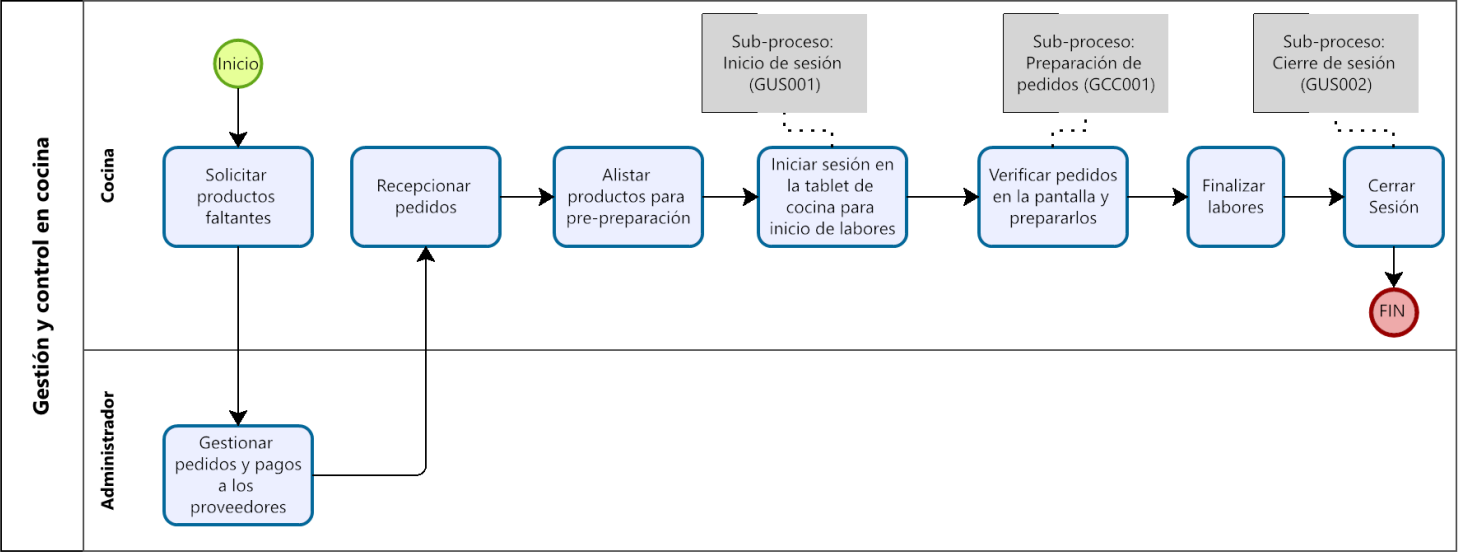
b. Cierre de caja



c. Registro de egresos

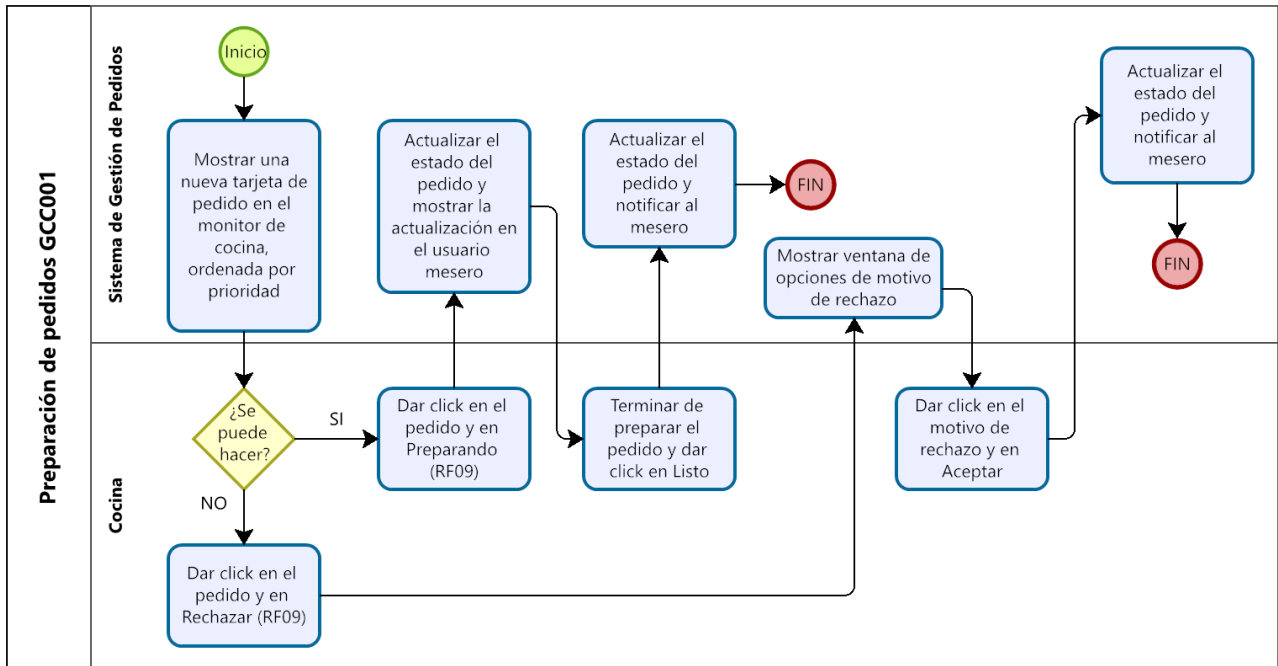


5. Gestión y Control en Cocina



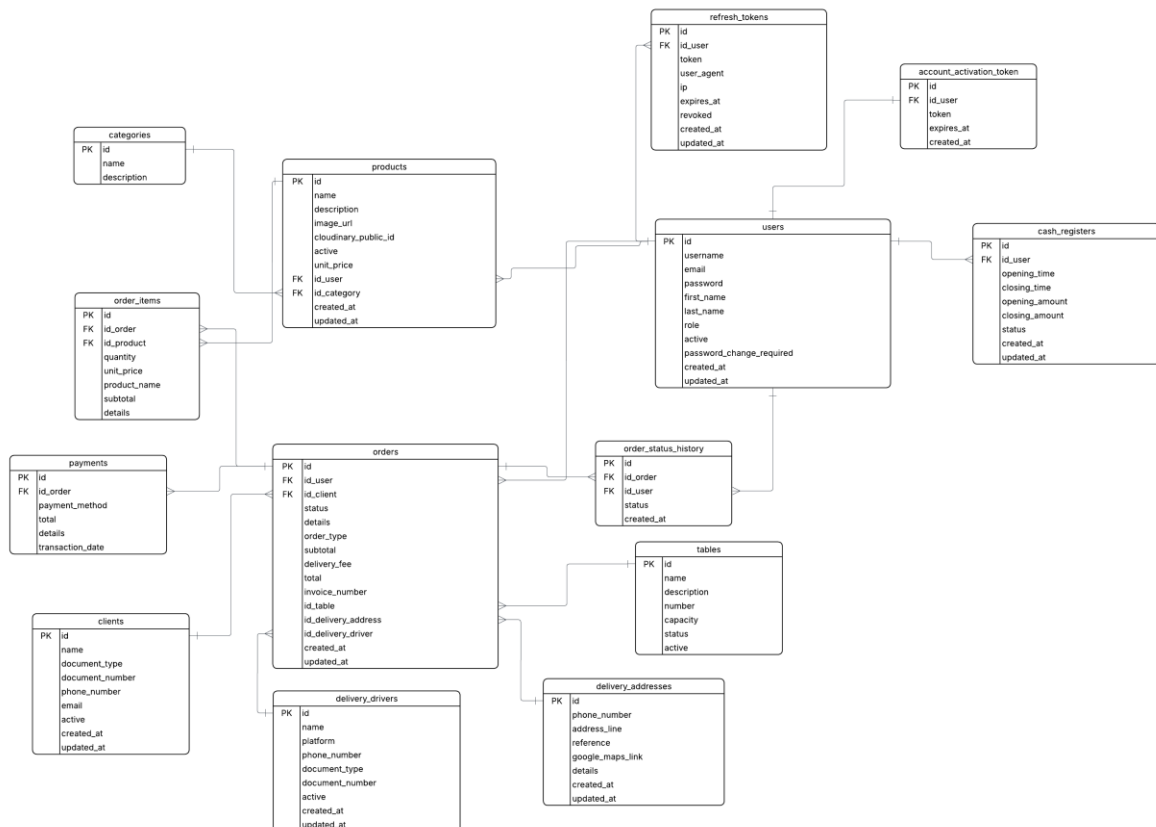
Sub-proceso relacionado:

a. Preparación de pedidos

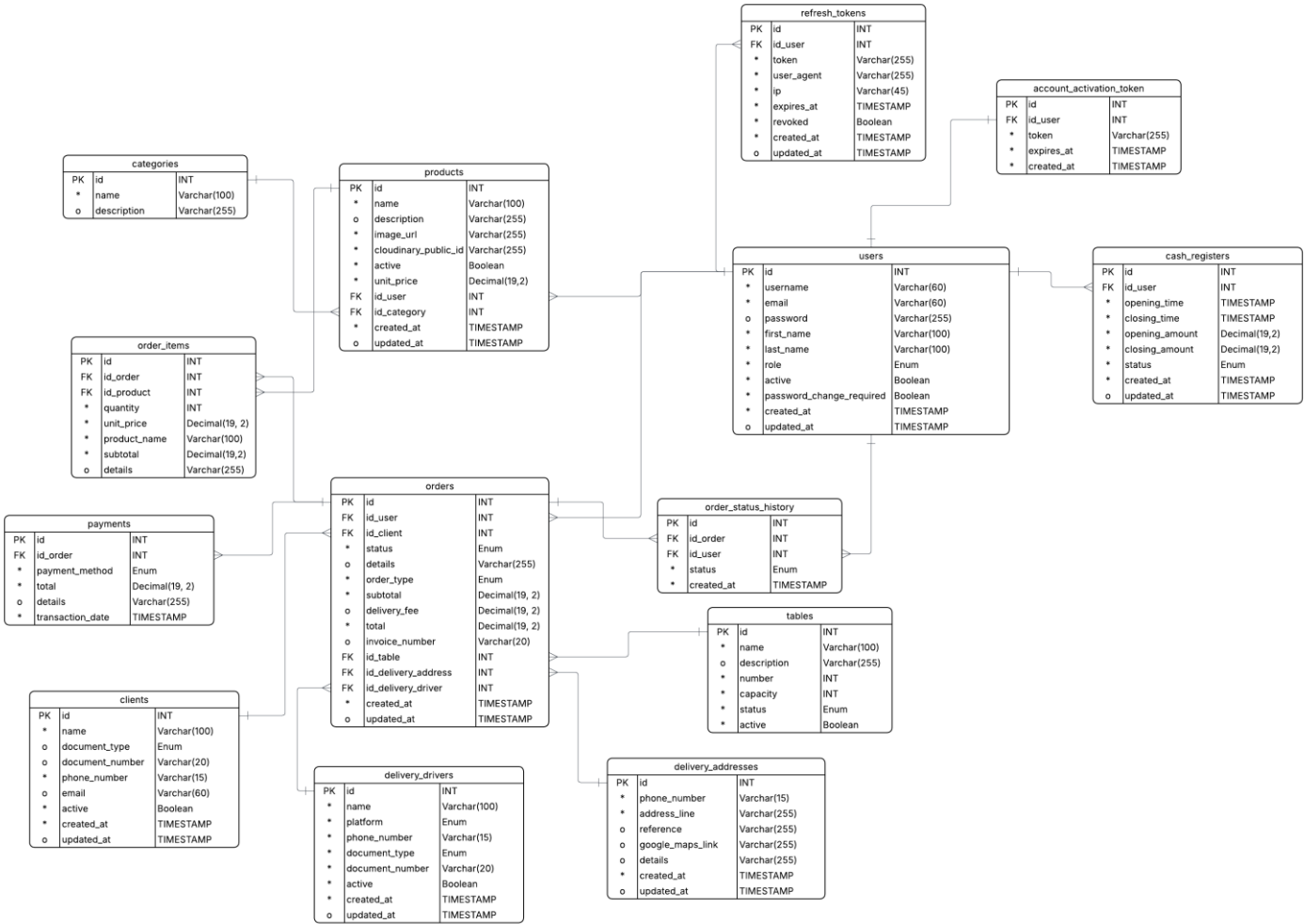


5.2. Diseño de Base de Datos: Modelo Lógico y Físico

Modelo Lógico:

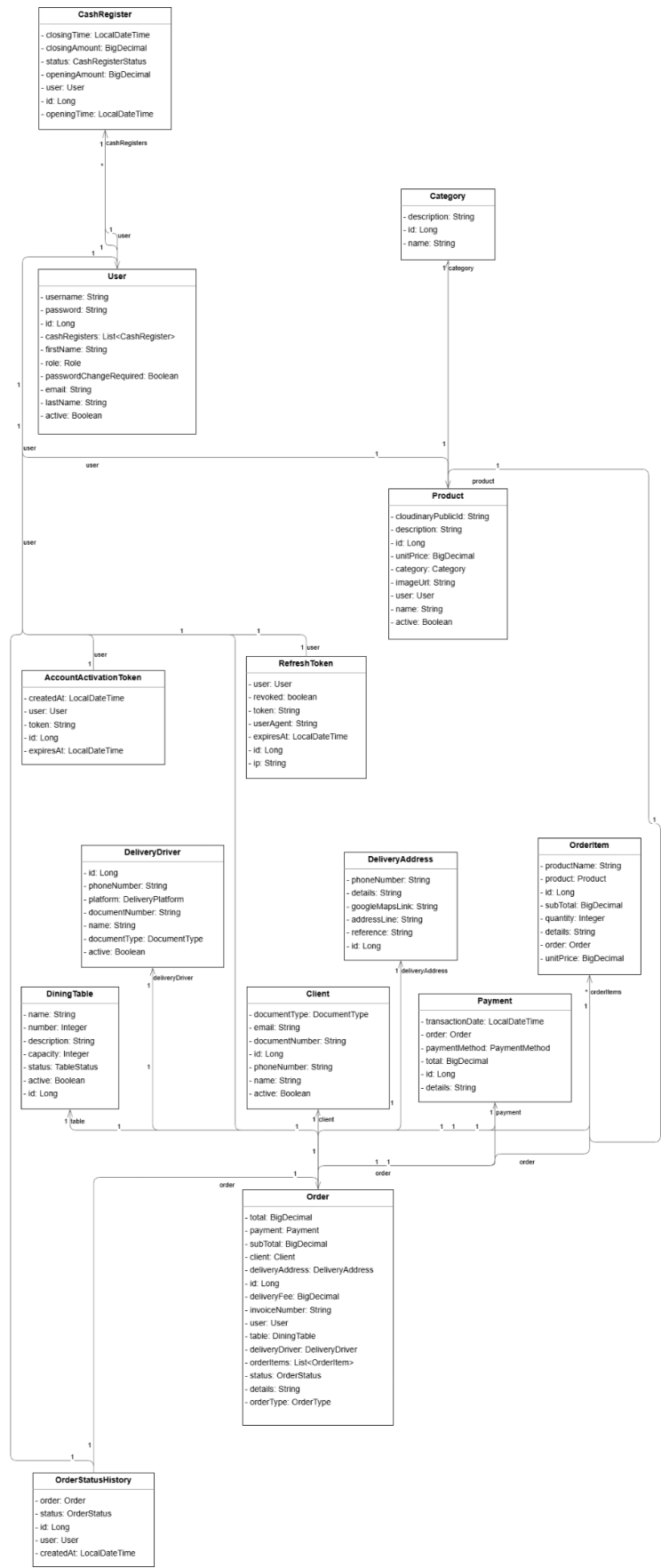


Modelo Físico:



5.3. Diagrama de Clases UML y Documentación Técnica (Markdown)

Diagrama de Clases UML:



Documentación Técnica (Markdown):

Restaurant Backend API

Backend para la operación integral de un restaurante con soporte para delivery y gestión en tiempo real de pedidos mediante WebSocket.

La API expone endpoints REST para:

- Autenticación y autorización
- Usuarios y roles
- Productos y categorías
- Clientes y direcciones
- Repartidores
- Mesas y caja
- Pedidos y dashboard
- Notificaciones en tiempo real vía WebSocket

Tabla de contenidos

- [1. Descripción general](#1-descripción-general)
- [2. Tecnologías principales](#2-tecnologías-principales)
- [3. Estructura del proyecto](#3-estructura-del-proyecto)
- [4. Instalación y ejecución](#4-instalación-y-ejecución)
- [5. Configuración](#5-configuración)
- [6. CORS](#6-cors)
- [7. Swagger / OpenAPI](#7-swagger--openapi)
- [8. Endpoints REST](#8-endpoints-rest)
- [9. Autenticación](#9-autenticación)
- [10. Manejo de errores](#10-manejo-de-errores)
- [11. Ejemplos de requests](#11-ejemplos-de-requests)
- [12. WebSocket](#12-websocket)

1. Descripción general

Propósito

Centralizar la lógica operativa del restaurante en una API segura, escalable y documentada, incluyendo:

- Autenticación JWT
- Filtros avanzados
- Paginación
- WebSocket en tiempo real
- Integraciones externas
- Arquitectura desacoplada

2. Tecnologías principales

Tecnología	Descripción
Java 21	Lenguaje principal
Spring Boot 3.5.13	Framework backend
Spring Security	Seguridad y autenticación
Spring Data JPA	Persistencia
PostgreSQL	Base de datos
Spring WebSocket	Comunicación en tiempo real
JWT (Nimbus JOSE + JWT)	Autenticación
Spring Mail + Thymeleaf	Correos HTML
Springdoc OpenAPI	Documentación Swagger
Cloudinary	Gestión de imágenes
Docker / Docker Compose	Contenedores

| Maven Wrapper | Build del proyecto |

3. Estructura del proyecto

```txt

```
src/main/java/com/app/restaurantbackend

├── config/
│ ├── security/ # Seguridad HTTP + JWT filter
│ ├── ws/ # Configuración WebSocket/STOMP
│ ├── openapi/ # Swagger/OpenAPI
│ ├── AsyncConfig
│ ├── CorsConfig
│ └── CloudinaryConfig
├── events/ # Eventos y listeners
├── persistence/
│ ├── entity/ # Entidades JPA
│ ├── repository/ # Repositorios
│ ├── specification/ # Filtros dinámicos
│ ├── enums/
│ ├── audit/
│ └── projections/
├── presentation/
│ ├── controller/ # REST controllers
│ ├── ws/ # WebSocket controllers
│ ├── dto/
│ └── advice/ # Global exception handler
├── service/
│ ├── interfaces/
│ ├── implementation/
│ ├── dto/
│ └── enums/
├── utils/
│ └── Utilidades, validaciones y excepciones
...

```

```txt

```
src/main/resources

├── application.yaml
├── application-dev.yaml
├── application-prod.yaml
├── data.sql
├── banner.txt
└── templates/email/welcome.html
...

```

4. Instalación y ejecución

Requisitos

- JDK 21
- Docker y Docker Compose
- Maven (opcional)

Opción A – Todo con Docker Compose

```

```bash
docker compose up --build
```

---

## Opción B – PostgreSQL en Docker + App local

### 1. Levantar base de datos

```bash
docker compose up -d db
```

### 2. Ejecutar aplicación

#### PowerShell

```powershell
$env:SPRING_PROFILES_ACTIVE="dev"
.\mvnw.cmd spring-boot:run
```

#### Linux / macOS

```bash
SPRING_PROFILES_ACTIVE=dev ./mvnw spring-boot:run
```

---

## Ejecutar pruebas

```bash
.\mvnw.cmd clean test
```

---

# 5. Configuración

> El proyecto utiliza archivos YAML (`application.yaml`) en lugar de `application.properties`.

---

## Variables de entorno

### Base de datos



Variable	Descripción
`DB_HOST`	Host PostgreSQL
`DB_PORT`	Puerto PostgreSQL
`DB_NAME`	Nombre de base de datos
`DB_USERNAME`	Usuario
`DB_PASSWORD`	Password



---

### JWT



Variable	Descripción
`JWT_SECRET`	Clave secreta
`JWT_EXPIRATION_IN_MINUTES`	Expiración access token
`REFRESH_TOKEN_EXPIRATION_IN_DAYS`	Expiración refresh token
`JWT_ISSUER`	Emisor JWT


```

Correo SMTP

| Variable | Descripción |
|-----------------|---------------|
| `MAIL_HOST` | Host SMTP |
| `MAIL_PORT` | Puerto SMTP |
| `MAIL_USERNAME` | Usuario SMTP |
| `MAIL_PASSWORD` | Password SMTP |

Cloudinary

| Variable | Descripción |
|-------------------------|-------------|
| `CLOUDINARY_CLOUD_NAME` | Cloud name |
| `CLOUDINARY_API_KEY` | API key |
| `CLOUDINARY_API_SECRET` | API secret |

Frontend

| Variable | Descripción |
|----------------|------------------------|
| `FRONTEND_URL` | URL frontend para CORS |

Perfil de desarrollo (`application-dev.yaml`)

| Propiedad | Valor |
|-------------------|---------------|
| `ddl-auto` | `create-drop` |
| `show-sql` | `true` |
| `swagger-ui.path` | `/docs` |
| `api-docs.path` | `/api-docs` |

Perfil de producción (`application-prod.yaml`)

| Propiedad | Valor |
|------------|------------|
| `ddl-auto` | `validate` |

6. CORS

La configuración CORS es dinámica y depende de:

```
```yaml
frontend:
 url: ${FRONTEND_URL:http://localhost:5173}
```
```

Configuración permitida

| Configuración | Valor |
|---------------|--|
| Métodos | `GET, POST, PUT, PATCH, DELETE, OPTIONS` |
| Headers | `*` |
| Credentials | `true` |
| Max Age | `3600` |

7. Swagger / OpenAPI

URLs disponibles

Swagger UI

```
```txt
http://localhost:8080/api/docs
```
```

OpenAPI JSON

```
```txt
http://localhost:8080/api/api-docs
```
```

8. Endpoints REST

Base URL

```
```txt
http://localhost:8080/api
```
```

8.1 Auth

| Método | Endpoint | Descripción |
|--------|----------------|----------------|
| POST | /auth/login | Login |
| POST | /auth/refresh | Renovar tokens |
| POST | /auth/activate | Activar cuenta |

Login Request

```
```json
{
 "username": "admin",
 "password": "admin"
}
```

#### ### Login Response

```
```json
{
  "accessToken": "<jwt>",
  "refreshToken": "<jwt>"
}
```

8.2 Products

Método	Endpoint	Descripción
GET	/products	Listar productos
POST	/products	Crear producto
PUT	/products/{id}	Actualizar producto
PATCH	/products/{id}/image	Actualizar imagen

8.3 Orders

Método	Endpoint	Descripción
GET	/orders	Listar pedidos
GET	/orders/{id}	Obtener detalle
GET	/orders/{id}/pre-check	Descargar precuenta

> La creación y actualización de pedidos se realiza vía WebSocket.

8.4 Users

Método	Endpoint	Descripción
GET	/users	Listar usuarios
POST	/users	Crear usuario
PATCH	/users/me/password	Cambiar password
PATCH	/users/{id}/active	Activar/desactivar usuario

9. Autenticación

Flujo

1. Login mediante /auth/login
2. Recepción de accessToken y refreshToken
3. Uso del bearer token en endpoints protegidos
4. Renovación mediante /auth/refresh

Header requerido

```
```http
Authorization: Bearer <access_token>
```
```

Endpoints públicos

```
```txt
/auth/**
/ws/**
/swagger-ui/**
/api-docs/**
/docs
```
```

10. Manejo de errores

La API utiliza:

```
```java
@RestControllerAdvice
```
```

y responde usando ProblemDetail.

Ejemplo de error

```
```json
{
 "type": "about:blank",
 "title": "VALIDATION_ERROR",
 "status": 400,
 "detail": "Invalid request parameters",
 "path": "/api/users",
 "errors": [
 "email: must be a well-formed email address"
]
}
```
```

11. Ejemplos de requests

Login

```
```bash
curl -X POST "http://localhost:8080/api/auth/login" \
-H "Content-Type: application/json" \
-d '{
 "username": "admin",
 "password": "admin"
}'
```
```

Obtener productos

```
```bash
curl -X GET "http://localhost:8080/api/products?page=0&size=10" \
-H "Authorization: Bearer <access_token>"
```
```

Crear producto con imagen

```
```bash
curl -X POST "http://localhost:8080/api/products" \
-H "Authorization: Bearer <access_token>" \
-F 'data={"name":"Pizza","description":"Familiar","unitPrice":39.90,"categoryId":1};type=application/json' \
-F "image=@C:/ruta/imagen.jpg;type=image/jpeg"
```
```

12. WebSocket

Endpoint SockJS

```
```txt
http://localhost:8080/api/ws
```
```

Cliente → Servidor

```
```txt
/app/orders/create
```
```

```
/app/orders/update  
```
```

```

```

```
Servidor → Cliente
```

```
```.txt  
/topic/orders/new  
/topic/orders/update  
```
```

```

```

```
Header requerido en CONNECT
```

```
```.txt  
Authorization: Bearer <access_token>  
```
```

## CAPÍTULO VI: CONSTRUCCIÓN Y SEGURIDAD

### 6.1. Implementación del Back-End (Mínimo 50% Java)

El back-end del sistema de gestión para restaurante fue desarrollado en Java utilizando el framework Spring Boot 3.5.

El sistema sigue el patrón de arquitectura en capas (Layered Architecture) compuesto por cuatro niveles claramente diferenciados:

- Capa de Presentación (presentation): Controllers REST, DTOs de request/response y manejo global de excepciones.
- Capa de Servicio (service): Interfaces e implementaciones de la lógica de negocio (16 servicios).
- Capa de Persistencia (persistence): Entidades JPA, repositorios Spring Data, especificaciones y enumeraciones.
- Capa de Configuración (config): Configuración de seguridad, CORS, WebSocket, Cloudinary y tareas asíncronas.

#### Stack Tecnológico:

| Tecnología               | Versión                | Propósito                               |
|--------------------------|------------------------|-----------------------------------------|
| Java (Amazon Corretto)   | 21 (JDK/JRE Alpine)    | Lenguaje principal del back-end         |
| Spring Boot              | 3.5.13                 | Framework principal                     |
| Spring Security          | 6.x (incluida en Boot) | Autenticación y autorización            |
| Spring Data JPA          | 6.x                    | Acceso y persistencia de datos          |
| PostgreSQL               | 16 Alpine (Docker)     | Motor de base de datos relacional       |
| Nimbus JOSE + JWT        | 10.9                   | Generación y verificación de tokens JWT |
| Spring WebSocket + STOMP | Incluida en Boot       | Comunicación en tiempo real             |
| Cloudinary HTTP5         | 2.3.2                  | Almacenamiento de imágenes en la nube   |
| Spring Mail + Thymeleaf  | Incluida en Boot       | Envío de correos HTML                   |
| SpringDoc OpenAPI        | 2.8.17                 | Documentación Swagger de la API         |

Estructura del Proyecto:

El proyecto sigue la convención estándar de Maven con el grupo `com.app.restaurantbackend`. A continuación, se presenta la organización de paquetes:

```
com.app.restaurantbackend
├─ config/
├─ events/
├─ persistence/
├─ presentation/
├─ service/
└─ utils/
```

Módulos Funcionales Implementados

| Módulo            | Descripción                                    | Endpoints principales                           |
|-------------------|------------------------------------------------|-------------------------------------------------|
| Autenticación     | Login, refresh token, activación de cuenta     | POST /auth/login, /auth/refresh, /auth/activate |
| Usuarios          | CRUD de empleados del restaurante              | GET/POST /users, PATCH /users/{id}/active       |
| Pedidos (Orders)  | Gestión integral de pedidos con paginación     | GET/POST /orders, estado, historial             |
| Productos         | Catálogo de productos con imagen en Cloudinary | GET/POST/PUT/DELETE /products                   |
| Categorías        | Categorización de productos                    | GET/POST /categories                            |
| Clientes          | Registro de clientes para delivery             | GET/POST /clients                               |
| Repartidores      | Gestión de repartidores de delivery            | GET/POST /delivery-drivers                      |
| Direcciones       | Direcciones de entrega del cliente             | GET/POST /delivery-addresses                    |
| Mesas             | Estado de mesas del restaurante                | GET/POST /tables                                |
| Caja Registradora | Apertura/cierre y reporte de caja              | GET/POST /cash-registers                        |
| Dashboard         | KPIs y resumen ejecutivo del negocio           | GET /dashboard                                  |

## 6.2. Estrategia de Replicación y Administración de BD

Se utiliza PostgreSQL 16 como sistema gestor de base de datos relacional (RDBMS).

### Persistencia mediante Volúmenes Docker:

Los datos de PostgreSQL se almacenan en un volumen Docker nombrado (db\_data) definido en el docker-compose.yml. Esto garantiza que los datos persistan independientemente del ciclo de vida del contenedor. El volumen es gestionado por el motor Docker del servidor anfitrión y puede ser respaldado o migrado de forma independiente al contenedor.

### Estrategia de Respaldo: (Backup)

| Tipo de Respaldo     | Frecuencia                | Herramienta                      |
|----------------------|---------------------------|----------------------------------|
| Completo (full dump) | Diario (fuera de horario) | pg_dump + almacenamiento externo |

## 6.3. Controles de Seguridad: Autenticación, Cifrado y Defensa Web

La seguridad del back-end está implementada sobre Spring Security 6, configurada mediante el bean SecurityFilterChain en SpringSecurityConfig. La cadena de seguridad aplica los siguientes controles en orden:

- Filtro JWT (JwtAuthFilter): intercepta cada petición HTTP antes del filtro de autenticación estándar y valida el token Bearer.
- Proveedor de autenticación (DaoAuthenticationProvider): verifica credenciales contra la base de datos usando BCrypt.
- Control de acceso por URL: rutas públicas limitadas a /auth/\*\*, /ws/\*\* y documentación Swagger.
- Manejo de excepciones de seguridad: respuestas JSON estandarizadas para 401 y 403.

### Autenticación con JWT (JSON Web Tokens)

El sistema implementa autenticación stateless basada en JWT utilizando la biblioteca Nimbus JOSE + JWT 10.9. El flujo de autenticación es el siguiente:

**Paso 1 – Login:** El cliente envía credenciales a POST /auth/login. Spring Security autentica mediante DaoAuthenticationProvider. Si las credenciales son válidas, se genera un Access Token (JWT) con expiración configurable y un Refresh Token de

duración extendida.

**Paso 2 – Acceso a recursos protegidos:** El cliente incluye el Access Token en el header Authorization: Bearer <token>. JwtAuthFilter extrae el token, verifica la firma HMAC-SHA256 y la expiración, extrae el subject (username) y carga el UserDetails desde la base de datos.

**Paso 3 – Renovación de sesión:** Cuando el Access Token expira, el cliente llama a POST /auth/refresh con el Refresh Token. El sistema valida el token en base de datos, lo revoca, genera un nuevo par de tokens y registra la nueva sesión.

**Estructura del JWT:**

Los tokens JWT son firmados con el algoritmo HS256 (HMAC + SHA-256) y contienen los siguientes claims:

| Claim            | Valor                                        | Descripción                                     |
|------------------|----------------------------------------------|-------------------------------------------------|
| sub (subject)    | username del usuario                         | Identificador principal del usuario autenticado |
| role             | ADMIN / MANAGER / WAITER / CASHIER / KITCHEN | Rol del usuario para autorización               |
| type             | ACCESS_TOKEN / REFRESH_TOKEN                 | Tipo de token                                   |
| iss (issuer)     | restaurant-backend (configurable)            | Emisor del token                                |
| iat (issued at)  | Timestamp de emisión                         | Momento de creación del token                   |
| exp (expiration) | iat + expiración configurada                 | Momento de vencimiento del token                |

**Protección contra Vulnerabilidades Web Comunes:**

| Vulnerabilidad (OWASP) | Mecanismo de Defensa                      | Implementación                                               |
|------------------------|-------------------------------------------|--------------------------------------------------------------|
| Inyección SQL          | JPA/JPQL con parámetros ligados           | Spring Data JPA Specifications + repositorios tipados        |
| Autenticación con JWT  | JWT firmado + BCrypt + rotación de tokens | Nimbus JOSE, BCryptPasswordEncoder, refresh token revocation |
| Exposición de          | Contraseñas no incluidas en               | DTOs de respuesta sin campo                                  |



|                                   |                                                         |                                                    |
|-----------------------------------|---------------------------------------------------------|----------------------------------------------------|
| datos sensibles                   | respuestas                                              | password; hash BCrypt en BD                        |
| CSRF (Cross-Site Request Forgery) | API REST stateless sin sesión de servidor               | CSRF deshabilitado; autenticación por Bearer token |
| Acceso no autorizado              | RBAC + validación JWT en cada request                   | SecurityFilterChain + JwtAuthFilter + roles        |
| CORS indiscriminado               | Origen único permitido por variable de entorno          | CorsConfig con allowedOrigins(frontendUrl)         |
| Enumeración de usuarios           | AuthServiceImpl devuelve 'Invalid username or password' |                                                    |
| Tokens de larga duración          | Access Token de corta vida (15 min por defecto)         | JWT_EXPIRATION_IN_MINUTES configurable en entorno  |

## CAPÍTULO VII: VERIFICACIÓN Y DESPLIEGUE V1

### 7.1. Pruebas Funcionales y No Funcionales (Validación del Servicio)

#### Pruebas Funcionales

##### Gestión de Acceso de Usuario

The first screenshot shows the 'PANEL DE CONTROL' (Dashboard) for the 'ADMIN\_CHIFA' session. It features a sidebar with navigation options like 'OPERACIONES', 'GESTIÓN', 'SALÓN', 'CATÁLOGO', and 'FINANZAS'. The main area displays key metrics: 'VENTAS DEL DÍA' (S/ 0.00), 'MESAS ACTIVAS' (00 / 08), and 'PEDIDOS CRÍTICOS' (00). Below these are four module cards: 'Gestión de Salón', 'Monitor de Cocina', 'Delivery', and 'Finanzas y Reportes', each with an 'INGRESAR' button.

The second screenshot shows the 'GESTIÓN DE USUARIOS' (User Management) page with 3 registered users. It includes a summary bar with counts for 'TOTAL', 'ACTIVOS', 'INACTIVOS', 'GERENTES', 'MESEROS', 'CAJEROS', and 'COCINA'. A search bar and a table of users are also present.

| USUARIO          | NOMBRE       | EMAIL                         | ROL    | ESTADO | DESDE        |
|------------------|--------------|-------------------------------|--------|--------|--------------|
| admin            | Admin Admin  | admin@admin.com               | Admin  | Activo | 18 may. 2026 |
| janeth_pevesuirl | Janeth Peves | janethpevesramirez@gmail.com  | Mesero | Activo | 18 may. 2026 |
| leslie_pevespkyv | Leslie Peves | djanethpevesramirez@gmail.com | Cocina | Activo | 18 may. 2026 |

The third screenshot shows the 'GESTIÓN DE USUARIOS' page after creating a new user, now with 4 registered users. A success message 'User created successfully' is displayed. The summary bar and table are updated to include the new user, 'ana\_guilléntopx' (Ana Guillén).

| USUARIO          | NOMBRE       | EMAIL                         | ROL     | ESTADO | DESDE        |
|------------------|--------------|-------------------------------|---------|--------|--------------|
| admin            | Admin Admin  | admin@admin.com               | Admin   | Activo | 18 may. 2026 |
| janeth_pevesuirl | Janeth Peves | janethpevesramirez@gmail.com  | Mesero  | Activo | 18 may. 2026 |
| leslie_pevespkyv | Leslie Peves | djanethpevesramirez@gmail.com | Cocina  | Activo | 18 may. 2026 |
| ana_guilléntopx  | Ana Guillén  | anag@gmail.com                | Gerente | Activo | 18 may. 2026 |

Cientes:

CHIFA

SISTEMA POS

OPERACIONES

Pedidos

GESTIÓN

Usuarios

Cientes

Conductores

SALÓN

Mesas

Órdenes

CATÁLOGO

Categorías

Productos

FINANZAS

Registros de Caja

GESTIÓN DE CLIENTES

38 CLIENTES REGISTRADOS

+ NUEVO CLIENTE

Buscar por nombre...

Filtros

BUSCAR

| CLIENTE                 | DOCUMENTO    | TÉLFONO   | EMAIL                     | ESTADO |
|-------------------------|--------------|-----------|---------------------------|--------|
| J Juan Pérez Rojas      | DNI 74281936 | 987654321 | juan.perez@gmail.com      | Activo |
| M María López Chávez    | DNI 71928364 | 986543210 | maria.lopez@hotmail.com   | Activo |
| C Carlos Ramírez Huamán | DNI 78519283 | 985432189 | carlos.ramirez@yahoo.com  | Activo |
| A Ana Torres Salas      | DNI 69827364 | 984321898 | ana.torres@gmail.com      | Activo |
| L Luis Gómez Vargas     | DNI 68291837 | 983218987 | luis.gomez@gmail.com      | Activo |
| S Sofía Herrera Núñez   | DNI 67192845 | 982189876 | sofia.herrera@hotmail.com | Activo |
| M Miguel Díaz Flores    | DNI 65918273 | 981898765 | miguel.diaz@gmail.com     | Activo |
| L Lucía Vargas León     | DNI 64827391 | 988987654 | lucia.vargas@yahoo.com    | Activo |
| P Pedro Castillo Ríos   | DNI 63728194 | 979876543 | pedro.castillo@gmail.com  | Activo |

4 / 6 requests

10.7 kB / 10.7 kB transferred

8.7 kB / 8.7 kB recs

Gestión de Mesas

CHIFA

SISTEMA POS

OPERACIONES

Pedidos

GESTIÓN

Usuarios

Cientes

Conductores

SALÓN

Mesas

Órdenes

CATÁLOGO

Categorías

Productos

FINANZAS

Registros de Caja

GESTIÓN DE MESAS

SALÓN DEL RESTAURANTE

+ NUEVA MESA

DISPONIBLES 5

Ocupadas 5

RESERVADAS 5

FUERA DE SERVICIO 5

Todas 20

Disponibles 5

Ocupadas 5

Reservadas 5

Fuera de Servicio 5

1 MESA 1

Mesa para dos pe...

2 personas

Disponible

2 MESA 2

Mesa estándar

4 personas

Disponible

3 MESA 3

Mesa cerca de ve...

6 personas

Disponible

4 MESA 4

Mesa familiar

6 personas

Disponible

5 MESA 5

Mesa íntima

2 personas

Disponible

6 MESA 6

Mesa ocupada

4 personas

Ocupada

7 MESA 7

Mesa ocupada

2 personas

Ocupada

8 MESA 8

Mesa ocupada

6 personas

Ocupada

9 MESA 9

Mesa ocupada

4 personas

Ocupada

10 MESA 10

Mesa ocupada

2 personas

Ocupada

11 MESA 11

Mesa reservada

4 personas

Reservada

12 MESA 12

Mesa reservada

2 personas

Reservada

10 / 16 requests

37.3 kB / 37.3 kB transferred

32.3 kB / 32.3 kB

Órdenes

CHIFA

SISTEMA POS

OPERACIONES

Pedidos

GESTIÓN

Usuarios

Cientes

Conductores

SALÓN

Mesas

Órdenes

CATÁLOGO

Categorías

Productos

FINANZAS

Registros de Caja

GESTIÓN DE ÓRDENES

20 ÓRDENES EN TOTAL

Todos los tipos

Salón

Para llevar

Delivery

Todos los estados

N° de comprobante...

Filtros

BUSCAR

| # ORDEN          | CLIENTE                               | TIPO            | ESTADO     | TOTAL     | PAGO      | FECHA                     |
|------------------|---------------------------------------|-----------------|------------|-----------|-----------|---------------------------|
| #1<br>DNI - 801  | Juan Pérez Rojas<br>987654321         | Salón<br>Mesa 1 | Completado | \$/ 25.00 | Efectivo  | 10 mar. 2025, 12:00 p. m. |
| #4<br>DNI - 804  | Ana Torres Salas<br>984321898         | Salón<br>Mesa 2 | Servido    | \$/ 22.00 | Pendiente | 10 abr. 2025, 12:30 p. m. |
| #8<br>DNI - 808  | Lucía Vargas León<br>988987654        | Salón<br>Mesa 3 | Preparando | \$/ 28.00 | Pendiente | 20 may. 2025, 01:15 p. m. |
| #11<br>DNI - 811 | Bodega Las Flores S.A.C.<br>977043215 | Salón<br>Mesa 4 | Servido    | \$/ 26.00 | Pendiente | 20 jun. 2025, 01:00 p. m. |
| #15<br>DNI - 815 | Logísticas Perú E.I.R.L.<br>97228987  | Salón<br>Mesa 5 | Preparando | \$/ 24.00 | Pendiente | 01 ago. 2025, 01:45 p. m. |
| #19<br>DNI - 819 | Paola Cruz<br>989876543               | Salón<br>Mesa 6 | Servido    | \$/ 29.00 | Pendiente | 01 oct. 2025, 01:20 p. m. |

500 ms

1,000 ms

1,500 ms

Name

orders?page=0&si...

200

xhr

get-orders

14.0...

38 ms

orders?page=0&si...

200

xhr

get-orders

14.0...

76 ms

info?n=177913958...

200

xhr

websocket

0.6 kB

6 ms

info?n=177913959...

200

xhr

websocket

0.6 kB

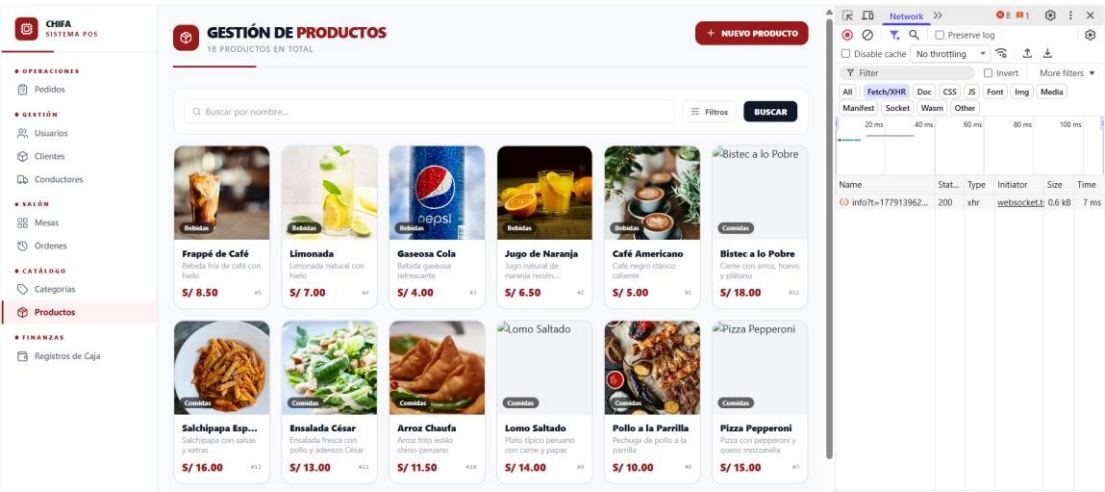
8 ms

4 / 6 requests

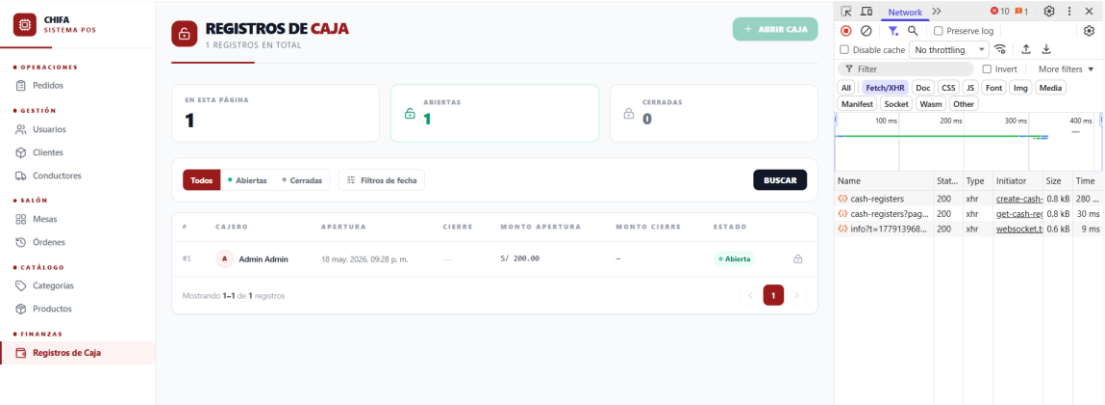
10.7 kB / 10.7 kB transferred

8.7 kB / 8.7 kB recs

Productos

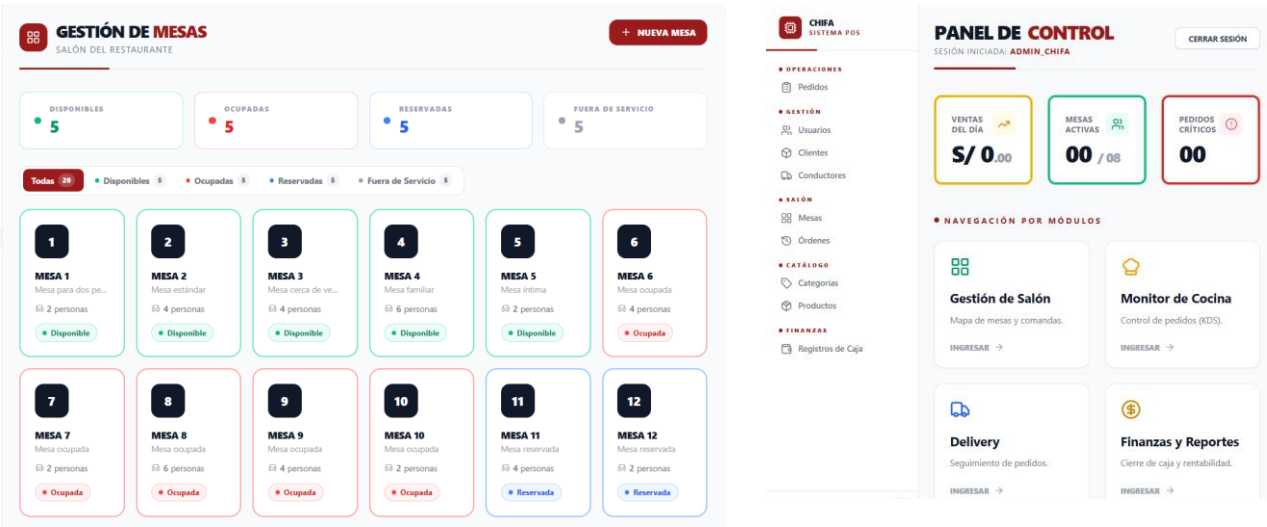


Gestión de Cajas

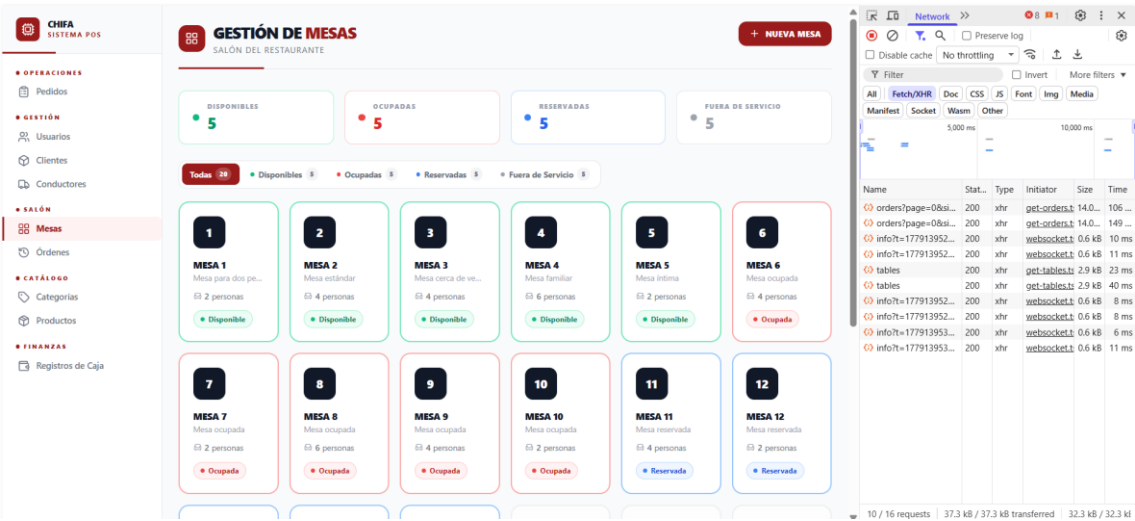


Pruebas No Funcionales

RNF01 - Usabilidad: La interfaz debe ser intuitiva y adaptada a dispositivos móviles para facilitar el uso operativo.

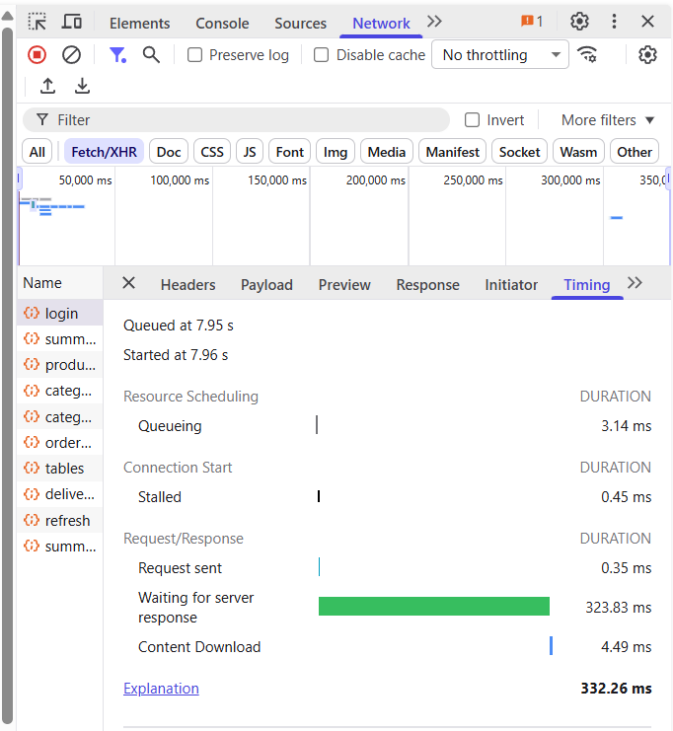


RNF02 - Disponibilidad: El sistema debe operar el 99.9% del tiempo durante el horario comercial del restaurante.

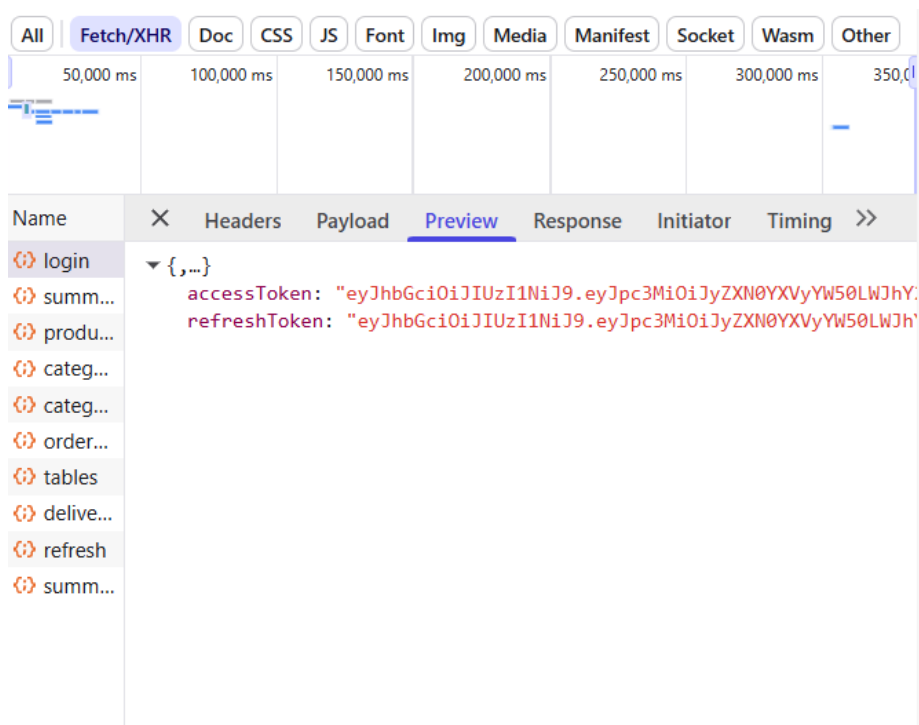


### RNF03 – Rendimiento

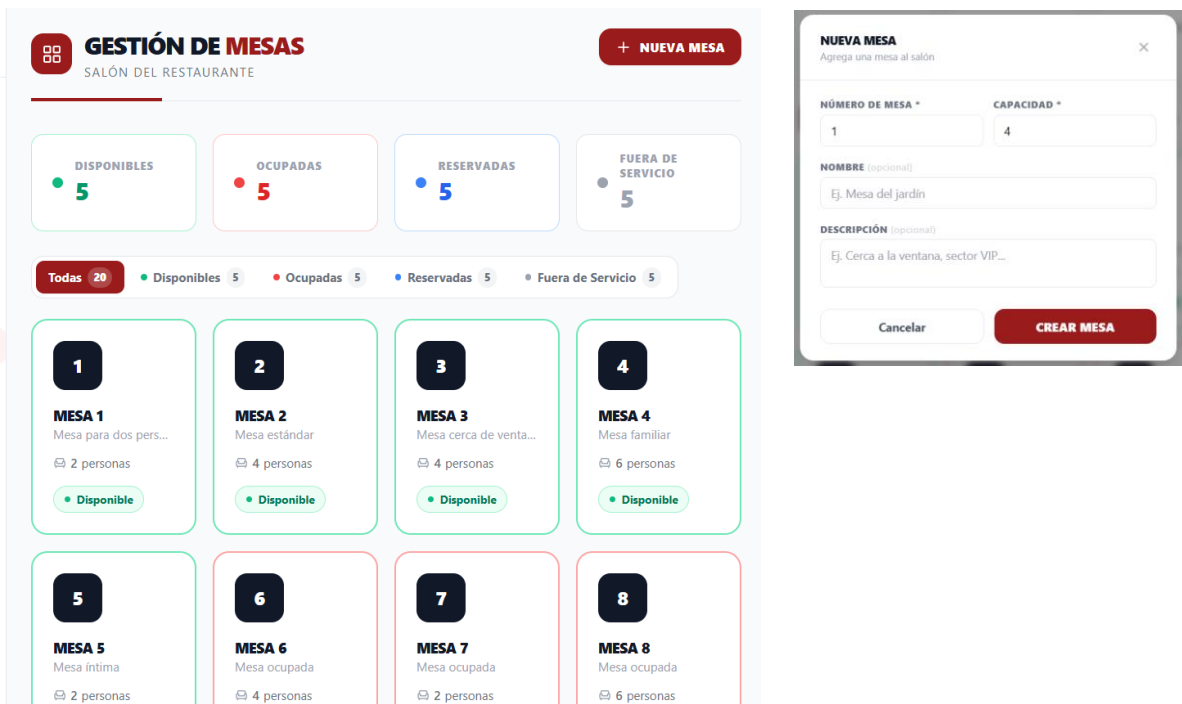
Para el login

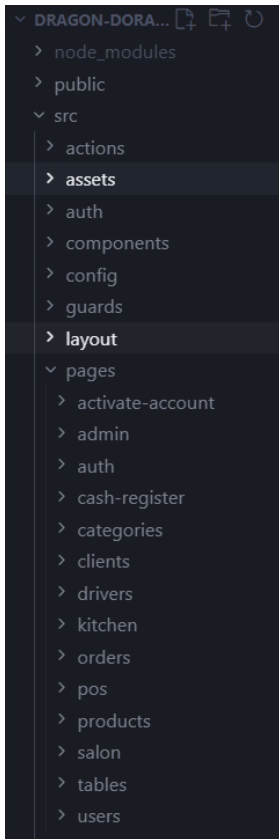


RNF04 – Seguridad: El sistema debe encriptar contraseñas



RNF05 – Escalabilidad: La arquitectura debe estar preparada para añadir más mesas o sucursales en el futuro.





```
src > App.tsx > ...
1 import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
2 import Login from './pages/auth/Login';
3 import Dashboard from './pages/admin/Dashboard';
4
5 import TableMap from './pages/salon/TableMap';
6 import KdsMonitor from './pages/kitchen/KdsMonitor';
7 import DashboardLayout from './layout/DashboardLayout';
8 import DriversPage from './pages/drivers/DriversPage';
9 import TablesPage from './pages/tables/TablesPage';
10 import ClientsPage from './pages/clients/ClientsPage';
11 import UsersPage from './pages/users/UsersPage';
12 import ActivateAccountPage from './pages/activate-account/ActivateAccountPage';
13 import CategoriesPage from './pages/categories/CategoriesPage';
14 import ProductPage from './pages/products/ProductsPage';
15 import CashRegisterPage from './pages/cash-register/CashRegisterPage';
16 import OrdersPage from './pages/orders/OrdersPage';
17 import ProtectedRoute from './guards/ProtectedRoute';
18 import PosPage from './pages/pos/PosPage';
19
20 export default function App() {
21 return (
22 <BrowserRouter>
23 <Routes>
24 <Route path="/" element={<Navigate to="/login" replace />} />
25 <Route path="/activate" element={<ActivateAccountPage />} />
26 <Route path="/login" element={<Login />} />
27
28 <Route element={<ProtectedRoute />>
29 <Route element={<DashboardLayout />>
30 <Route path="/dashboard" element={<Dashboard />} />
31 <Route path="/pedidos" element={<PosPage />} />
32 <Route path="/conductores" element={<DriversPage />} />
33 <Route path="/mesas" element={<TablesPage />} />
34 <Route path="/clientes" element={<ClientsPage />} />
35 <Route path="/usuarios" element={<UsersPage />} />
36 <Route path="/categorias" element={<CategoriesPage />} />
37 <Route path="/productos" element={<ProductsPage />} />
38 <Route path="/cash-register" element={<CashRegisterPage />} />
39 <Route path="/orders" element={<OrdersPage />} />
40 <Route path="/kitchen" element={<KdsMonitor />} />
41 <Route path="/salon" element={<TableMap />} />
42 </Route>
43 </Route>
44 </Routes>
45 </BrowserRouter>
46);
47 }
```

## 7.2. Evidencia de Despliegue en Plataforma Cloud (Versión 1)

El despliegue se realizó en un VPS

### Docker para el back

| CONTAINER ID | IMAGE                                    | COMMAND                  | CREATED        | STATUS                  | PORTS                                       |
|--------------|------------------------------------------|--------------------------|----------------|-------------------------|---------------------------------------------|
| 0ba5a8e0b4d5 | andersonburga/dragon-rojo-backend:latest | "java -jar app.jar"      | 25 minutes ago | Up 25 minutes           | 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp |
| 8bc454d4335e | postgres:16-alpine                       | "docker-entrypoint.s..." | 25 minutes ago | Up 25 minutes (healthy) | 0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp |

### Logs de backend docker

```
root@vm13079813:/opt/janeth# docker logs dba

RESTAURANT BACKEND

restaurant-backend 0.0.1-SNAPSHOT
Powered by Spring Boot 3.5.13
2026-05-18T21:06:56.216Z INFO 1 --- [restaurant-backend] [main] c.a.r.RestaurantBackendApplication : Starting RestaurantBackendApplication v0.0.1-SNAPSHOT using Java 21.0.11 with PID 1 (/app/app.jar started by root in /app)
2026-05-18T21:06:56.221Z DEBUG 1 --- [restaurant-backend] [main] c.a.r.RestaurantBackendApplication : Running with Spring Boot v3.5.13, Spring v6.2.17
2026-05-18T21:07:01.626Z INFO 1 --- [restaurant-backend] [main] c.a.r.RestaurantBackendApplication : The following 1 profile is active: 'dev'
2026-05-18T21:07:01.626Z INFO 1 --- [restaurant-backend] [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-05-18T21:07:01.626Z INFO 1 --- [restaurant-backend] [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring data repository scanning in 319 ms. Found 14 JPA repository interfaces.
2026-05-18T21:07:04.420Z INFO 1 --- [restaurant-backend] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-05-18T21:07:04.455Z INFO 1 --- [restaurant-backend] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-05-18T21:07:04.460Z INFO 1 --- [restaurant-backend] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.33]
2026-05-18T21:07:04.549Z INFO 1 --- [restaurant-backend] [main] o.a.c.c.C.[Tomcat].[localhost].[/app] : Initializing Spring embedded WebApplicationContext
2026-05-18T21:07:04.555Z INFO 1 --- [restaurant-backend] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 7919 ms
2026-05-18T21:07:05.924Z INFO 1 --- [restaurant-backend] [main] o.hibernate.jpa.internal.util.LogHelper : HH0000294: Processing PersistenceUnitInfo [name: default]
2026-05-18T21:07:06.111Z INFO 1 --- [restaurant-backend] [main] org.hibernate.Version : HH000012: Hibernate ORM core version 6.6.45.Final
2026-05-18T21:07:06.254Z INFO 1 --- [restaurant-backend] [main] o.h.c.internal.RegionFactoryInitiator : HH0000026: Second-level cache disabled
2026-05-18T21:07:07.490Z INFO 1 --- [restaurant-backend] [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignoring JPA class transformer
2026-05-18T21:07:07.510Z INFO 1 --- [restaurant-backend] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-05-18T21:07:08.463Z INFO 1 --- [restaurant-backend] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 ~ Added connection org.postgresql.jdbc.PgConnection@27d5eb00
2026-05-18T21:07:08.479Z INFO 1 --- [restaurant-backend] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 ~ Start completed.
2026-05-18T21:07:08.783Z INFO 1 --- [restaurant-backend] [main] org.hibernate.orm.connections.pooling : HH0001005: Database info:
Database driver: undefined/unknown
Database version: 16.14
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-05-18T21:07:14.664Z INFO 1 --- [restaurant-backend] [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
hibernate:
set client_min_messages = WARNING
Hibernate:
alter table if exists account_activation token
drop constraint if exists FK3w80c1pq3n2yafn39j7rh439
Hibernate:
alter table if exists cash_registers
drop constraint if exists FKma20av2a64p13no05ynt988
```



## Logs Base de datos

```
root@vmi3079813:/opt/janeth# docker logs dbc45
PostgreSQL Database directory appears to contain a database; Skipping initialization

2026-05-18 21:06:42.210 UTC [1] LOG: starting PostgreSQL 16.14 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2026-05-18 21:06:42.210 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
2026-05-18 21:06:42.210 UTC [1] LOG: listening on IPv6 address ":::", port 5432
2026-05-18 21:06:42.210 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL_5432"
2026-05-18 21:06:42.226 UTC [26] LOG: database system was shut down at 2026-05-18 21:05:44 UTC
2026-05-18 21:06:42.245 UTC [1] LOG: database system is ready to accept connections
2026-05-18 21:12:42.328 UTC [26] LOG: checkpoint starting: time
2026-05-18 21:12:51.832 UTC [26] LOG: checkpoint complete: wrote 195 buffers (1.2%); 0 WAL file(s) added, 0 removed, 0 recycled; write=19.358 s, sync=0.093 s, total=19.595 s; sync files=110, longest=0.014 s, average=0.001 s; distance=895 kB, estimate=895 kB; lsn=0/1B22880, redo lsn=0/1B227C8
root@vmi3079813:/opt/janeth#
```

## Repositorio frontend nginx

```
root@vmi3079813:/etc/nginx/sites-available# cat restaurant-frontend
server {
 listen 8081;

 server_name tu_dominio.com;

 location / {
 root /var/www/restaurant-frontend/dist;
 index index.html;
 try_files $uri $uri/ /index.html;
 }
}
root@vmi3079813:/etc/nginx/sites-available#
```

## Configuración de frontend nginx

```
root@vmi3079813:/etc/nginx/sites-available# cat restaurant-frontend
server {
 listen 8081;

 server_name tu_dominio.com;

 location / {
 root /var/www/restaurant-frontend/dist;
 index index.html;
 try_files $uri $uri/ /index.html;
 }
}
root@vmi3079813:/etc/nginx/sites-available#
```

## Estado activo nginx

```
root@vmi3079813:/etc/nginx/sites-available# sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
 Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
 Active: active (running) since Mon 2026-05-18 23:04:44 CEST; 31min ago
 Docs: man:nginx(8)
 Process: 2674476 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Process: 2674478 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 2674480 (nginx)
 Tasks: 5 (limit: 9483)
 Memory: 4.1M (peak: 4.9M)
 CPU: 126ms
 CGroup: /system.slice/nginx.service
 └─2674480 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
 └─2674481 "nginx: worker process"
 └─2674482 "nginx: worker process"
 └─2674483 "nginx: worker process"
 └─2674484 "nginx: worker process"

May 18 23:04:44 vmi3079813 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
May 18 23:04:44 vmi3079813 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
root@vmi3079813:/etc/nginx/sites-available#
```



### 7.3. Retrospectiva del Sprint 4

Al concluir el Sprint 4, el cual estuvo enfocado en la fase final de integración, pruebas masivas del sistema y el despliegue de la versión 1 (V1) en el entorno de producción, el equipo de desarrollo llevó a cabo la sección de retrospectiva basada en la metodología ágil Scrum. El objetivo de esta sesión fue evaluar nuestra dinámica de trabajo, identificar los aciertos tecnológicos y reconocer los puntos críticos que requieren optimización en las futuras iteraciones del sistema.

A continuación, se detallan las conclusiones de la retrospectiva estructuradas bajo los tres ejes fundamentales del marco ágil:

#### 1. ¿Qué funcionó bien en este sprint? (Aciertos)

- **Despliegue Cloud Exitoso y Portabilidad:** La estrategia de contenerización con Docker y Docker Compose para el back-end (Spring Boot 3.5.13) y la base de datos (PostgreSQL 16) garantizó una transición sin fricciones desde el entorno de desarrollo local hacia el servidor de producción VPS. Los logs del contenedor evidencian que la aplicación inició de manera limpia bajo la arquitectura Alpine.
- **Rendimiento Óptimo del Servidor (RNF03):** Las pruebas no funcionales de rendimiento arrojaron métricas excelentes, logrando un tiempo de respuesta de apenas 323.83 ms en la carga y verificación de solicitudes críticas como el inicio de sesión (*Login*), cumpliendo holgadamente con los estándares definidos en los SLA.
- **Estabilidad del Servidor Web de Fachada:** La configuración de Nginx como proxy inverso para servir el front-end construido en React.js demostró una alta disponibilidad. El estado del servicio se mantuvo 100% activo y estable durante todas las pruebas de carga iniciales.
- **Integración de Seguridad Robusta:** Se validó correctamente el ciclo de vida stateless de los tokens JWT (firmados mediante el algoritmo HS256 con Nimbus JOSE) y el cifrado de credenciales en la base de datos, mitigando las vulnerabilidades críticas del entorno web.

#### 2. ¿Qué no funcionó tan bien o presentó dificultades? (Cuellos de Botella)

- **Complejidad en la Sincronización de Entornos:** Durante las primeras horas del despliegue, se presentaron ligeros descuadres en las variables de entorno relacionadas con los perfiles de ejecución (application-dev.yaml y application-prod.yaml), específicamente en los puertos de escucha asignados a PostgreSQL.

(5434 mapeado internamente al 5432), lo que requirió una reconfiguración rápida del archivo `docker-compose.yml`.

- **Carga Inicial de Assets Dinámicos:** Aunque la arquitectura cloud contempla el uso de Cloudinary para las imágenes del menú, la velocidad de renderizado inicial en dispositivos móviles (tablets de los mozos) experimentó una ligera latencia antes de activarse el mecanismo de *lazy loading* y *caching* del front-end.
- **Curva de Adaptación del Entorno Táctil:** Al realizar las simulaciones de la comanda digital simulando alta presión en salón, se identificó que un par de componentes de la interfaz responsiva de React requerían un área de clic (*padding*) marginalmente mayor para evitar errores accidentales de selección por parte de los mozos.

### 3. Compromisos de Acción para Sprints Futuros (Plan de Mejora)

- **Automatización del Pipeline (CI/CD):** Para mitigar el riesgo de errores en despliegues manuales (identificado en nuestra matriz de riesgos), nos comprometemos a implementar GitHub Actions en el repositorio para automatizar las pruebas unitarias con JUnit 5 y el despliegue automático en el VPS tras cada *merge* aprobado en la rama main.
- **Optimización de la Política de Backups Semánticos:** Si bien se validó que el volumen de datos de Docker (`db_data`) persiste correctamente de forma aislada, es imperativo automatizar mediante un *script* de *cron-job* el volcado diario (`pg_dump`) hacia un almacenamiento externo como un Bucket S3 de AWS para asegurar la continuidad del negocio ante desastres.
- **Ajustes de UX en el Módulo de Salón:** Modificar el archivo `App.tsx` y las hojas de estilos de Tailwind CSS en las vistas del mapa de mesas y comanda digital para expandir las zonas interactivas táctiles de las tarjetas de los platos, garantizando que el sistema sea extremadamente fluido bajo el entorno de alta rotación del restaurante.

## CAPÍTULO VIII: GESTIÓN DE SOLICITUDES Y CAMBIOS

### 8.1. Catálogo de Solicitudes de Servicio

El presente catálogo define formalmente el portafolio de servicios tecnológicos y solicitudes estándar disponibles para los clientes que adopten el Sistema de Gestión de Pedidos Gastronómicos. Al operar bajo un modelo de niveles de servicio (SLA), las solicitudes se encuentran estandarizadas para garantizar la predictibilidad, disponibilidad (99.9%) y continuidad operativa de los establecimientos comerciales.

| Línea de Servicio                   | Código     | Nombre del Servicio / Requerimiento                 | Descripción y Alcance                                                                                                                                                                          | Tiempo de Respuesta (SLA) |
|-------------------------------------|------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| 1. Gestión de Accesos e Identidades | REQ-SEC-01 | Soporte para Alta, Baja o Modificación de Usuarios  | Aprovisionamiento de credenciales para nuevo personal con asignación de roles basados en control de accesos.                                                                                   | Hasta 24 horas            |
|                                     | REQ-SEC-02 | Blanqueo / Reseteo de Contraseñas                   | Restablecimiento forzado de credenciales de acceso por bloqueo de cuenta, olvido o políticas de seguridad interna del cliente.                                                                 | Hasta 4 horas             |
| 2. Soporte Operativo y Datos        | REQ-OPS-01 | Configuración y Modificación de Parámetros de Salón | Actualizaciones estructurales en el aplicativo que no implican código, tales como alteración del mapa gráfico de mesas, cambios en la capacidad de comensales o distribución física del salón. | Hasta 48 horas            |
|                                     | REQ-OPS-02 | Soporte en Mantenimiento y Carga de Catálogo/Carta  | Soporte masivo para la actualización de precios, categorías de productos gastronómicos, carga de imágenes en el repositorio e indexación en el menú digital.                                   | Hasta 48 horas            |

|                                                |            |                                                                       |                                                                                                                                                                                      |                                        |
|------------------------------------------------|------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
|                                                | REQ-OPS-03 | <b>Fallas Críticas</b>                                                | Si el sistema se cae por completo, emergencia crítica que impide el funcionamiento del sistema                                                                                       | Hasta 4 horas                          |
| <b>3. Consultas e Información</b>              | REQ-INF-01 | <b>Atención de Consultas de Usuario Final</b>                         | Asistencia e inducción rápida en el uso de los módulos del sistema (Interfaz táctil, Monitor KDS, panel de reportes de caja) para personal técnico o administrativo.                 | Respuesta Instantánea                  |
|                                                | REQ-INF-02 | <b>Extracción y Consolidación de Reportes Especiales</b>              | Generación o exportación manual de históricos de transacciones financieras, cierres de caja o logs de auditoría que excedan las pantallas estándar del Dashboard de la app.          | Hasta 24 horas                         |
| <b>4. Requerimientos de Evolución y Ventas</b> | REQ-VTA-01 | <b>Cotización de Implementación de Nuevos Módulos o Integraciones</b> | Evaluación técnica y comercial para la activación de características excluidas en la fase inicial, tales como pasarelas de pago automatizadas o APIs de terceros (Rappi, PedidosYa). | Hasta 48 horas (Emisión de Cotización) |
|                                                | REQ-VTA-02 | <b>Cotización de Despliegue e Interconexión de Nuevas Sucursales</b>  | Aprovisionamiento de infraestructura cloud adicional (VPS, BD relacional aislada o multitenant) para expandir el sistema hacia un nuevo local físico del cliente.                    | Hasta 5 días hábiles                   |

## 8.2. Proceso de Gestión de Cambios (infraestructura y Aplicaciones)

### 1. Objetivos del proceso

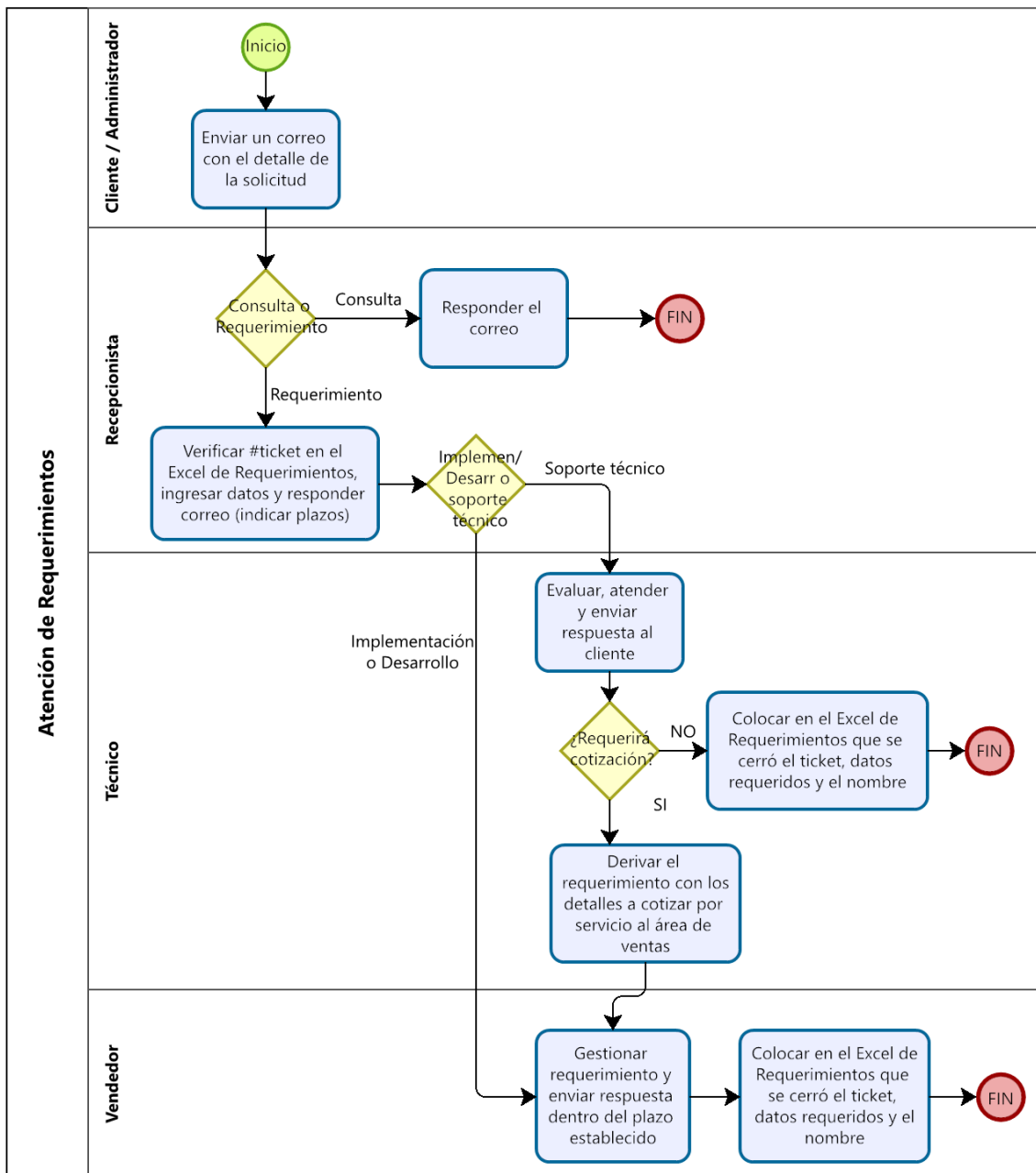
Garantizar una atención oportuna, ordenada y eficiente de todas las consultas, incidentes y solicitudes de implementación solicitadas por los usuarios del sistema, asegurando el cumplimiento de los tiempos de respuesta (SLA) establecidos y el registro centralizado de cada caso.

### 2. Matriz de roles, responsabilidades y controles

| <b>Rol</b>                     | <b>Responsabilidad principal</b>                                                                                                                                                                  |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cliente / Administrador</b> | <b>Genera y envía el detalle de la solicitud o consulta vía correo electrónico.</b>                                                                                                               |
| <b>Recepcionista</b>           | <b>Centraliza la recepción de correos, evalúa el tipo de solicitud, asigna el número de ticket correlativo en el control de requerimientos y deriva al área correspondiente.</b>                  |
| <b>Técnico</b>                 | <b>Evalúa, atiende y resuelve los requerimientos de soporte técnico; identifica si el servicio requiere una cotización adicional y registra el cierre del ticket.</b>                             |
| <b>Vendedor</b>                | <b>Evalúa las solicitudes de nuevas implementaciones, desarrollos o servicios adicionales, elabora y envía la cotización comercial al cliente, y registra el cierre del ticket en el control.</b> |

- A. Validaciones requeridas: Evaluación inicial de la solicitud para clasificarla correctamente como consulta, soporte o desarrollo comercial.
- B. Registros o evidencias generadas: Uso obligatorio del Excel de Requerimientos para registrar el número de ticket correlativo, datos de la solicitud, estado (abierto/cerrado) y el nombre del responsable de la atención para el seguimiento o si posteriormente se requiere de una auditoría.

### 3. Diagrama de Flujo de actividades



### 4. Pasos del procedimiento

| DESCRIPCIÓN DEL PROCEDIMIENTO |                                                                                                                                                                                                                     |               |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| Nº                            | Descripción de la Actividad                                                                                                                                                                                         | Persona       |
| 01                            | Enviar un correo con el detalle de la solicitud.                                                                                                                                                                    | Cliente       |
| 02                            | Recepcionar correo y evaluar<br><b>¿Consulta que se puede responder al instante o es un requerimiento?</b><br><b>Consulta</b> = Continuar a la actividad 03.<br><b>Requerimiento</b> = Continuar a la actividad 04. | Recepcionista |

|    |                                                                                                                                                                                                                                                                         |               |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 03 | Responder el correo<br><b>Fin del procedimiento.</b>                                                                                                                                                                                                                    | Recepcionista |
| 04 | Verificar en el Excel de Requerimientos el número de ticket correlativo que le corresponde, ingresar los datos y responder al correo indicando ello junto con los plazos de atención según corresponda (según los niveles definidos en el catálogo de servicios).       | Recepcionista |
| 05 | <b>¿Será una nueva implementación/desarrollo o es para soporte técnico?</b><br><b>Implementación o desarrollo</b> = Continuar a la actividad 06.<br><b>Soporte técnico</b> = Continuar a la actividad 09.                                                               | Recepcionista |
| 06 | Derivar la solicitud al área de ventas para evaluación y cotización.                                                                                                                                                                                                    | Recepcionista |
| 07 | Gestionar el requerimiento y enviar la respuesta al cliente dentro del plazo establecido (Hasta 48 horas para propuestas comerciales) e indicando que la propuesta deberá ser respondida al mismo correo del agente de ventas dentro del plazo válido de la cotización. | Vendedor      |
| 08 | Colocar en el Excel de Requerimientos que se cerró el ticket, datos requeridos y el nombre de qué vendedor lo atendió.<br><b>Fin del procedimiento.</b>                                                                                                                 | Vendedor      |
| 09 | Derivar la solicitud al área de soporte técnico para evaluación, atención y respuesta.                                                                                                                                                                                  | Recepcionista |
| 10 | Evaluar, atender y enviar respuesta al cliente.<br><b>¿Requerirá cotización por servicio?</b><br><b>Si</b> = Continuar con la actividad 11.<br><b>No</b> = Continuar con la actividad 12.                                                                               | Técnico       |
| 11 | Derivar el requerimiento con los detalles a cotizar por servicio al área de ventas.<br>Volver a la actividad 07.                                                                                                                                                        | Técnico       |
| 12 | Colocar en el Excel de Requerimientos que se cerró el ticket, datos requeridos y el nombre de qué técnico lo atendió.<br><b>Fin del procedimiento.</b>                                                                                                                  | Técnico       |

## 5. Indicadores

- % de Cumplimiento de SLA: (Tickets atendidos dentro del plazo establecido /

Total de tickets recibidos) x 100.

- Tiempo Promedio de Asignación: Tiempo transcurrido desde que llega el correo hasta que el recepcionista asigna el ticket y deriva al área responsable.
- Tasa de Conversión de Nuevos Requerimientos: (Solicitudes de implementación cotizadas que fueron aprobadas por el cliente / Total de solicitudes derivadas a ventas) x 100.



## CAPÍTULO IX: INGENIERÍA DE LA PLATAFORMA

### 9.1. Pipeline de Integración Continua (CI) y Herramientas

El ecosistema de herramientas e infraestructura seleccionado para habilitar el pipeline de automatización del proyecto se compone de los siguientes elementos técnicos:

- **Repositorio en GitHub:** Contiene el código fuente del frontend React en su rama principal main.
- **GitHub Actions:** Motor de automatización que ejecuta un workflow cada vez que se realiza un push a la rama main.
- **Conexión SSH:** GitHub Actions se conecta de forma remota al VPS utilizando credenciales almacenadas como Secrets.
- **Runner virtual (ubuntu-latest):** Entorno de ejecución proporcionado por GitHub sobre el cual corre el job de despliegue.
- **Action appleboy/ssh-action@v1.2.0:** Herramienta utilizada para establecer la conexión SSH hacia el servidor remoto y ejecutar el script de comandos directamente en él.
- **GitHub Secrets:** Mecanismo diseñado específicamente para información sensible que enmascara automáticamente los datos en los logs de ejecución (mostrando asteriscos), garantizando que datos críticos no queden expuestos.

#### Configuración de Parámetros Seguros (Secrets)

Para resguardar la información crítica de la infraestructura, se crearon los siguientes Secrets en el repositorio de GitHub:

| Nombre del Secret | Descripción                                            | Valor / Contenido                                                    |
|-------------------|--------------------------------------------------------|----------------------------------------------------------------------|
| VPS_HOST          | Dirección IP del VPS al que se conecta GitHub Actions. | Dirección IP del servidor (información sensible de infraestructura). |
| VPS_USER          | Usuario del sistema utilizado para la conexión SSH.    | Nombre de usuario (por ejemplo, root).                               |
| VPS_SSH_KEY       | Llave privada SSH                                      | Contenido completo de la                                             |

| Nombre del Secret | Descripción                                               | Valor / Contenido                         |
|-------------------|-----------------------------------------------------------|-------------------------------------------|
|                   | utilizada por GitHub Actions para autenticarse en el VPS. | llave privada generada (formato ed25519). |

A diferencia de las variables estándar que son visibles en los logs, el uso de Secrets garantiza que la IP, el usuario y la llave privada estén protegidos contra accesos no autorizados, ya que constituyen información crítica de seguridad.

## 9.2. Documentación de Automatización de Despliegues

### 9.2.1. Contexto Inicial de la Infraestructura

Antes de implementar el pipeline de CI/CD, el entorno presentaba las siguientes características:

- El proyecto frontend (React) ya se encontraba clonado en el VPS, en la ruta `/var/www/restaurant-frontend`.
- Nginx ya estaba previamente configurado para servir el contenido estático desde la carpeta `dist` del proyecto.
- No era necesario copiar archivos manualmente con `cp`, ya que el build se genera directamente dentro del propio directorio del proyecto, en la carpeta que Nginx ya apunta.
- El acceso al VPS se realizaba originalmente mediante usuario y contraseña, por lo que no existían llaves SSH configuradas para automatización.
- Node.js y npm estaban instalados en el VPS, pero gestionados a través de `nvm` (Node Version Manager), bajo el usuario `root`.

### 9.2.2. Flujo de Automatización Esperado

Ante cada push a la rama `main`, el pipeline ejecuta automáticamente la secuencia minimalista de comandos necesarios en el VPS para mantener el sitio actualizado:

1. `cd /var/www/restaurant-frontend` — Accede al directorio del proyecto.
2. `git pull origin main` — Trae los últimos cambios del código fuente.
3. `npm install` — Instala o actualiza las dependencias del proyecto según el `package.json` y `package-lock.json`.
4. `npm run build` — Genera la nueva versión de producción dentro de la carpeta `dist`, que es la carpeta que Nginx sirve directamente.

### 9.2.3. Configuración de Accesos No Interactivos (SSH)

La configuración de acceso SSH se realizó siguiendo el modelo de autenticación por par de llaves (pública/privada):

- **Generación del par de llaves en el VPS:** Se generó un nuevo par de llaves SSH directamente en el VPS, identificándolo de forma descriptiva para asociarlo al propósito de despliegue automatizado:  
`ssh-keygen -t ed25519 -C "github-actions-restaurant-frontend" -f ~/.ssh/github_actions_restaurant`

- **Autorización de la llave pública y ajuste de permisos:** Se agregó la llave pública al archivo de llaves autorizadas del usuario en el VPS y se configuraron permisos estrictos para cumplir con las especificaciones de seguridad requeridas por el demonio SSH: `cat ~/.ssh/github_actions_restaurant.pub >> ~/.ssh/authorized_keys`  
`chmod 700 ~/.ssh chmod 600 ~/.ssh/authorized_keys`
- **Registro de la llave privada:** El contenido obtenido mediante `cat ~/.ssh/github_actions_restaurant` se copió y se pegó por completo como valor del `Secret VPS_SSH_KEY` en la configuración del repositorio de GitHub.

#### 9.2.4. Mitigación de Errores y Diagnóstico Técnico

Durante la implementación del pipeline se presentaron y resolvieron dos problemas principales relacionados con las particularidades del entorno del VPS:

1. **Fallo de autenticación SSH (handshake failed):** Provocado porque el archivo `~/.ssh/authorized_keys` estaba inicialmente vacío debido a que el servidor se gestionaba por contraseña. Se solucionó estableciendo el esquema de autenticación por par de llaves asimétricas dedicado para GitHub Actions, poblando la clave pública correspondiente y configurando los accesos seguros indicados.
2. **Error de comando no encontrado (npm: command not found):** Ocurría porque las sesiones SSH de GitHub Actions para ejecutar scripts remotos operan de forma no interactiva (sin shell de login completo). Esto causa que se omita la carga automática de los archivos de inicialización (`~/.bashrc` o `~/.profile`) donde reside la configuración de `nvm`. Se solucionó forzando la carga manual de variables y scripts de inicialización de `nvm` dentro del script del workflow antes de invocar los comandos de despliegue: `export NVM_DIR="/root/.nvm" [ -s "$NVM_DIR/nvm.sh" ] && . "$NVM_DIR/nvm.sh"`

#### 9.2.5. Archivo de Workflow Final (deploy-frontend.yml)

A continuación, se presenta el contenido final del archivo `.github/workflows/deploy-frontend.yml`, incorporando todas las correcciones descritas anteriormente:

```
YAML
name: Deploy Frontend
on:
 push:
 branches:
 - main
jobs:
 deploy:
 runs-on: ubuntu-latest

 steps:
 - name: Deploy en VPS
 uses: appleboy/ssh-action@v1.2.0
 with:
 host: ${ secrets.VPS_HOST }
 username: ${ secrets.VPS_USER }
 key: ${ secrets.VPS_SSH_KEY }
 script: |
```

```
set -e
export NVM_DIR="/root/.nvm"
[-s "$NVM_DIR/nvm.sh"] && . "$NVM_DIR/nvm.sh"
```

```
cd /var/www/restaurant-frontend
git pull origin main
npm install
npm run build
```

**Nota técnica de estabilidad (set -e):** La inclusión de set -e es una opción de configuración de bash que hace que el script se detenga inmediatamente si cualquiera de los comandos que lo componen falla. Esto actúa como un control de seguridad que evita que el pipeline continúe con la ejecución sobre un estado del código potencialmente inconsistente o incompleto.

#### 9.2.6. Validación del Despliegue

Tras aplicar ambas correcciones, se validó el funcionamiento de extremo a extremo mediante un push a la rama main: GitHub Actions inicia el workflow automáticamente, la action establece la conexión remota vía SSH, se actualiza el repositorio con git pull, se inicializa el entorno de Node/npm para la correcta ejecución de npm install y npm run build, y los cambios quedan reflejados en vivo en la carpeta dist. Dado que Nginx sirve directamente este directorio, las actualizaciones se visualizan de inmediato en el navegador sin intervenciones manuales ni necesidad de reiniciar el servidor web.

The screenshot shows the GitHub interface for the repository 'dragon-dorado-frontend'. The repository is public and has 8 branches and 0 tags. The main branch is selected. The file list shows various files and folders, including .github/workflows, public, src, .env.template, .gitignore, README.md, eslint.config.js, index.html, package-lock.json, package.json, pnpm-lock.yaml, postcss.config.js, tailwind.config.js, tsconfig.json, vite-env.d.ts, and vite.config.js. The commit history shows the latest commit by janetheeves, titled 'Load nvm before deploy build', which was made 2 minutes ago and has 35 commits.

| File               | Commit Message                                | Time          |
|--------------------|-----------------------------------------------|---------------|
| .github/workflows  | Load nvm before deploy build                  | 2 minutes ago |
| public             | Interfaces del Sprint 1 (Login, Dashboar...   | 2 months ago  |
| src                | feat(ui): mejorar navegación y scroll hori... | 7 hours ago   |
| .env.template      | add axios configuration                       | last month    |
| .gitignore         | add axios configuration                       | last month    |
| README.md          | docs: corregir error de formato markdo...     | 2 months ago  |
| eslint.config.js   | Interfaces del Sprint 1 (Login, Dashboar...   | 2 months ago  |
| index.html         | Interfaces del Sprint 1 (Login, Dashboar...   | 2 months ago  |
| package-lock.json  | feat(sidebar): hacer clickeable el logo pa... | last month    |
| package.json       | feat: protect routes and add initial confi... | last month    |
| pnpm-lock.yaml     | feat: protect routes and add initial confi... | last month    |
| postcss.config.js  | Interfaces del Sprint 1 (Login, Dashboar...   | 2 months ago  |
| tailwind.config.js | Interfaces del Sprint 1 (Login, Dashboar...   | 2 months ago  |
| tsconfig.json      | add typescript, change jsx for tsx, add ac... | last month    |
| vite-env.d.ts      | add axios configuration                       | last month    |
| vite.config.js     | feat: exqueleton for pos orders               | last month    |

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

← Deploy Frontend

✓ Load nvm before deploy build #2

Re-run all jobs

...

Summary

All jobs

deploy

Run details

Usage

Workflow file

deploy

succeeded now in 15s

Search logs

⚙

> ✓ Set up job1s

✓ Deploy en VPS13s

1 ▶ Run appleboy/ssh-action@v1.2.0

22 ▶ Run echo "\$GITHUB\_ACTION\_PATH" >> \$GITHUB\_PATH

27 ▶ Run entrypoint.sh

73 Will download drone-ssh-1.8.0-linux-amd64 from <https://github.com/appleboy/drone-ssh/releases/download/v1.8.0>

74 ===== CLI Version =====

75 Drone SSH version 1.8.0

76 =====

77 From github.com:AngelSalguero8/dragon-dorado-frontend

78 \* branch main -> FETCH\_HEAD

79 8fcb56d..9483537 main -> origin/main

80 Updating 8fcb56d..9483537

81 Fast-forward

82 .github/workflows/deploy-frontend.yml | 3 +++

83 1 file changed, 3 insertions(+)

84 up to date, audited 262 packages in 3s

85 62 packages are looking for funding

86 run `npm fund` for details

87 2 high severity vulnerabilities

88 To address all issues, run:

89 npm audit fix

90 Run `npm audit` for details.

91 > dragon-dorado-frontend@0.0.0 build

92 > vite build

93 vite v8.0.9 building client environment for production...

94

95 transforming...✓ 1988 modules transformed.

96 rendering chunks...

97 computing gzip size...

98 dist/index.html0.47 kB | gzip: 0.30 kB

99 dist/assets/index-NFihCTa5.css39.75 kB | gzip: 7.20 kB

100 dist/assets/index-7qncO\_U2.js593.02 kB | gzip: 159.47 kB

101 ✓ built in 4.52s

102 [plugin builtin:vite-reporter]

103 (!) Some chunks are larger than 500 kB after minification. Consider:

104 - Using dynamic import() to code-split the application

105 - Use build.rollupOptions.output.codeSplitting to improve chunking: <https://rollupjs.org/reference/OutputOptions/codeSplitting>

106 - Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.

107 =====

108 ✓ Successfully executed commands to all hosts.

109 =====

> ✓ Complete job0s

## CAPÍTULO X: GESTIÓN DE CAPACIDAD Y RENDIMIENTO

### 10.1. Análisis de Capacidad de Servicio

| Componente                              | Capacidad                                                                                                              | Uso actual                                                                                         | % Utilización                                              |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Aplicación (Backend Spring Boot)</b> | Puerto 8080, JVM Amazon Corretto 21, Alpine Linux, perfil prod activo                                                  | Procesa solicitudes REST y WebSocket; HikariPool activo para conexiones BD                         | ~30–50% en hora punta (estimado)                           |
| <b>APIs REST</b>                        | 11 módulos: Auth, Users, Orders, Products, Categories, Clients, Drivers, Addresses, Tables, Cash, Dashboard            | Endpoints activos en producción; Swagger UI disponible en /api/docs                                | Latencia promedio registrada: 323.83 ms (pruebas de login) |
| <b>Base de datos (PostgreSQL 16)</b>    | Contenedor postgres:16-alpine, puerto externo 5434 → interno 5432; volumen Docker db_data; buffers: 195 buffers (1.2%) | Escucha en IPv4 (0.0.0.0) e IPv6; sincronización de archivos: 110 sync files; HikariPool conectado | Bajo (uso operacional inicial, sin saturación detectada)   |
| <b>Servidor web</b>                     | Puerto 8081; archivos servidos desde /var/www/restaurant-frontend/dist; SPA con regla try_files                        | Active: active (running); estable durante todas las pruebas de carga                               | Memoria: 4.1 MB RAM (extremadamente eficiente)             |

## Infraestructura (VPS Contabo)

| CPU                                                                                                    | Memoria RAM                                                                                                | Almacenamiento                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| VPS Contabo (especificaciones del plan contratado); contenedores en Alpine para mínimo overhead de CPU | Nginx estabilizado en 4.1 MB; Spring Boot (JVM) con HikariPool; total disponible según plan VPS contratado | Volumen Docker db_data para persistencia de PostgreSQL; backup diario con pg_dump; alerta por crecimiento de logs de contenedores detectada en Sprint 6 |

## 10.2. Planificación y Herramientas de Monitoreo de Rendimiento

### KPIs definidos

| ID     | Indicador                           | Descripción                                                                                                           | Meta                  | Línea base                    |
|--------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------|-------------------------------|
| KPI-01 | Tasa de error en pedidos            | Pedidos con datos incorrectos / total pedidos del día. Medición: correcciones registradas en sistema                  | < 0.5% al mes 3       | ~15% (comandas físicas)       |
| KPI-02 | Tiempo promedio de atención (salón) | Desde que el cliente ordena hasta que el plato llega a mesa. Timestamp comanda enviada vs. estado 'Listo para servir' | ≤ 12 min (hora punta) | ~18 min estimado              |
| KPI-03 | Tiempo de despacho delivery         | Desde registro del pedido hasta que el motorizado sale del local. Timestamp registro vs. estado 'Despachado'          | ≤ 20 min              | Sin referencia digital previa |
| KPI-04 | Tiempo de cierre de caja            | Desde apertura del proceso de cuadre en el sistema hasta su finalización. Registro inicio y fin del módulo de caja    | ≤ 10 min              | 25 min (proceso manual)       |
| KPI-05 | Satisfacción del usuario interno    | Encuesta post-implementación a mozos, cocineros y cajeros (escala 1–5). Frecuencia: mensual (primeros 3 meses)        | ≥ 4.2 / 5             | Sin referencia previa         |
| KPI-06 | Tasa de adopción del sistema        | % del personal que usa el sistema activamente (≥ 1 acción por turno) Medición: log de sesiones activas                | 80% sem1 → 100% sem3  | 0% (proceso manual)           |

|                                       |                                                                                                                                                                                                                  |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SLA — Disponibilidad</b>           | ≥ 99.9% de uptime durante horario comercial                                                                                                                                                                      |
| <b>SLA — Latencia comanda</b>         | ≤ 2 segundos desde envío al servidor hasta recepción en monitor KDS (WebSocket)                                                                                                                                  |
| <b>SLA — Generación de reportes</b>   | ≤ 5 segundos para carga del dashboard gerencial                                                                                                                                                                  |
| <b>SLA — Resolución de incidentes</b> | ≤ 4 horas para fallas críticas                                                                                                                                                                                   |
| <b>Estado actual</b>                  | Backend Spring Boot: 323.83 ms promedio en solicitudes críticas (login / verificación JWT) — cumple holgadamente SLA. Nginx: 4.1 MB RAM, estado active (running) sin interrupciones detectadas                   |
| <b>Alertas recientes</b>              | Crecimiento de logs Docker durante operación continua (Sprint 6). Variables de entorno en puertos PostgreSQL durante primer despliegue (5434 → 5432) — resuelto con reconfiguración de docker-compose.yml        |
| <b>Herramientas de monitoreo</b>      | systemctl status nginx (estado del servidor web) · docker logs (auditoría de transacciones del backend) · pg_dump (backup manual de BD) · Planificado: Prometheus + Grafana para métricas automáticas de CPU/RAM |

### 10.3. Retrospectiva del Sprint 6

Al concluir el Sprint 6, el cual estuvo enfocado en la estabilización de la plataforma, la auditoría del rendimiento de la base de datos PostgreSQL 16 y la validación de la capacidad de respuesta del servidor VPS Contabo bajo escenarios de uso continuo, el equipo de desarrollo llevó a cabo la sesión de retrospectiva basada en el marco ágil Scrum. El objetivo de esta sesión fue evaluar la dinámica de trabajo del equipo, consolidar los logros en la ingeniería de la plataforma e identificar los puntos críticos finales para garantizar la continuidad operativa multi-cliente.

A continuación, se detallan las conclusiones de la retrospectiva estructuradas bajo los tres ejes fundamentales del marco ágil:

#### 1. ¿Qué funcionó bien en este sprint? (Aciertos)

- Estabilidad y Consumo Eficiente de Recursos: Las herramientas de monitoreo nativas del VPS demostraron que el servidor web Nginx mantuvo una alta disponibilidad sin registrar fugas de memoria, estabilizándose en un consumo sumamente bajo (4.1 MB de RAM). Esto confirma que la



arquitectura ligera elegida (Docker + Alpine Linux) es óptima para entornos de producción de bajo costo pero alta concurrencia.

- Comportamiento Excepcional de la Capa de Datos: La base de datos PostgreSQL 16 respondió de manera sólida ante las solicitudes síncronas de múltiples clientes simultáneos, manteniendo una correcta sincronización de los archivos de control operativo (110 sync files) y una administración eficiente en la precarga de buffers de escritura (195 buffers, 1.2%).
- Tiempos de Respuesta dentro del SLA: El backend (Spring Boot 3.5) ejecutándose bajo el perfil de producción (prod) logró procesar las solicitudes críticas —como la autenticación y verificación de tokens JWT— en un promedio de 323.83 ms. Este rendimiento cumple holgadamente con los límites establecidos en los acuerdos de nivel de servicio corporativos (< 2 segundos para el envío de comanda).

## **2. ¿Qué no funcionó tan bien o presentó dificultades? (Cuellos de Botella)**

- Latencia Acumulada por Crecimiento de Logs: Al mantener el sistema operando de forma continua en pruebas de carga, se identificó que el crecimiento desmedido de los logs de auditoría interna de los contenedores Docker generó un consumo innecesario de almacenamiento en el disco del VPS, lo que podría saturar la infraestructura y ralentizar el análisis de eventos a largo plazo.
- Complejidad en el Monitoreo de Redes Híbridas: Aunque el motor transaccional quedó parametrizado correctamente para escuchar peticiones simultáneas en IPv4 e IPv6 (listening on 0.0.0.0), se experimentaron ligeras dificultades de configuración en las reglas de firewall del VPS para auditar limpiamente el tráfico proveniente de conexiones móviles externas.
- Desalineación de Parámetros en el Primer Despliegue: Durante las fases operacionales iniciales, se detectó un problema de configuración en las variables de entorno relativas a los puertos de la base de datos PostgreSQL (mapeo externo 5434 hacia el interno 5432), lo cual generó un fallo temporal de conexión antes de ser corregido en el archivo docker-compose.yml.

## **3. Compromisos de Acción para Sprints Futuros (Plan de Mejora)**

- Configuración de Logrotate en Docker Daemon: Implementar de forma prioritaria políticas de rotación en el demonio de Docker del VPS, limitando el tamaño de los archivos a un máximo de 10 MB y reteniendo un límite de 3 archivos por contenedor para evitar cuellos de botella por

almacenamiento.

- Automatización de Alertas Tempranas de Rendimiento: Integrar herramientas avanzadas de recolección de métricas automáticas (Prometheus y Grafana) para disparar notificaciones directas al canal técnico cuando el uso de CPU o RAM del contenedor de PostgreSQL supere el 80% durante las horas punta del restaurante.
- Estandarización del Script de Backups y Redes: Consolidar en la infraestructura un script automatizado vía cron-job para realizar volcados diarios seguros de la base de datos (pg\_dump) y transferirlos a almacenamiento externo tipo S3 de AWS, a la par de una revisión estructural de las reglas del firewall del VPS para auditar limpiamente las conexiones móviles externas.

## CONCLUSIONES

- El análisis del contexto empresarial del Restaurante Dragón Dorado evidencia que la dependencia de procesos manuales y comandas físicas genera errores operativos críticos (tasa de error inicial estimada en ~15%), retrasos en la comunicación con cocina y una falta total de control financiero, lo cual justifica plenamente la centralización y automatización de la comanda digital y el control de caja a través de la solución informática implementada.
- La aplicación del framework Scrum con sprints de tres semanas y un equipo de cinco integrantes con roles definidos (Product Owner, Scrum Master y Desarrolladores backend, frontend y QA/Cloud) garantizó una gestión ágil y adaptable del proyecto, facilitando la entrega incremental de valor reflejada en la estabilización y auditoría del rendimiento de la plataforma lograda hacia el Sprint 6.
- Los Acuerdos de Nivel de Servicio (SLA) y los requerimientos no funcionales establecieron estándares claros y medibles de calidad para el negocio, validándose de manera óptima que el backend bajo el perfil de producción (prod) procesa solicitudes críticas de seguridad (autenticación JWT) en un promedio de 323.83 ms, cumpliendo holgadamente con el límite de latencia general establecido de  $\leq 2$  segundos.
- El prototipado de alta fidelidad desarrollado en Figma bajo un diseño Cyber-Minimalista (#1a1a1a) con acentos institucionales, garantizó que interfaces como el Mapa de Mesas y la comanda digital táctil respondieran a los principios de usabilidad de Nielsen, asegurando una curva de adopción acelerada del personal con miras a alcanzar la meta del 100% de uso activo en la tercera semana post-implementación.
- La estrategia de ingeniería de la plataforma —basada en la contenerización con Docker en un entorno Alpine Linux y el uso de Nginx como proxy inverso— demostró una altísima eficiencia técnica en el servidor VPS Contabo, manteniendo el servidor web un estado operativo estable (*Active: active (running)*) con un consumo de memoria RAM sumamente bajo de apenas 4.1 MB y una precarga eficiente en la capa de datos de PostgreSQL 16 sin registrar saturación inicial.
- La automatización del despliegue mediante un pipeline de Integración Continua (CI) en GitHub Actions usando la rama principal main y conexiones SSH enmascaradas con GitHub Secrets, eliminó los riesgos de error humano por configuraciones manuales. Asimismo, el análisis semántico de la infraestructura permitió identificar cuellos de botella preventivos (como el crecimiento de logs de contenedores y la falta de automatización de backups en AWS S3) que ya cuentan con un plan de mitigación técnica a corto plazo.

